Institut Supérieur d'Informatique et de Mathématiques de Monastir

Cours :Architecture des Systèmes d'Entreprise (ESA)

cfuebxri

Chapitre1: Introduction à l' Architecture d'un Système d'Entreprise

Enseignante: Dr.Aljia BOUZIDI
aljia.bouzidi95@gmail.com

3ème Licence en Sciences d'Informatique
Année Universitaire :2025-2026

Objectifs

- Appréhender un Système d'entreprise à haut niveau, dans sa globalité
- Comprendre le concepts de l'architecture d'un système d'entreprise et l'architecture logicielle
- Comprendre le cycle de vie d'un logiciel
- Comprendre le cycle de développement logiciel (SDLC)

Contenu du chapitre1

1. **Partie 1:** Le système d'Entreprise
2. **Partie 2 :** L'architecture d'un Système d'Entreprise
3. **Partie 3 :** Mise en œuvre d'un Système d'Entreprise
4. **Partie 4 :** Les Activités de Développement Logiciel SDLC
5. **Annexes**



Partie 1: Le Système d'Entreprise

Contenu de la Partie 1

1. Analyse Systémique de l'Entreprise
2. Les systèmes d'Entreprise
3. Les Sous Systèmes de l'Entreprise
4. Enjeux de l'Informatisation pour l'Entreprise
5. Système d'Information vs. Système Informatique
6. Exemples de Parties d'un SI

Analyse Systémique de l'Entreprise(1/2)

- **Apparue dans les années 1970 :**
- **Entreprise = Système**
  - L'entreprise est alors considérée comme un **ensemble d'éléments** (des moyens humains, matériels, financiers et techniques) **en interrelations**
  - Toute organisation humaine (l'État, une famille, …) peut être perçue comme **un système**

« Ensemble d'éléments en **interaction dynamique**, organisé en fonction d'un **but** »



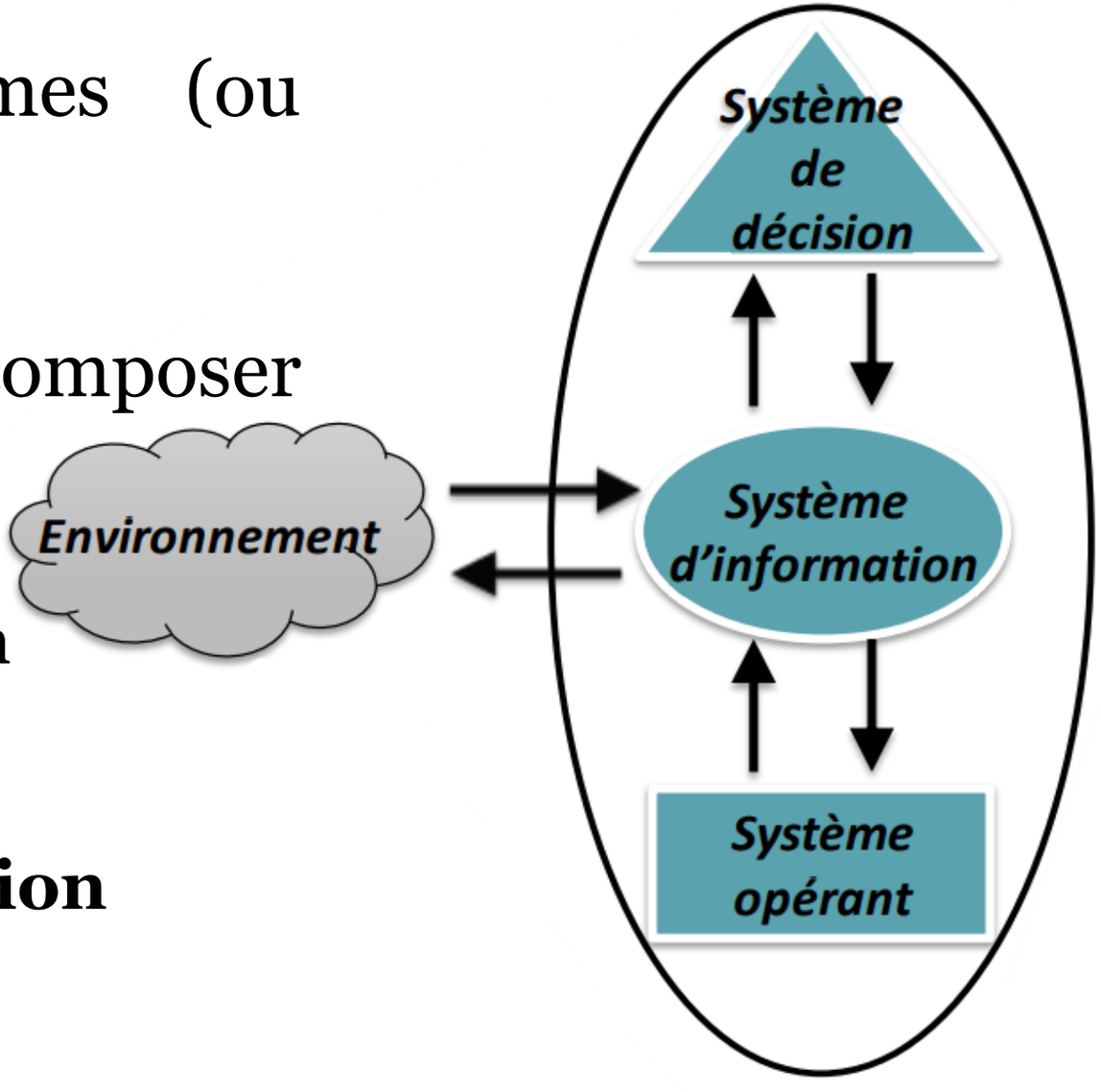
Analyse Systémique de l'Entreprise(2/2)

- Un système d'entreprise (SE) est un ensemble des ressources (personnel, matériel, logiciel, etc.) participant au stockage, classification, traitement, et diffusion de l'information au sein d'un entreprise)

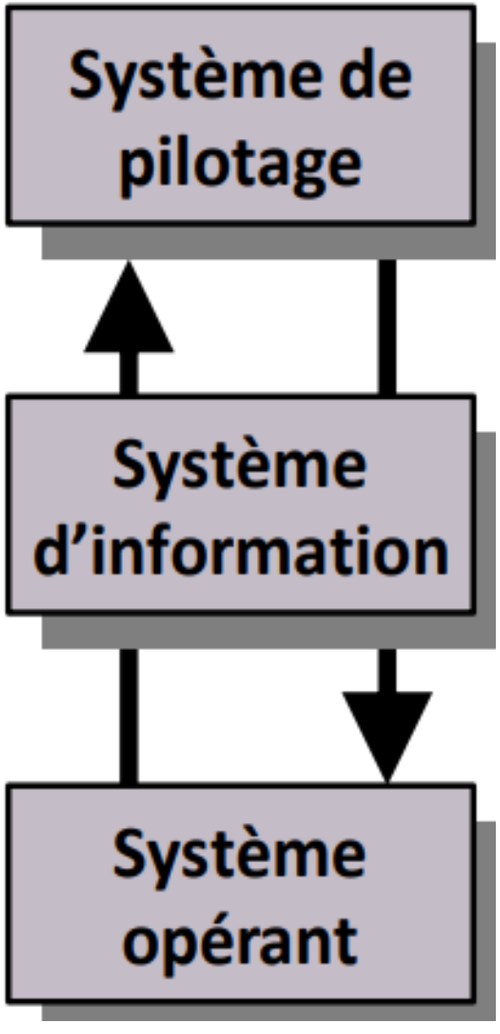


Les Sous Systèmes de l'Entreprise(1/3)

- Les éléments du système sont eux-mêmes des systèmes (ou sous-systèmes)
- L'entreprise peut se décomposer en **3 sous-systèmes** :
  - Le **système de décision** ou de **pilotage**
  - Le **système d'information**
  - Le **système opérant**



Les Sous Systèmes de l'Entreprise(2/3)



Activité :

- **réfléchir** : adaptation à l'environnement, conception
- **décider** : prévisions, allocation, planification
- **contrôler** : qualité



Activité :

- générer des informations
- mémoriser
- diffuser
- traiter



- **Notes:** Un même employé peut être un acteur de chacun des trois sous-systèmes

Activité :

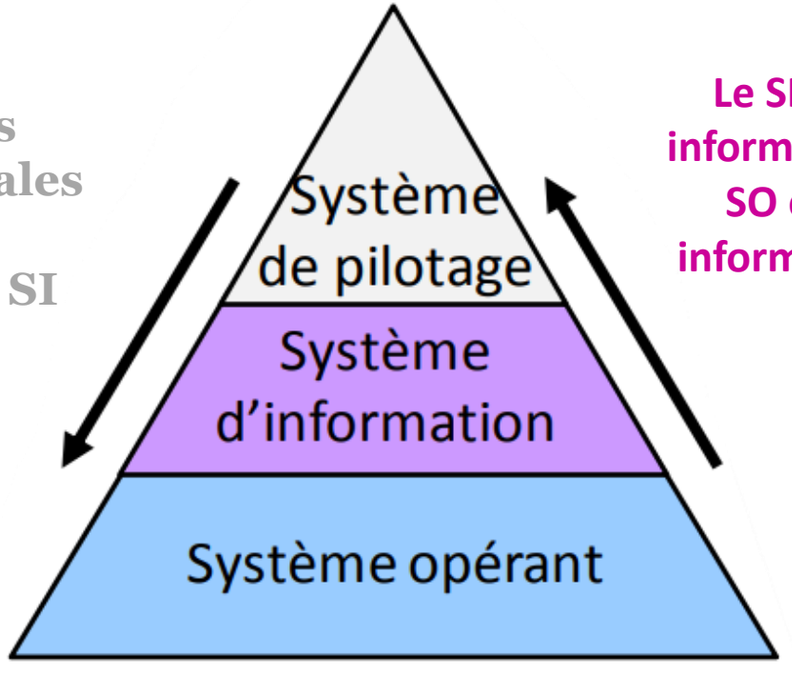
- Transformer
- Produire



Les Sous Systèmes de l'Entreprise(3/3)

- Un outil de **communication interne** :

Le SP transmet des informations globales au SO par l'intermédiaire du SI



Le SI collecte et traite les informations élémentaires du SO et les transforme en informations élaborées pour le SP

Le SO produit l'information élémentaire, exécute les ordres du SP

Enjeux de l'Informatisation pour l'Entreprise(1/2)

- Toute entreprise possède un SI



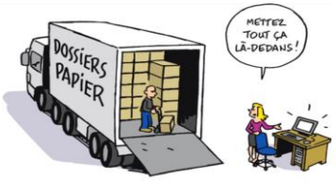
- **Pourquoi informatiser le SI ?**
  - **Augmenter la productivité** en améliorant l'efficacité des utilisateurs



  - **Améliorer les conditions** de travail : enrichissement des tâches



  - **Rendre un meilleur service** (de qualité, rapide, etc.) aux partenaires de l'entreprise



Enjeux de l'Informatisation pour l'Entreprise(2/2)

- **Aide à la décision (Business intelligence)**
  - Remonter les informations utiles et synthétiques de l'entreprise, accompagnées de prévisions et analyses afin d'aider les décisionnaires à choisir les bonnes stratégies
  - Planification stratégique
  - Planification managériale

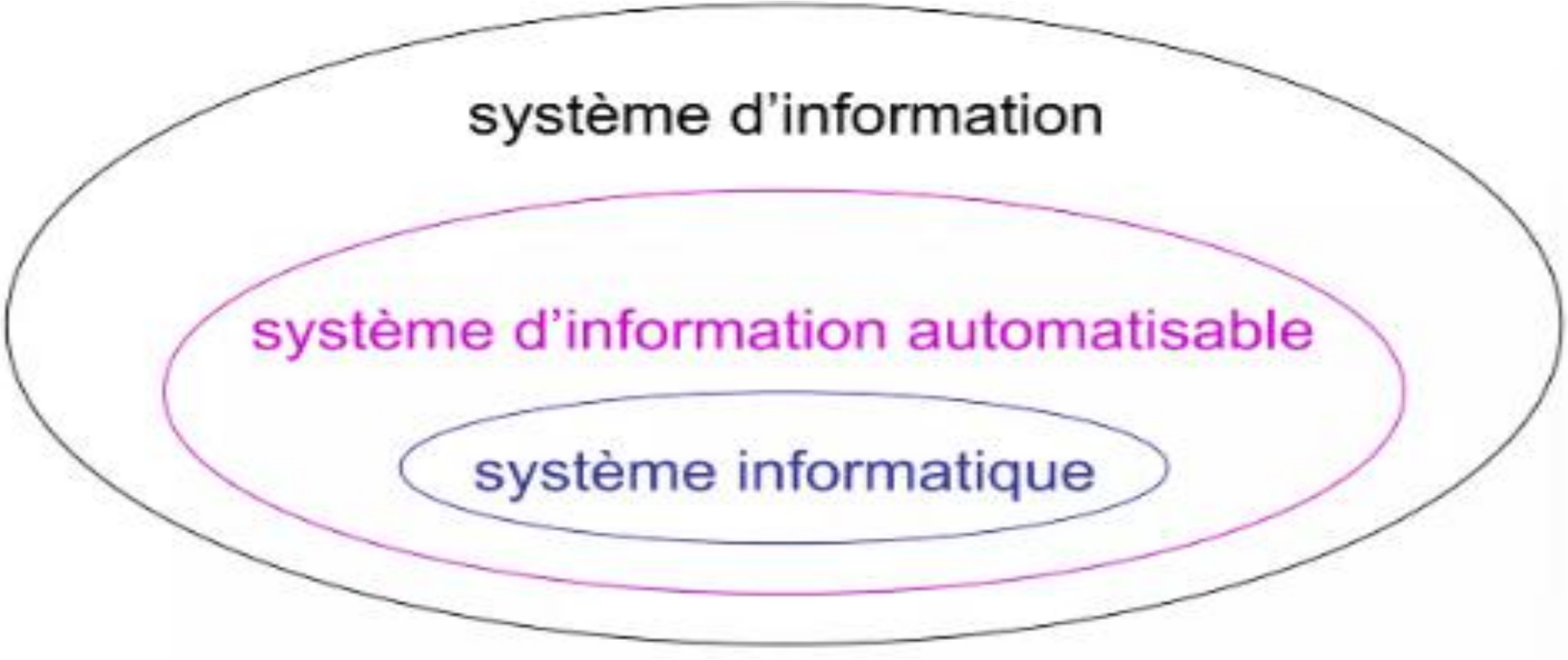


- **Fidélisation de la clientèle**, …

L'informatisation exige le développement des logiciels représentant des parties du SI

Système d'Information vs. Système Informatique

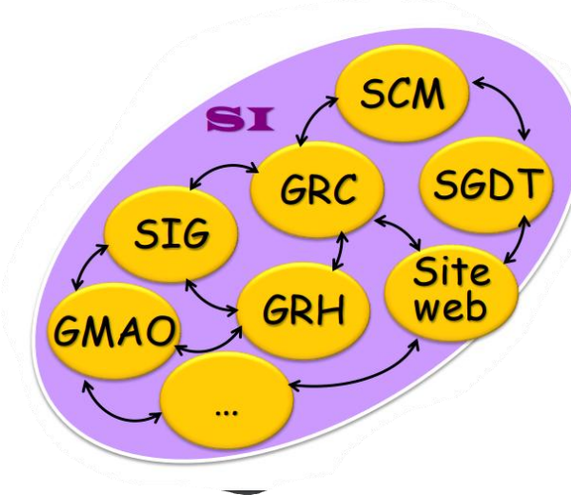
- Le système informatique est la partie informatisée du système d'information automatisable



Exemples de Parties d'un SI

- **Classique**
  - Comptabilité, paie, facturation

- **Gestion commerciale**
  - **GRC** : **G**estion de la **R**elation **C**lient (**CRM** : **C**ustomer **R**elationship **M**anagement)
  - **SIM** : Système d'information Marketing
    - Collecter et traiter données pour piloter le marketing



- **G**estion des **R**essources **H**umaines (**GRH**)
  - Suivre les carrières, compétences, formations, salaires, congés, … des personnels

Pour **automatiser** ces parties, l'entreprise doit concevoir un des logiciels en se fondant sur une **méthodologie**.



Partie 2: Architecture d'un Système d'Entreprise

Contenu de la Partie 2

1. Architecture des Systèmes d'entreprise
2. Architecture logicielle
3. Conception Architecturale

Architecture des Systèmes d'entreprise (1/3)

- **Architecture d'un système d'entreprise (ESA)**

  :

  - se concentre sur la **structure globale** et le **fonctionnement de l'organisation** en intégrant des éléments **stratégiques, métiers et technologiques**.
  - Son objectif est d'**aligner** les **systèmes informatiques** sur les **objectifs de l'entreprise**, en optimisant les ressources, les processus et les technologies.



Architecture des Systèmes d'entreprise (2/3)

- L'adoption d'une architecture des systèmes d'entreprise efficace incluent:
  - **Analyse de l'architecture** : aide à effectuer l'analyse du système au niveau architectural. Cela aide à soutenir le rpocessus de conception du système
  - **Compréhension de l'entreprise/ du système**: offre une base concrète pour comprendre efficacement les opérations commerciales d'une entreprise, ce qui se traduit paru une meilleure gestion de l'entreprise
  - **Business/ System palaning**: offre un outil utile pour planifier de nombreuses activités commerciales, des orientations stratégiques à l'amélioration locale



Architecture des Systèmes d'entreprise (3/3)

- **Restructuration et intégration de système :** aide à rendre la restructuration et l'intégration du système possibles chaque fois d'un changement dans les opérations commerciales se produit dans l'entreprise
- **Évolution du système:** offre les bases nécessaires pour évaluer les résultats des transformations majeures d'une entreprise, comme la modernisation (migration) et la décommissionnement, et l'ajout des nouveaux services



Architecture logicielle (1/2)

- **Logiciel** :
  - est ensemble de **séquence d'instructions** interprétables par une machine et d'un jeu de données
    - ✓ Séquence d'instructions = Programmes
    - ✓ Données =Fichiers ou Bases de données
- **L'architecture logicielle :**
  - c'est une infrastructure composée de composants actifs, d'un mécanisme d'intéraction entre ces composants et d'un ensemble de règles qui gouvernent cette interaction
  - elle **concentre** sur **la structure d'un système** ou d'une **application logicielle** spécifique. Elle concerne **l'organisation interne des composants** logiciels, les **interactions entre** eux, et la manière dont les **exigences** fonctionnelles et non fonctionnelles (performance, sécurité, etc.) sont satisfaites.



Architecture logicielle (2/2)

- Structure d'un logiciel en termes de composants consiste à:
  - Définir les fonctionnalités associées aux composants
  - Définir les interfaces entre les composants
  - Déterminer les dépendances entre les composants
  - Décomposer les composants en sous composants



Conception Architecturale (1/2)

- **Définir la structure du système :**
  - **Découper** le système en **composants** ou **modules**.
  - Déterminer c**omment** ces **composants interagissent** entre eux (interfaces, API).
  - **Identifier** les **relations** hiérarchiques, les **dépendances**, et les **communications** entre les **parties du système**.

- **Choisir des patrons d'architecture :**
  - Utiliser des modèles ou **motifs d'architecture** reconnus, comme l'architecture en couches, les microservices, ou MVC (Model-View-Controller), pour structurer le système de manière cohérente.



Conception Architecturale (2/2)

- **Prendre en compte les contraintes non fonctionnelles** :
  - Intégrer des aspects comme la performance, la sécurité, la scalabilité, la maintenabilité, et la résilience dès les premières étapes de conception.

- **Sélectionner les technologies appropriées** :
  - Choisir les langages de programmation, les bases de données, les frameworks, et les outils de développement en fonction des exigences du projet.
- **Conception architecturale = Proposer une architecture logicielle**





Partie 3: Mise en œuvre d'un Système d'Entreprise

Contenu de la Partie 3

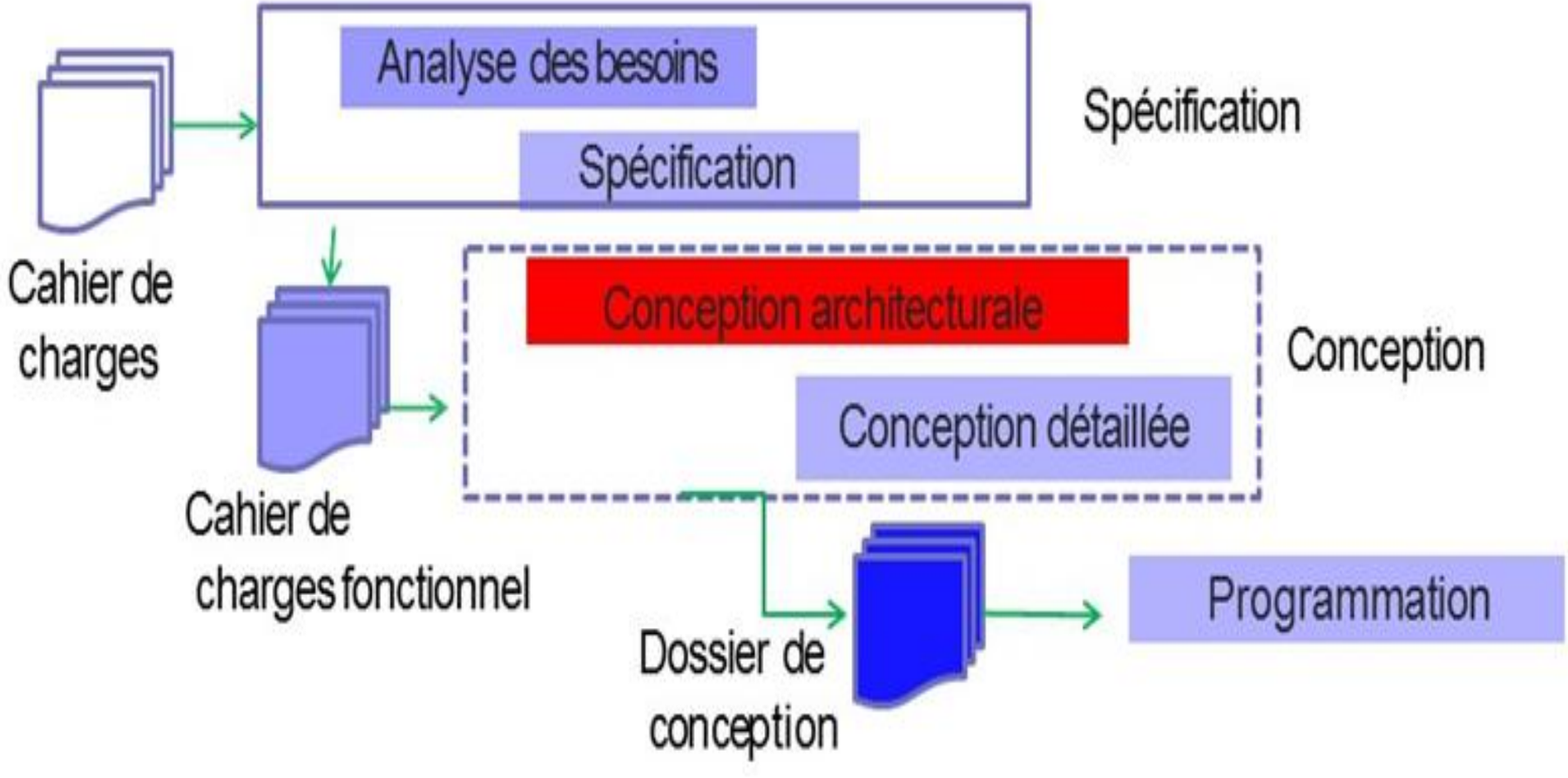
1. Cycle de vie d'un système d'Entreprise
2. Étape de mise en place d'un système d'Entreprise
3. Différentes Approches de Mise en Œuvre d'un Système d'Information
4. Méthode d'Informatisation
5. Taxinomie des méthodes
6. Développement d'un Système d'Information Robuste
7. Conduite de Projet
8. Méthodologies Classiques
9. Les Processus Unifiés
10. Les Méthodes Agiles

Cycle de vie d'un système d'Entreprise

- Le cycle de vie d'un logiciel est modélisation conventionnelle de la succession d'étapes permettant la mise en œuvre d'un produit logiciel
  - Ces étapes sont basées sut une **philosophie d'analyse détaillée** du travail nécessaire pour faire progresser un système d'entreprise **depuis la perception du besoin** du système **jusqu'à la livraison** du produit final



Étape de mise en place d'un système d'Entreprise (1/2)





Étape de mise en place d'un système d'Entreprise (2/2)

Expression des besoins	• Définition d'un cahier des charges • Interviews avec l'acteur métier
Spécification	• Ce que le système doit être et comment il va être utilisé • Formalisation du cahier des charges et des contraintes
Analyse	• Définir les éléments intervenants dans le système, leurs structures et leurs relations selon 3 vues: comportementale (fonctionnelle), statique et dynamique
Conception	• Conception générale: élaboration de l'architecture générale • Conception détaillée: définir avec précision chaque module
Réalisation	• Implémentation du système (codage)
Tests & Validation	• Contrôle de qualité • Vérifier la conformité avec le cahier des charges
Livraison & Maintenance	• Action correctives: corriger les erreurs • Action évolutives: intégrer des chargements





Différentes Approches de Mise en Œuvre d'un Système d'Information (1/4)

- Le **D**éveloppement d'**A**pplication de **G**estion (**DAG**)
- **S**ystèmes d' **A**pplication **O**rganisationnels (**SAO**)
- Les **S**ystèmes d'**I**nformation **S**tratégiques (**SIS**)

Différentes Approches de Mise en Œuvre d'un SI (2/4) : Le Développement d'Application de Gestion (DAG)

- Le **D**éveloppement d'**A**pplication de **G**estion (**DAG**) se fait :
  - Par reprises des procédures existantes.
  - Mise en œuvre d'une logique d'automatisation.
  - Production d'états type standardisés.

- **Objectifs** :
  - Automatisation.
  - Contrôle des coûts.

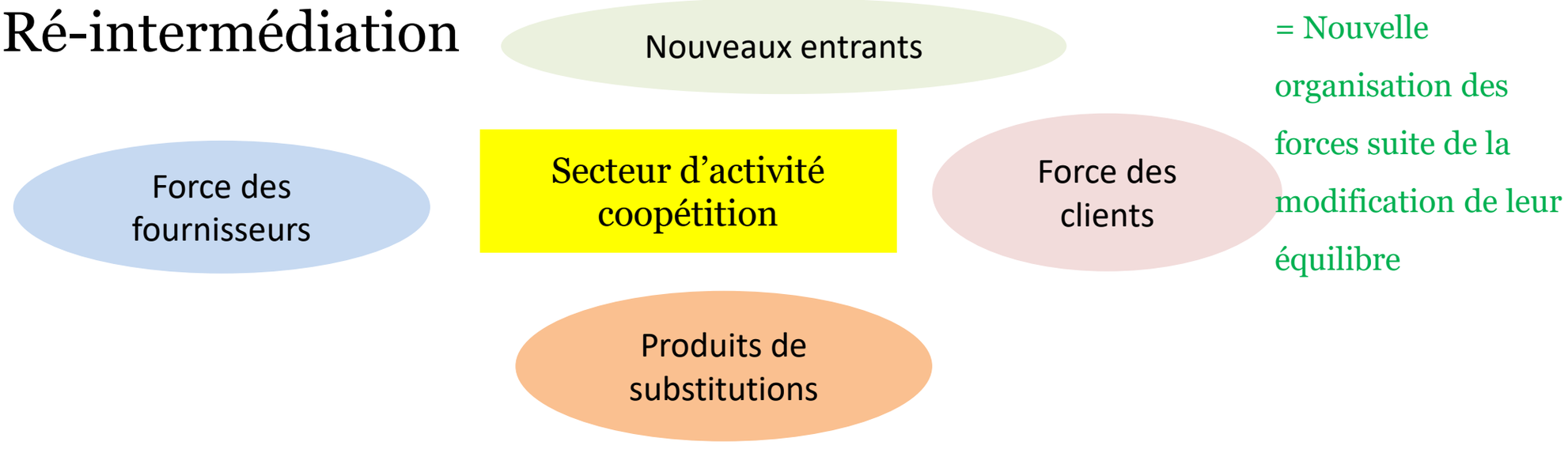
- **Caractéristiques** :
  - Traitement des données, information qualitative.
  - Le destinataire de l'information est une machine.
  - Automatisation dénouée d'intelligence.
  - Pauvreté sémantique des données

Différentes Approches de Mise en Œuvre d'SI(3/4) : Les Systèmes d' Application Organisationnels (SAO)

- Le **S**ystème d'**A**pplication **O**rganisationnel (**SAO)** est un système de représentation et de mesure des activités de l'organisation
- **Objectifs** :
  - Aide à la décision
- **Caractéristiques** :
  - L'ensemble de l'organisation et son environnement
  - Informations et représentation riches destinées à l'humain
  - Richesse sémantique des données

Différentes Approches de Mise en Œuvre d'un SI(4/4) : Les Systèmes d'Information Stratégiques (SIS)

- La vision SIS:
  - Le système d'information devient un avantage concurrentiel
  - Il est conçu suivant des Facteurs clés identifiés :
    - Il procure un avantage pour le client
    - Il permet de différencier l'offre par rapport à la concurrence
    - Il augmente le pouvoir de négociation
    - Il modifie les barrières d'entrée (protection contre les nouveaux entrants)
    - Il est un catalyseur de l'innovation
- Ré-intermédiation



= Nouvelle organisation des forces suite de la modification de leur équilibre

➡ **Le SI doit servir à déterminer comment atteindre une position favorable dans le nouvel équilibre**

Méthode d'Informatisation: (1/3)
Définition

Une méthode d'informatisation :

- Définit un processus d'informatisation du système d'entreprise (totalement ou partiellement (i.e pour tout ou partie du cycle de vie du logiciel)
- Possède une portée (champ d'étude i.e domaine étudié)
- Décrit une démarche i.e un ensemble de travaux en les ordonnant (succession d'étapes)

Méthode d'Informatisation :(2/3)

Règles

- S'appuyer sur des concepts théoriques: définition des concepts
- Proposer une démarche: cadre général pour définir le travail à accomplir par les intervenants
- Permettre sa mise en œuvre par des outils: Pour faciliter la manipulation des concepts
- Atteindre un but: l'informatisation éventuelle (=>augmentation et faisabilité)

Une méthode ne remplace pas ni l'expérience, ni la connaissance, ni le talent

Méthode d'Informatisation : (3/3)

Composants

- **Modèles** : ensemble de concepts et des règles destiné à expliquer et construire la représentation de phénomènes organisationnels
- **Langages**: destinés à l'élaboration des spécifications, à faciliter la communication
- **Démarche**
- **Outils et techniques**: aident à la mise en œuvre des modèles, langages, démarche

Taxinomie des méthodes(1/3): Fondements théoriques

- **Cartésienne (démarche dite analytique; résolution des problèmes un à un):**
  - Approche fonctionnels (analyse et conception des systèmes d'entreprise par rapport à la définition des besoins et descendante (du général au particulier)
  - Exemple: SADT, CORIG
- **Systémique (démarche dite globalisante; résolution globale des problèmes):**
  - Approche conceptuelle (processus de modélisation par niveau d'abstraction successifs)
  - Repose sur l'identification de projets qui structurent l'organisation (sans qu'il y ait obligatoirement un besoin)
  - Exemple, MERISE, AXIAL, IA-NIAM

Taxinomie des méthodes(2/3): Fondements théoriques (suite)

- **A objet (application du paradigme objet à tout le processus):**
  - Les objets (de l'application, de service distribués) et les utilitaires communs échangent des informations (démarches et réponses de services) à l'aide de messages
  - Exemple: OOA, OMT, MCO, HOOD, OOSE, Merise Objet
- **Formelle (utilisation des mathématiques):**
  - Spécification et conception formelles exprimées à l'aide du langage mathématique qu'il faut ensuite prouver
  - Exemple: B, Z

Taxinomie des méthodes(3/3): Démarche

○ Linéaire

- Succession linéaire des travaux (démarche découpée en étapes découpées en phases découpées en taches découpées en opérations)
- **Analyse descendante** (des problèmes **généraux au problèmes particuliers**) par **décomposition hiérarchique** des travaux
- Itération et condition possibles
- Exemple: MERISE, AXIAL, etc.

○ Non linéaire:

- Analyse **ascendante** par **intégration progressive des résultats**

Développement d'un Système d'Information Robuste:
Facteurs Clés de Réussite

1) Une vision claire – présentation stratifiée
2) Un support technologique adéquat
3) Adhésion

Staff décisionnel

Staff opérationnel

Mise en place d'un SI ⟹ Conduite de Projets informatiques

Conduite de Projet(1/2)

I. Quelle démarche suivre?

- **Projet:** Atteindre d'un objectif= Résultat au problème posé

  Problématique: Quelle est la meilleure manière ou le meilleur chemin



- **Démarche/ méthodologies:**
  - **Approche de résolution du problème.** Elle est constituée par un ensemble de principes généraux qui guident le choix d'une méthode adaptée à une tâche ou projet spécifique
- **Méthode:**
  - description pas-à-pas des étapes impliquées dans l'accomplissement d'une tâche particulière
  - C'est une démarche, muni d'un ensemble d'outils permettant de résoudre un problème posé.
  - La méthode permet de statuer sur l'état d'avancement d'un projet
  - Elle ne peut être la même pour deux projets différents, la méthode est spécifique à un projet.

Conduite de Projet(2/2)

Trois principaux démarches peuvent être suivies lors du développement d'un logiciel:

Méthodologie classique	Méthodologie agile	Processus unifié
• Modèle en cascade	• Méthode scrum	• Two Tracks Unified Process
• Modèle en V	• Modèle de kanban	• eXtreme Unified process
• Modèle en spirale	• Modèle de RAD	• Enterprise Unified Process
• Modèle incrémental	• Modèle XP	• Rational Unified Process
• ….	• ….	• …

Méthodologies Classiques(1/7)

- Les **méthodologies de développement logiciel classiques** sont souvent regroupées sous le terme générique de "**SDLC**" (Software Development Life Cycle), ce qui signifie "Cycle de Vie du Développement Logiciel".

- Le **SDLC** englobe différentes approches et méthodologies utilisées pour gérer le processus de développement, de la conception initiale à la maintenance continue du logiciel.

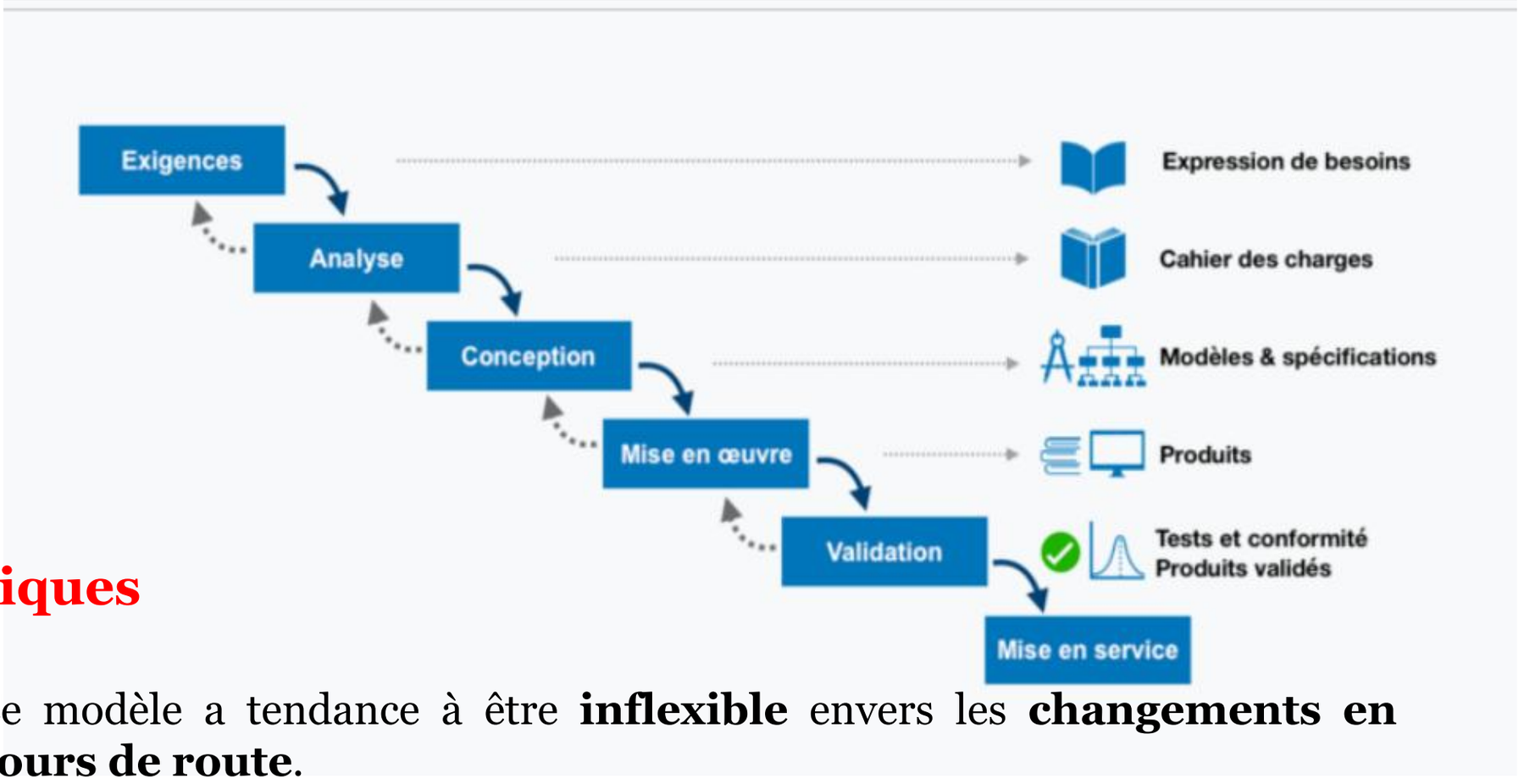
- Etaient largement utilisées avant l'émergence des méthodes agiles et d'autres approches plus modernes.

- La méthodologie classique se caractérise généralement par une **approche planifiée**, **séquentielle** et rigide du développement de logiciels.

Méthodologies Classiques(2/7):

Modèle en cascade (Waterfall)

- Le modèle en Cascade **(en chute d'eau)** suit une **séquence linéaire** d'étapes, où chaque étape doit être **complétée avant de passer à la suivante**.

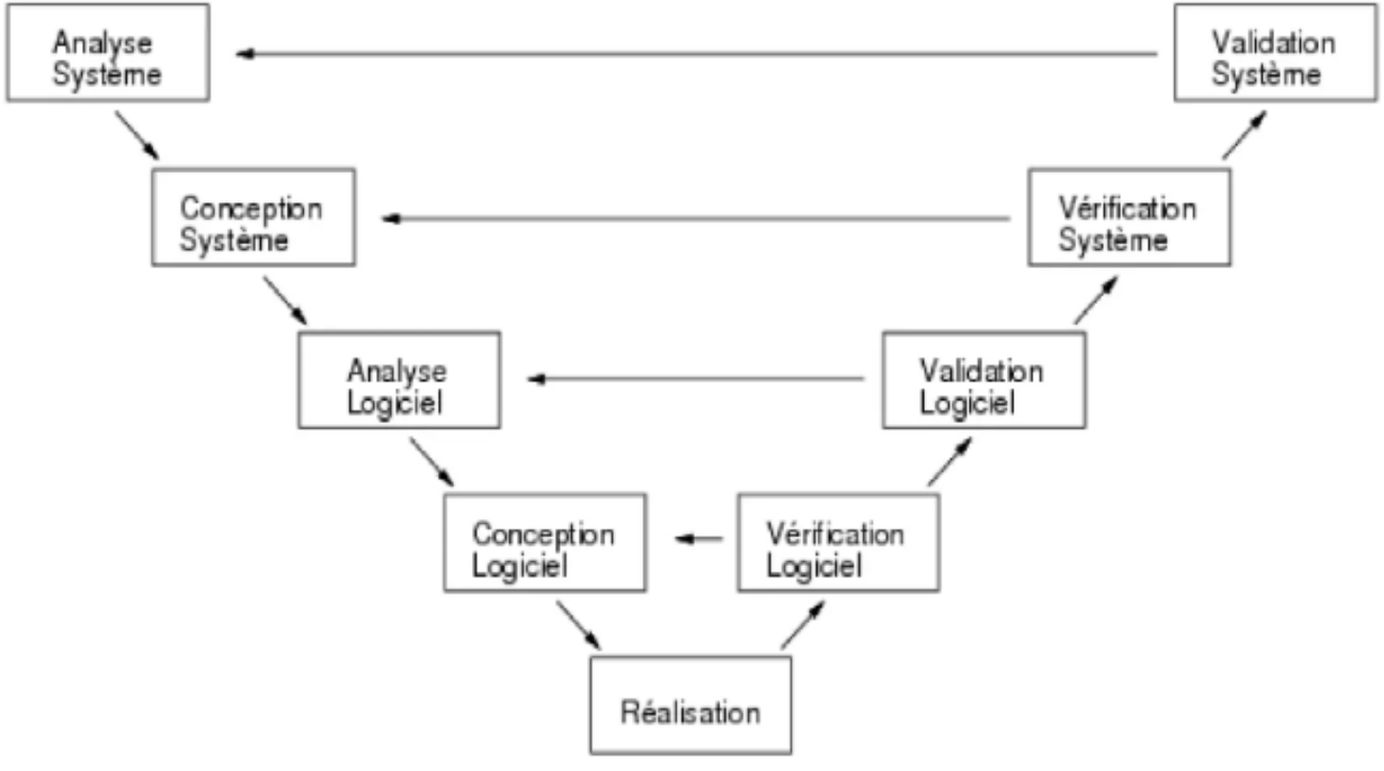


- **Critiques**
  - Ce modèle a tendance à être **inflexible** envers les **changements en cours de route**.
  - Recouvrement de phases
  - L'ordre des phases doit être respecté

Méthodologies Classiques(3/7):

Modèle en V

- Le **modèle en V** est une **extension du modèle en cascade**.
- Chaque **phase du développement** a une **phase correspondante de test**.
- Les **activités de test** sont conçues **pour valider** que chaque étape de développement a été **correctement implémentée**. Si une phase échoue aux tests, elle doit retourner en arrière pour être corrigée.

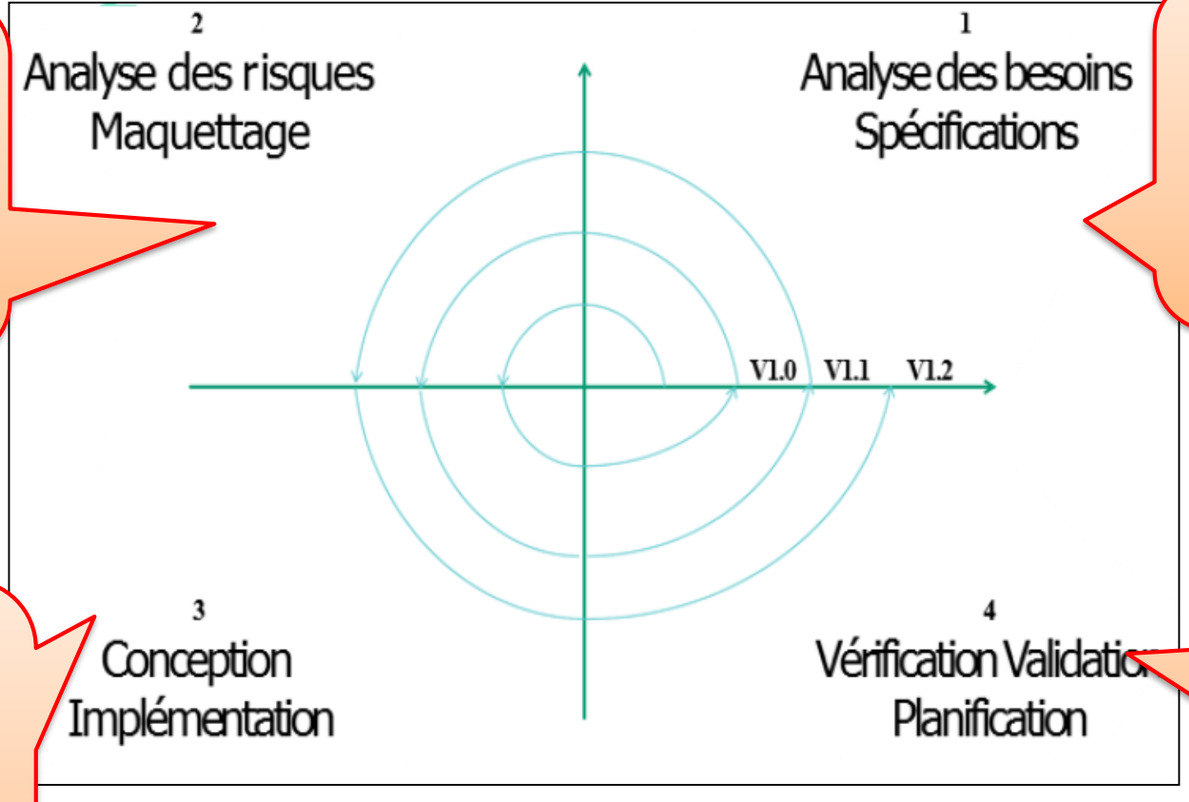


Méthodologies Classiques(4/7):

Modèle en Spirale

- Le **modèle en spirale** fait appel à plusieurs itérations faisant eux même appel à l'un des modèles classiques de cycle de vie (cascade ou V).

Analyse des risques, ***évaluation*** des alternatives et éventuellement ***maquettage***



Détermination des objectifs du cycle en cours, les alternatives pour les atteindre et les contraintes à prendre en compte

Conception et ***développement*** de la solution retenue, pour cela un modèle classique (cascade ou V) peut être utilisé;

Vérification des résultats et ***planification*** du cycle suivant;

Méthodologies Classiques(5/7):

Modèle en Spirale (suite)

- Chaque **itération** représente une **évolution** vers la **version suivante** du logiciel.

- Le modèle en spirale a été moins utilisé que les deux précédents. Son utilisation nécessite de grandes compétences et convient surtout aux **systèmes** d'entreprise **innovants** à cause de **l'importance** qu'il accorde à **l'analyse des risques**.

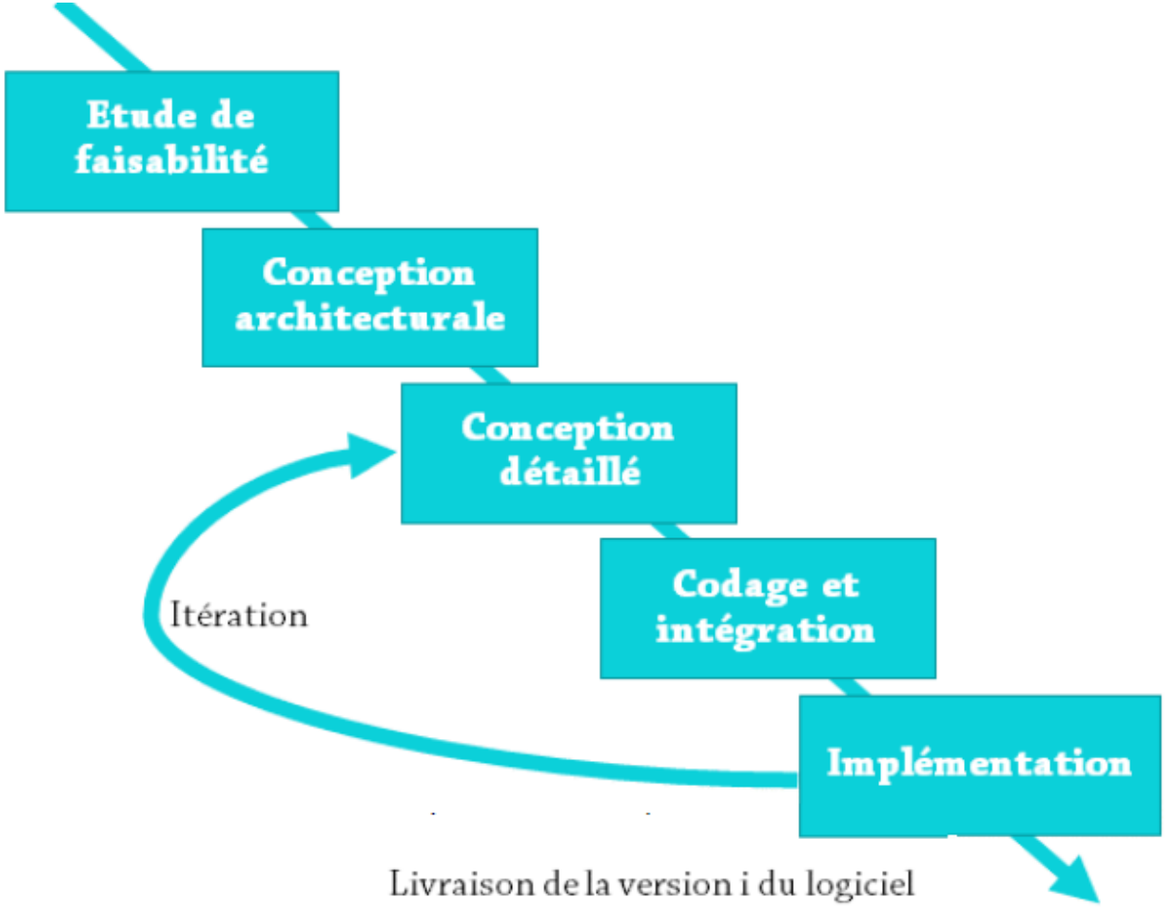
- Le modèle en spirale **diminue** considérablement **le risque d'échec** lors des systèmes-logiciels de **grande taille**

- Ce modèle est, également, générique et peut être combiné avec d'autres méthodes de développement agile ou classiques, c'est pourquoi il est, également, appelé modèle de **second ordre**

Méthodologies Classiques(6/7):

Modèle Incrémental

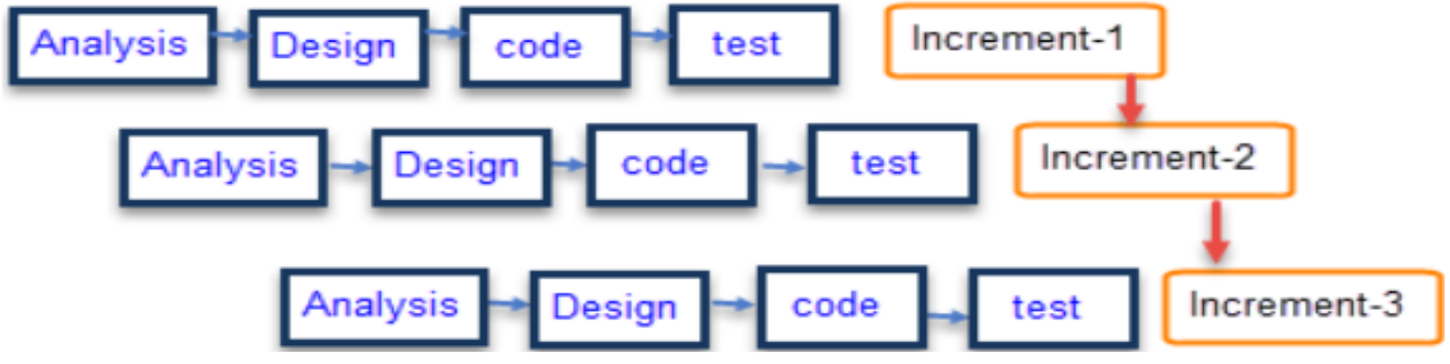
- Le **modèle incrémental** est basé sur la réalisation de sous-ensembles assurant chacun une partie fonctionnelle de la solution.



Méthodologies Classiques(7/7):

Modèle Incrémental (suite)

- **Lors** de la **définition de l'architecture** des systèmes, un **découpage** en **composants fonctionnels** permet des livraisons successives.
- Le prototype initial est itéré jusqu'à obtention du produit final.
- Le **nombre de livraisons** est **déterminé** lors de la **définition de l'architecture**.
- Le **premier composant** est celui qui **représente** les **coûts** les **plus importants**, puisque l'ensemble de l'infrastructure des systèmes d'entreprise est à mettre en place.
- Chaque **composant** est développé **selon un modèle** (planification, analyse, conception, développement, test, recette, mise en production) et vient s'**intégrer** dans la **solution déjà livrée**.

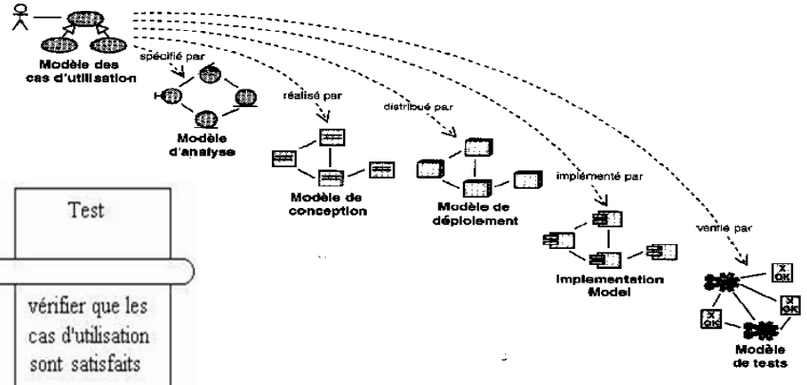


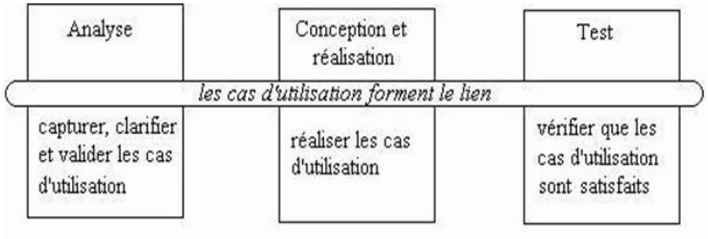
Les Processus Unifiés(1/8)

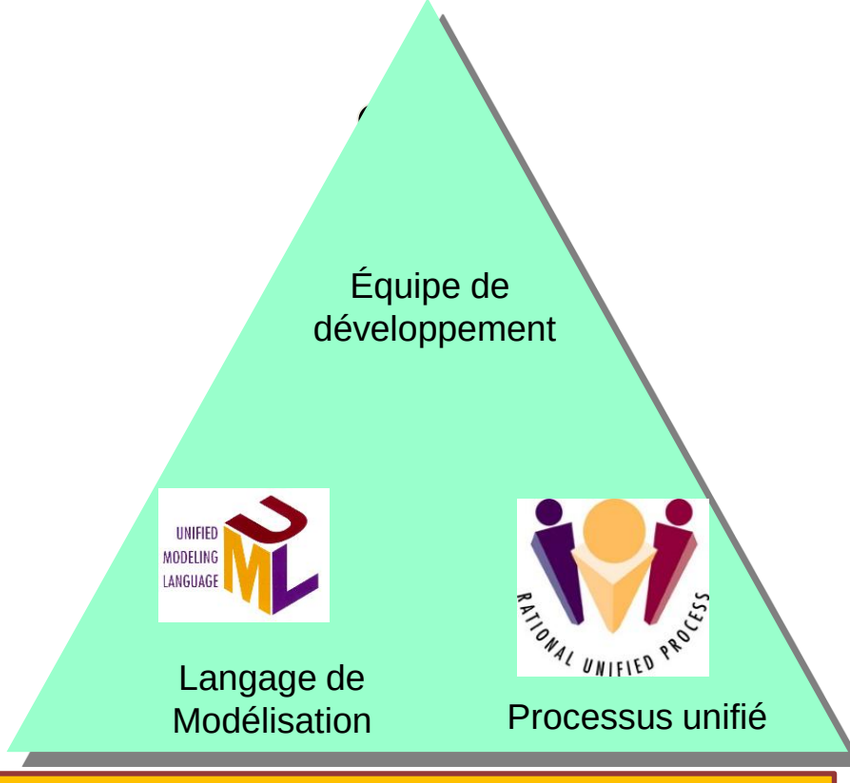
- Le processus unifié est considéré comme une démarche de développement d'application mis en œuvre par les auteurs d'UML (**Grady BOOCH, Ivar Jacobson et James RUMDAUGH**).

Principes de la démarche UP

- Est guider par les cas d'utilisation





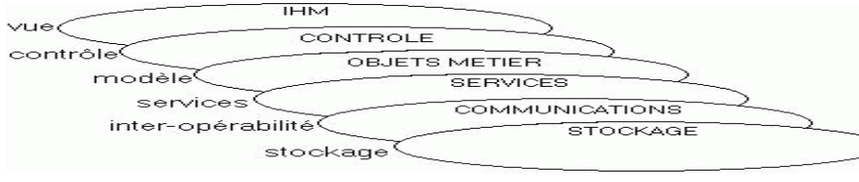


- Le développement du système peut se décomposer par cas d'utilisation.
- De même la réception du logiciel sera articulée par cas d'utilisation

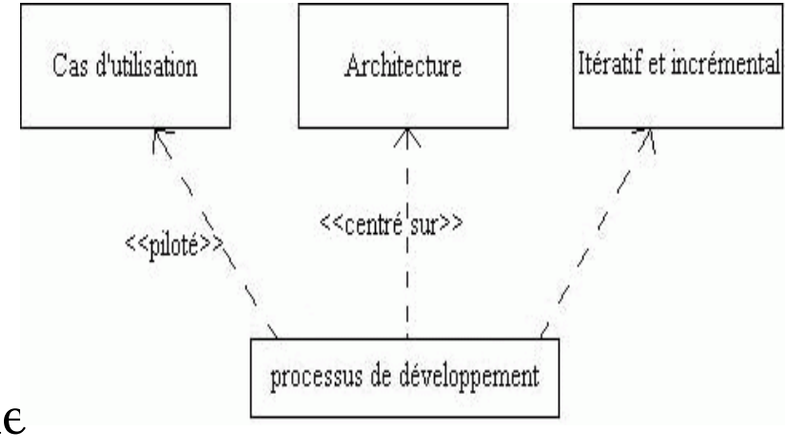
Les Processus Unifiés(2/8)

Principes de la démarche UP (suite)

- Le processus unifié est itératif et incrémental
  - UP préconise le principe de développement par itérations
  - Chaque itération constitue une brique d'un livrable
  - Le développement se fait par incrément successifs
- Le processus unifié est centré sur l'architecture
  - UP préconise de définir aussitôt que possible l'architecture type du système



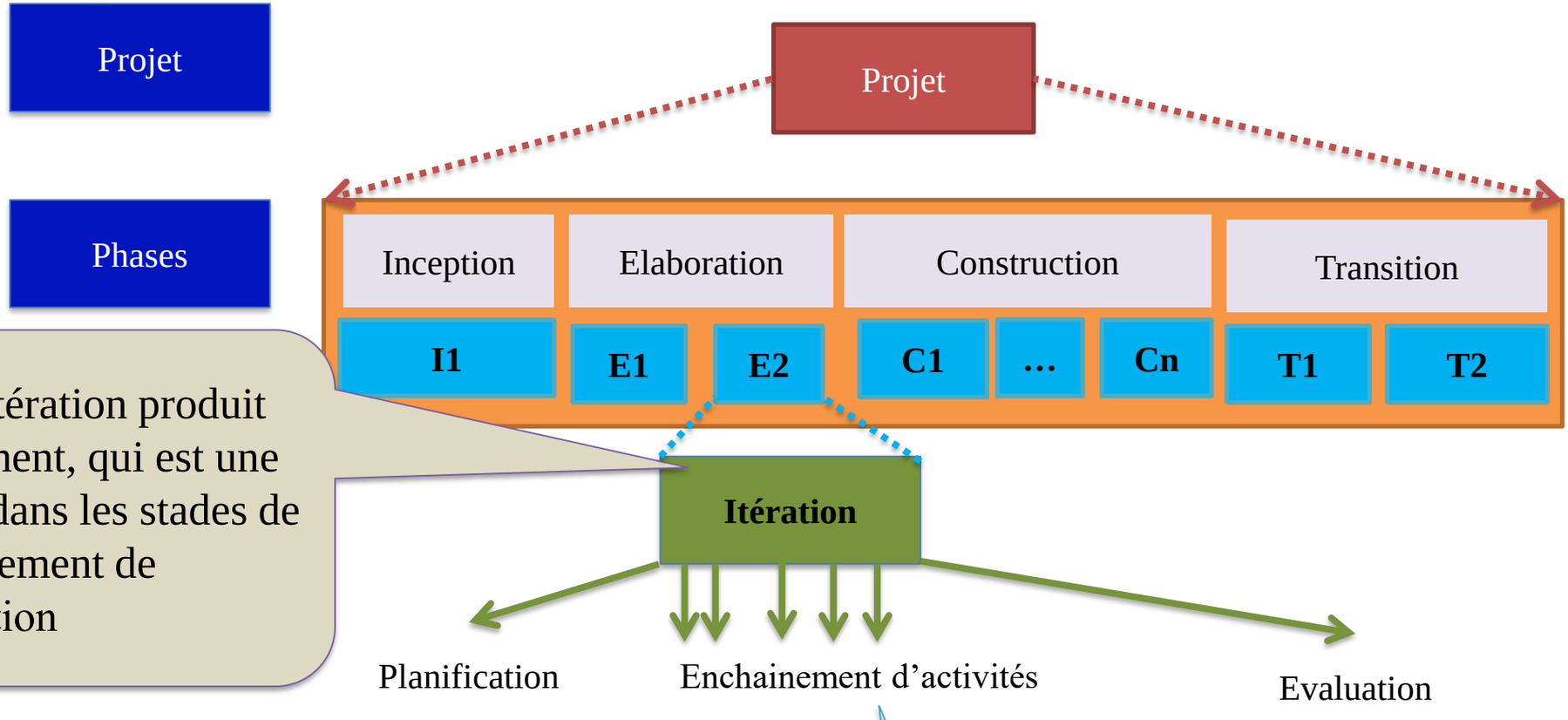
- Le processus unifié est guidé par les besoins (exigences) des utilisateurs
  - Pour servir les attentes des utilisateurs, on centre le processus de development sur leurs besoins.
  - **On fait apparaître ces besoins à l'aide de la technique des cas d'utilisation :**
    - en capturant les besoins fonctionnels d'un système
    - en orientant le travail de chaque itération
  - vont guider le processus à travers l'utilisation des différents modèles UML qui représentent le système.



Les Processus Unifiés(3/8)

Concepts et dimensions du processus unifié (suite)

- **Le concept itération :**



Les Processus Unifiés (4/8):

Rational Unified Process (RUP)

- Quelques spécifications ajoutées par le **RUP** par rapport au **PU**:

1. **Guides et pratiques détaillées** : Le RUP fournit des guides plus détaillés, des modèles et des bonnes pratiques pour chaque phase et itération du développement. Cela aide les équipes à avoir une compréhension plus claire des étapes à suivre et des tâches à accomplir à chaque étape du processus.

2. **Personnalisation facilitée** : Le RUP met davantage l'accent sur la personnalisation du processus en fonction des besoins spécifiques du projet. Il fournit des ressources pour aider les équipes à adapter le processus aux caractéristiques uniques de leur projet.

3. **Gestion des changements et des risques plus explicite** : Le RUP intègre explicitement la gestion des changements et des risques dans chaque phase et itération. Les équipes sont encouragées à identifier, évaluer et gérer les risques dès le début du projet.

Les Processus Unifiés (5/8):

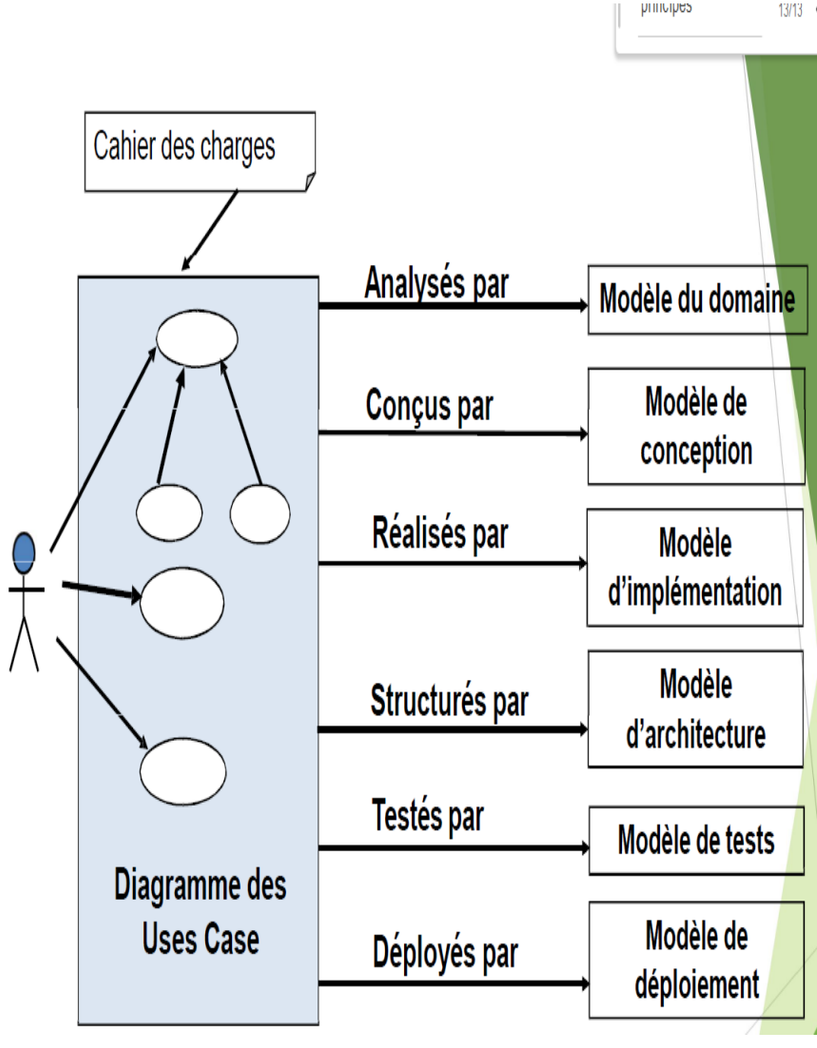
Rational Unified Process (RUP) (suite)

- Quelques spécifications ajoutées par le **RUP** par rapport au **PU**:

5. **Conception basée sur les cas d'utilisation plus approfondie** : Le RUP met un accent particulier sur l'utilisation de cas d'utilisation pour guider la conception logicielle. Il fournit des directives plus détaillées sur la manière de capturer les exigences et de les traduire en éléments concrets du système.

6. **Orientation vers l'architecture logicielle** : Le RUP place une importance particulière sur l'architecture logicielle. Il propose des pratiques et des directives pour développer et documenter une architecture solide dès le début du projet.

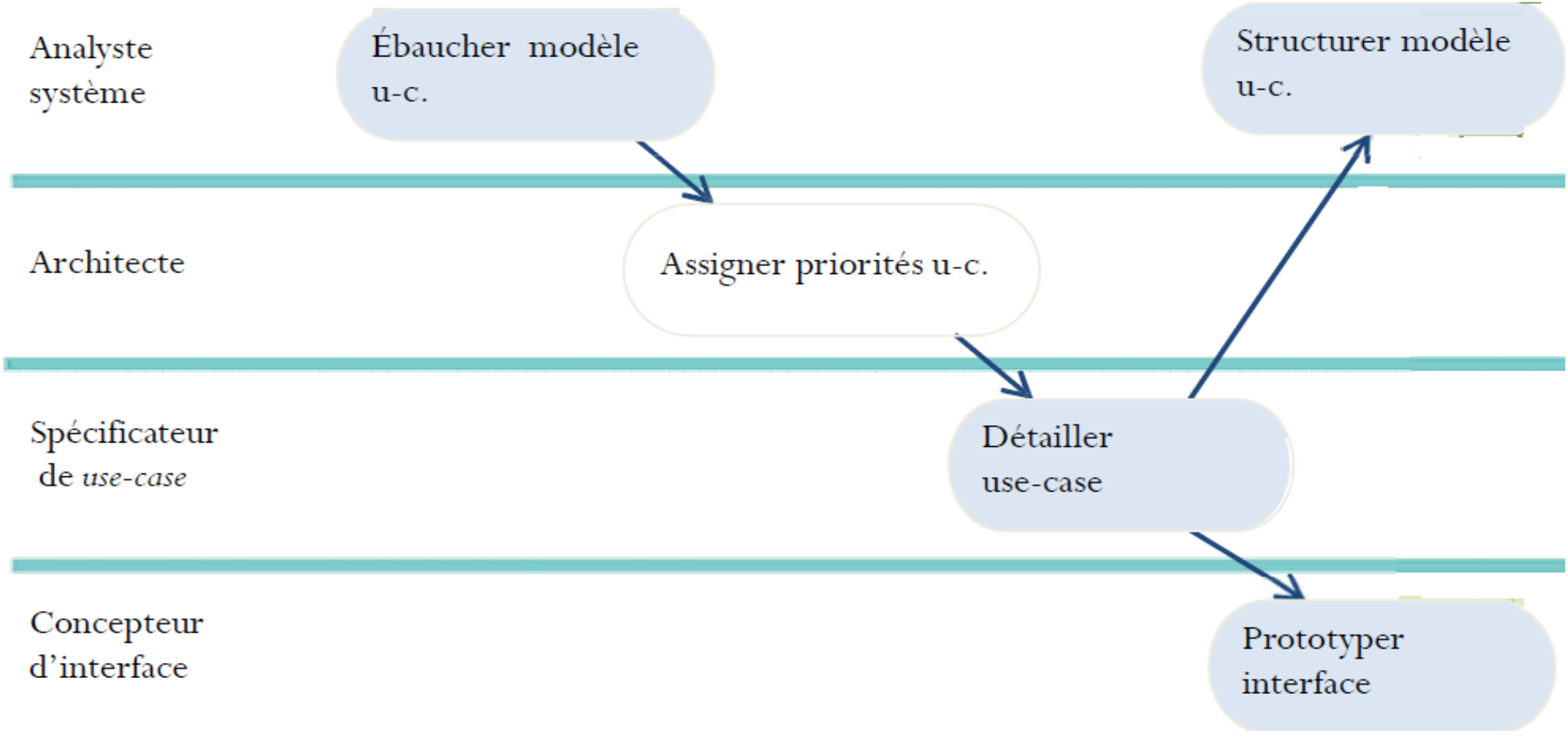
7. **Adaptation aux méthodologies agiles** : Bien que le RUP ne soit pas aussi agile que les méthodologies telles que Scrum ou XP, il intègre certaines idées agiles, comme l'itération et la collaboration étroite avec les parties prenantes.



Les Processus Unifiés (6/8):

Rational Unified Process (RUP) (suite)

Enchainement des activités de spécification des besoins

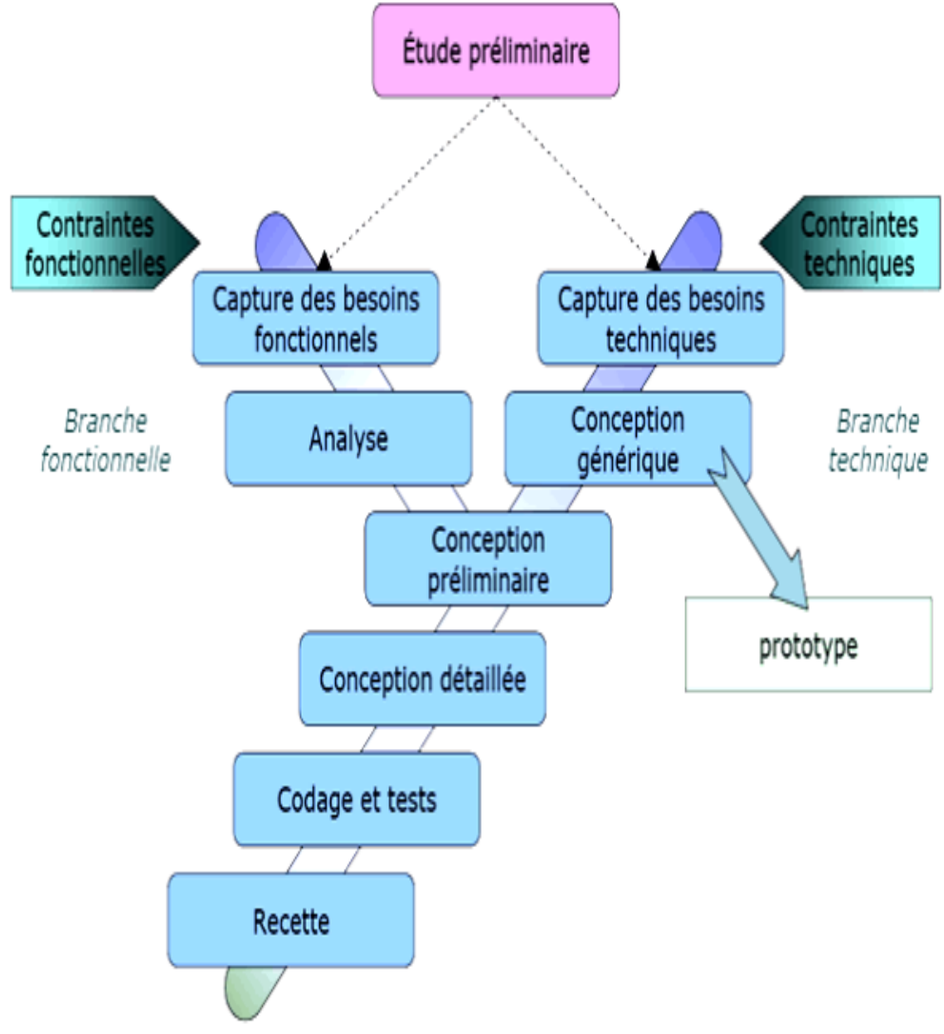


Les Processus Unifiés (7/8):

Two Tracks Unified Process (2TUP)

- Le 2TUP propose un cycle de développement en **Y,** qui dissocie les aspects techniques des aspects fonctionnels.

- Il commence par **une étude préliminaire** qui consiste à **identifier les acteurs** qui vont interagir avec le système à construire, **les messages** qu'échangent les acteurs et le système, à produire le **cahier de charge** et à modéliser le **contexte** (le système est une boite noire, les acteurs l'entourent et sont reliés à lui, sur l'axe qui lie un acteur au système on met les messages que les deux s'échangent avec le sens).



Les Processus Unifiés (8/8):

Two Tracks Unified Process (2TUP)

- Le processus s'articule ensuite autour de **trois branches:**

  - **Branche fonctionnelle :**
    - capitalise la connaissance du métier de l'entreprise.
    - Cette branche capture les besoins fonctionnels, ce qui produit un modèle focalisé sur le besoins des utilisateurs finaux

  - **Branche technique :**
    - capitalise un savoir-faire technique et/ou des contraintes techniques.
    - Les techniques développées pour le système sont indépendantes des fonctions à réaliser

  - **Branche de réalisation :**
    - Consiste à réunir les deux branches, permettant de mener une conception applicative et enfin la livraison d'une solution adaptée aux besoins.

Les Méthodes Agiles (1/10)

- Une **méthode agile** est une approche itérative et collaborative, capable de prendre en compte les besoins initiaux du client et ceux liés aux évolutions.

- **Le manifeste Agile :**

~~Processus et outils~~



Personnes et interactions

- La cohésion d'une équipe est importante que le suivi des conventions
- Ce principe insiste sur la qualité des individus impliqués et l'esprit de collaboration
- Cet élément reflète la motivation qui doit laisser au second plan la rigidité du processus ou la complexité des outils

~~Documentation exhaustive~~



Un produit opérationnel

- L'objectif d'un développement logiciel est de produire un logiciel, la documentation doit être considérée comme un support complémentaire aidant à sa compréhension
- Il est préférable d'investir ses efforts sur le produit, plutôt que sur les éléments qui le décrivent.

~~Négociation d'un contrat~~



Collaboration avec le client

- L'agilité vise la collaboration des deux parties pour atteindre un objectif commun
- Le client doit s'impliquer et diriger l'équipe dans les étapes de réalisation
- Il doit s'approprier l'évolution de son produit . L'agilité évite le gaspillage des négociations et des demandes de changement

~~Suivi d'un plan~~



Adaptation au changement

- L'agilité reconnait l'apprentissage fait au cours d'un système d'entreprise. Que tout n'est pas connu d'avance et que des adaptations sont inévitables
- Le changement est intégré au modèle de développement. La planification originale ne doit pas être trop détaillée car certains aspects ne seront probablement jamais réalisés
- Ceci évite de détailler des éléments trop tôt dans le système d'entreprise.

Les Méthodes Agiles (2/10)

- ***Les points forts des méthodes agiles***

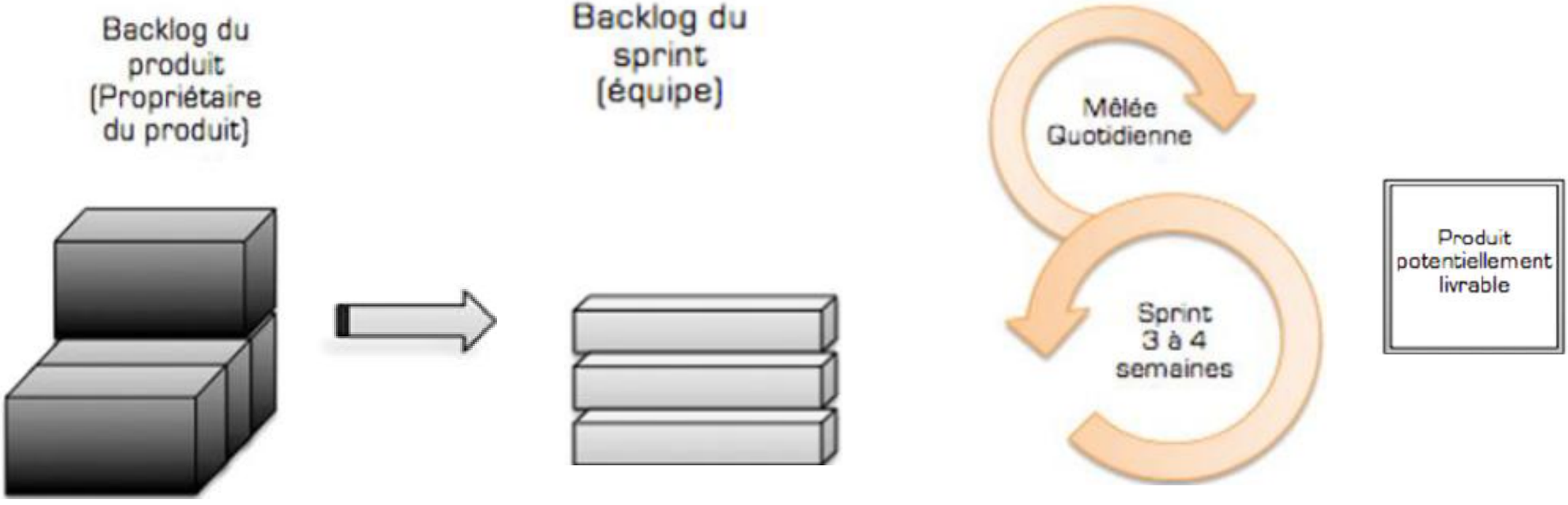
  - L'équipe fait peu de hors sujet car ces méthodes assurent une bonne et constante communication entre le client et l'entreprise;
  - La documentation est réduite, ainsi l'efficacité en terme de productivité est en augmentation;
  - La collaboration avec le client s'effectue de façon quotidienne;
  - Une version fonctionnelle du logiciel est livrée fréquemment;
  - La recherche constante de l'excellence technique (des tests sont réalisés en continu);
  - Le résultat est percevable petit à petit, ce qui permet d'éviter les mauvaises surprises;

En clair, la méthodologie Agile est une méthode moderne, favorisant un gain de productivité non négligeable, et une baisse des coûts de production;

- ***Les principales méthodes agiles*****:** Scrum, Kanban, EXtreme Programming (XP) , Rapid Application Development (RAD)...

Les Méthodes Agiles(3/10) : Scrum

- Aujourd'hui « **Scrum** » est la méthode agile la plus populaire.
- La méthode agile Scrum, s'appuie sur des « **sprints** » qui sont des intervalles de temps assez courts pouvant aller de deux semaines jusqu'à un mois.
- Généralement et de préférence un **sprint** s'étend sur **deux semaines**. À la fin de chaque sprint, l'équipe présente ce qu'elle a ajouté au produit.



Les Méthodes Agiles(4/10) : Scrum (suite)

- Scrum regroupe trois acteurs:
  - Le **product Owner** (ou « le directeur de produit ») : il communique les objectifs premiers des clients et utilisateurs finaux, coordonne l'implication des utilisateurs et des parties importantes.
  - Le **Scrum Master** : membre de l'équipe, il a pour but d'optimiser la capacité de production de l'équipe. Pour se faire, le Scrum master aide l'équipe à travailler de façon autonome tout en s'améliorant d'avantage
  - **L'équipe opérationnelle** (qui regroupe idéalement moins de dix personne): la particularité d'une équipe Scrum est qu'elle est dépourvue de toute hiérarchie interne. Une équipe Scrum est autoorganisée.



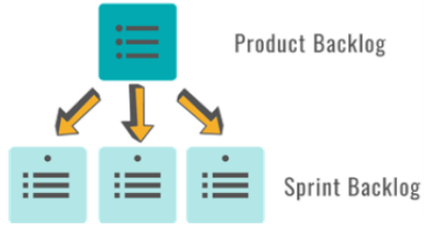


Dr. Maissa HAMOUDA | ISIMM 2022/2023



Les Méthodes Agiles(5/10) : Scrum (suite)

- Autres composants:
  - Le **product backlog** (ou « carnet du produit») : ce document conteint les exigences intiales dressés puis hiérarchisés avec le client en début de système d'entreprise. Mais il va évoluer tout au long de la durée des systèmes d'entreprise, en fonction des divers besoins du client.



  - Le **Sprint backlog (ou carnet du sprint)**: au début de chaque sprint, l'équipe définit un but. Puis lors de la runion de sprint, l'équipe de développement choisit les éléments du carnet à réaliser. L 'ensemble de ces éléements constitue alors le sprint backlog.



  - **User story (ou récit des utilisateurs) :** Ce terme désigne les fonctionnalités décrites par le client.
  - La **mêlée (Scrum)** : c'est une réunion d'avancement organisée de manière quotidienne durant le sprint

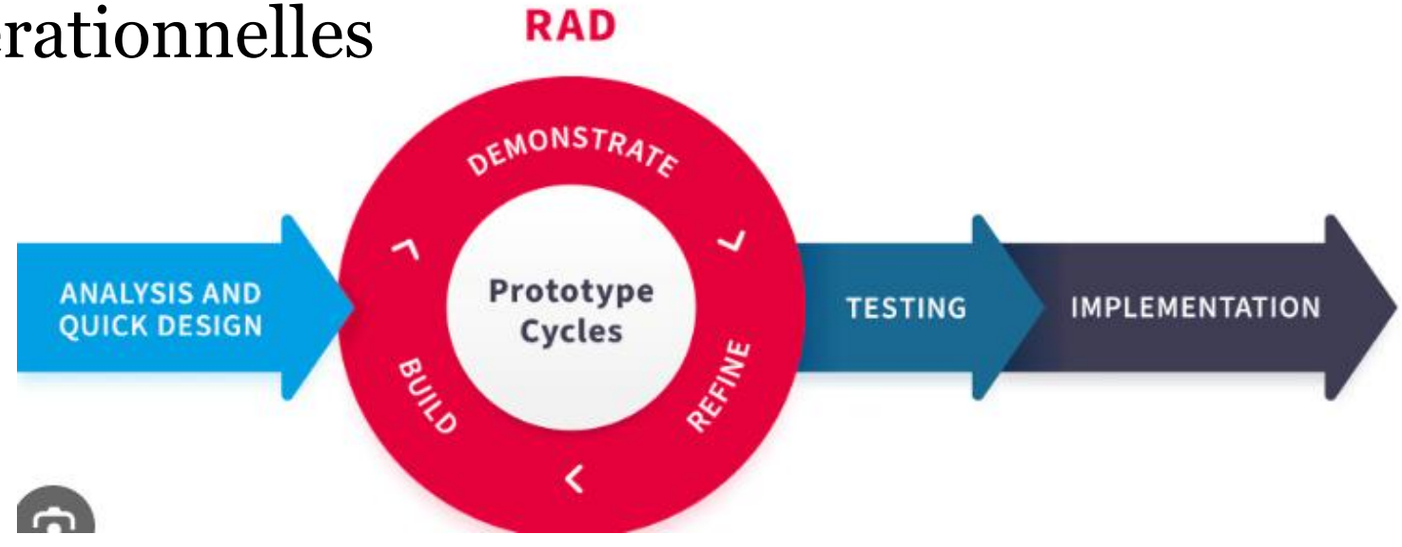


Les Méthodes Agiles (6/10):

La Méthode RAD (Rapid Application Development)

- Des raccourcis plus rapides et moins coûteux pour ce processus sont appelés **approches Rapid SDLC (méthode RAD)**.

- Le modèle RAD associe plusieurs techniques et outils afin d'optimiser les détails de développement à objectif de qualité donné. Le but est livrer rapidement un minimum de fonctions opérationnelles



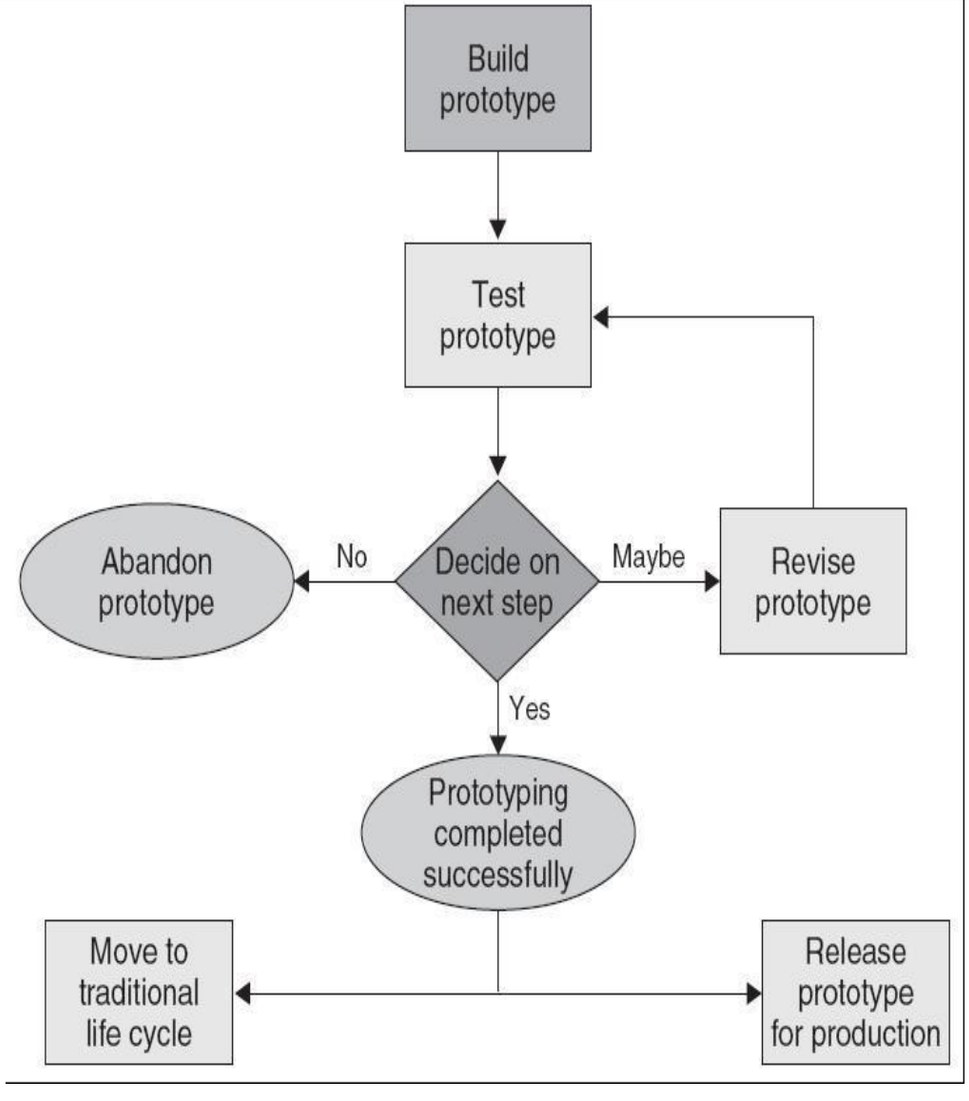
Les Méthodes Agiles (7/10):

La Méthode RAD (Rapid Application Development (suite)

1. Le **prototypage** est l'une de ces approches **RAD:**

- Cette approche ignore les phases de l'analyse et la conception; au lieu de cela, il implémente un prototype du système réel en utilisant de fausses données.

- C'est pour voir si c'est génère les résultats souhaités dans les plus brefs délais.



Les Méthodes Agiles (8/10):

La Méthode RAD (Rapid Application Development (suite)

2. Une autre approche est le **développement de l'utilisateur final**

(End-User developpement) :

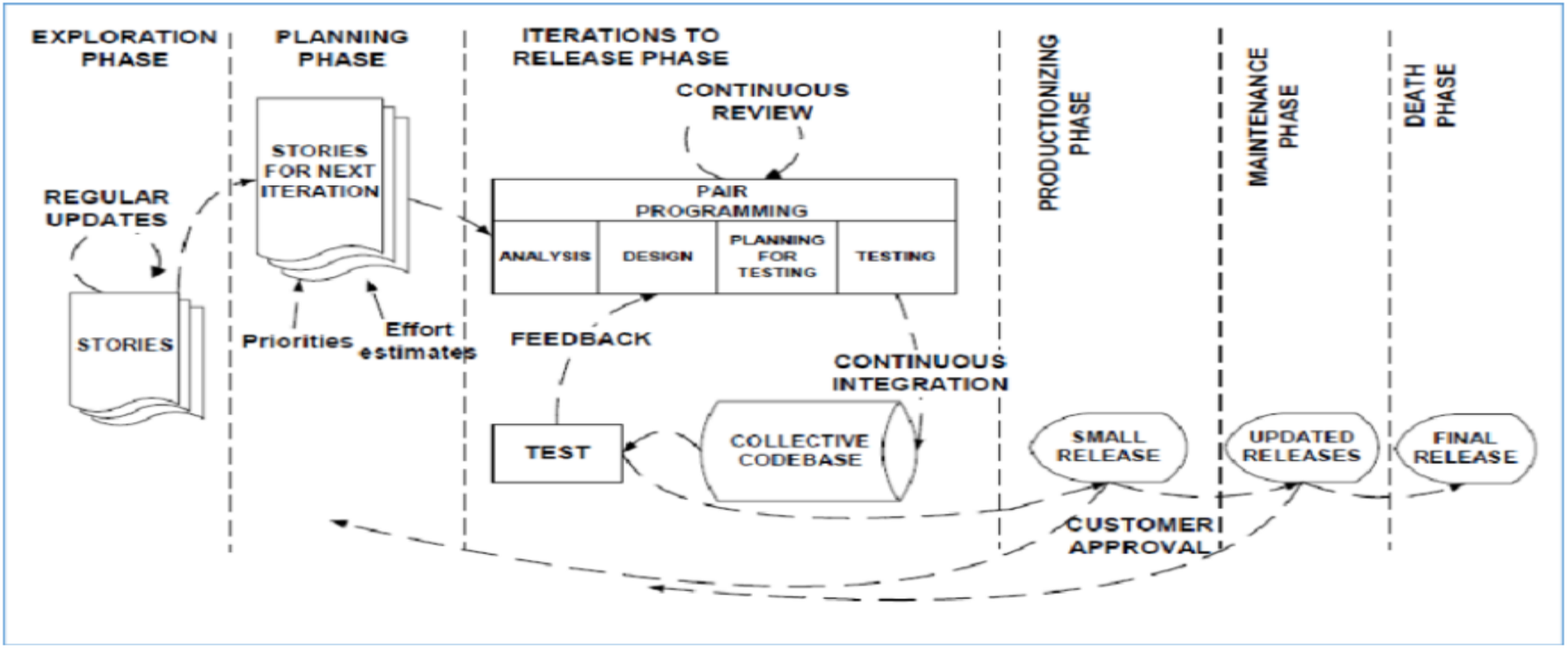
- permet aux utilisateurs finaux de créer leurs propres applications.
- les utilisateurs finaux sont formés à développer des applications personnalisées.

Les Méthodes Agiles (9/10):

Le Modèle XP(eXtreme Programming) (suite)

- Le modèle **Extreme Programming (XP)** est une méthode agile de développement logiciel qui se concentre sur l'**efficacité**, la **qualité** du **code**, et la **capacité** à **s'adapter aux changements des exigences**.

- Il s'adapte aux petit équipes travaillant dans un contexte changeant



Les Méthodes Agiles (10/10):
Le Modèle XP(eXtreme Programming) (suite)

○ Principes clés :

- **Simplicité** : Se concentrer sur ce qui est nécessaire et éviter les fonctionnalités superflues.
- **Feedback rapide** : Des retours constants du client et des tests fréquents.
- **Amélioration continue** : Refactorer le code régulièrement pour améliorer sa structure.
- **Collaboration** : Forte communication entre les développeurs et avec le client.

○ Pratiques principales :

- **Développement itératif** : Petites itérations (**1 à 2 semaines**).
- **Test-Driven Development (TDD)** : Les tests automatisés sont écrits avant le code.
- **Pair Programming** : Travail en binôme pour écrire et réviser le code.
- **Intégration continue** : Le code est fréquemment intégré et testé.



Partie 4: Les Activités de Développement de Logiciel SDLC

Contenu de la Partie 4

1. Les Etapes de Création d'un Logiciel
2. Les activités qui produisent des modèles
3. Les activités qui produisent des modèles
4. Etude préalable
5. Spécification de Besoins

   Analyse
6. Recueil des besoins Vs Analyse

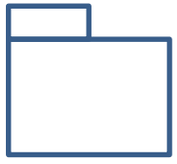
   Conception
7. Analyse Vs Conception
8. Comparaison entre Merise & UML
9. Implémentation

Les Etapes de Création d'un Logiciel

1. **Etude préalable**
2. **Spécification des besoins**
3. **Analyse**
4. **Conception**
5. **Implémentation**
6. **Test**

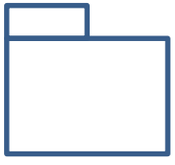
Les Activités qui Produisent des Modèles (1/2)

Etude préalable et Besoins .....



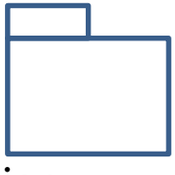
Modèle des cas d'utilisation

Analyse .....



Modèle d'analyse

Conception .....



Modèle de conception



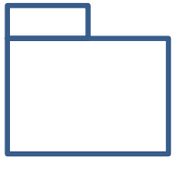
Modèle de déploiement

Implémentation .....



Modèle d'implémentation

Tests .....



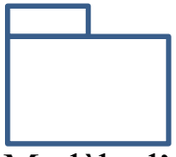
Modèle des tests

Les Activités qui Produisent des Modèles (2/2)

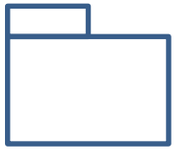
Les modèles sont principalement constitués de diagrammes UML



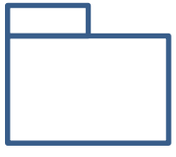
Modèle des cas d'utilisation



Modèle d'analyse



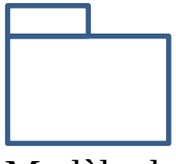
Modèle de conception



Modèle de déploiement



Modèle d'implémentation



Modèle des tests

- Diagramme de cas d'utilisation
- Diagramme de classes
- Diagramme d'objets
- Diagramme de composants
- Diagrammes de déploiement
- Diagrammes de séquence
- Diagrammes de collaboration
- Diagrammes Etat-transition
- Diagrammes d'activité

Etude Préalable (1/3)

Objectifs

- Analyse du fonctionnement de l'entreprise et diagnostic général de l'existant
- Recensement des critiques (positives ou négatives de l'entreprise et son système informatique et des besoins des utilisateurs
- Opportunité (financement, moyens humains, etc.) et faisabilité (technique) des automatisations
- Rédaction d'un cahier des charges

Etude Préalable(2/3)

Etapes de l'étude préalable

1. Définir le contexte général des systèmes d'entreprise

- Cadre de la réalisation du système
- Vue générale sur les activités de l'entreprise
- Objectifs du développement d'un nouveau système

2. Etude de l'existant (les solutions similaires)

- Description des solutions existantes (par différentes représentations littéraires/schématiques et modèles de communication /traitement/données) en collectant toutes les informations (informatisées ou non) utiles et nécessaires
- Opprotunité de la nouvelle solution à réaliser (amélioration, correction ou nouveautés):
  - pallier les dysfonctionnements et améliorer le système

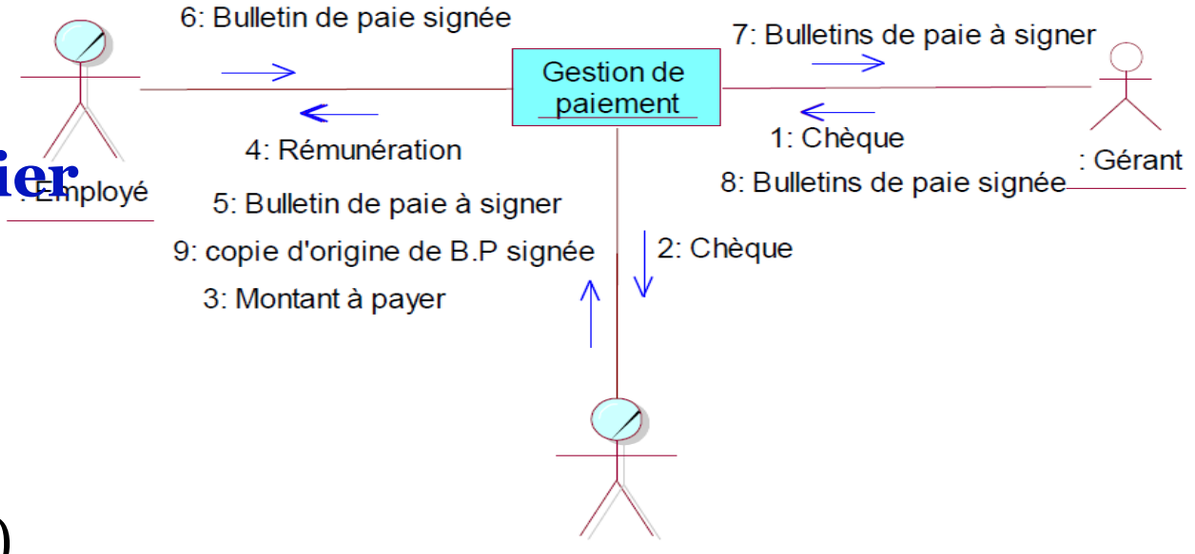
3. Choix d'un cycle de développement adéquat aux besoins du nouveau système)

- Etudes des différents méthodologies de développement des logiciel
- Choisir la méthodologie et la méthode la plus adéquate au besoins fixés

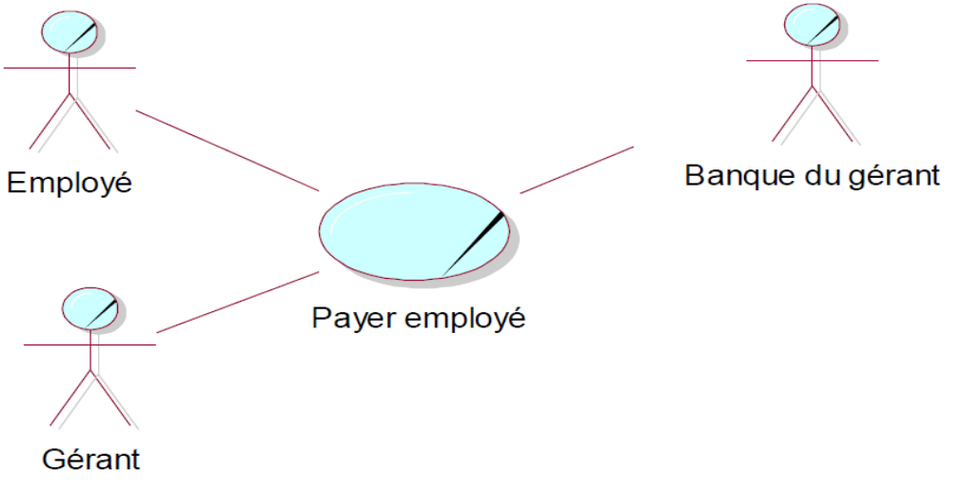
Etude Préalable(3/3)

Modélisation métier lors de l'étude préalable

- **Mise en contexte de la problématique**
  - **Diagramme de contexte métier**



- **Clarifier l'existant**
  - Structure de l'organisation (si **possible classes du domaine**)
  - Vocabulaire commun
  - modèle de processus métier: BPMN,
  - Diagramme d'activité,
  - **diagramme de cas d'utilisation métier**



Analyse et Spécification de Besoins (1/6)



Lors de l'étape de **l'analyse et spécification des besoins**, les **besoins fonctionnels , Besoins non fonctionnels** et le **Besoins techniques** sont identifiés et spécifiés

Analyse et Spécification de Besoins (2/6)

Besoins fonctionnels



- **Objectifs:**
  - Obtenir un schéma général de structuration des traitements et données, à un niveau conceptuel (c'est-à-dire indépendant de tout matériel ou logiciel de base)

    Quoi faire?

- **Critère d'un schéma conceptuel**
  - Communication (avec utilisateurs et autres informaticiens)
  - Conforme
  - Valide : complet et cohérent
  - Réalisable (automatisable en partie)
- **Principe d'indépendance des traitement et des données**
  - Pour cela, un logiciel (le SGBD) doit être capable, au moment de l'exécution des programmes, de retrouver les données nécessaires aux traitement à effectuer
  - Indépendance logique (respectivement physique) lorsque le schéma conceptuel (respectivement logique) des données peut être modifiés sans changer les programmes

Analyse et Spécification de Besoins (3/6)



Capture des besoins

Etapes

1.Identification des besoins fonctionnels

1.1 Identification des acteurs

1.2Diagramme de contexte dynamique

1.3 Expression littéraires des besoins fonctionctionnels

2. Identification des besoins non fonctionnels

- Performance
- Sécurité
- Ergonomie et convivialité
- Maintenance et évolution
- Etc.

3.Spécification des besoins Fonctionnels

Diagrammes de cas d'utilisation (général/ détaillés)

Analyse des besoins

Etapes

1.Description textuelles des cas d'utilisations

2. Diagrammes de séquence d'analyse

3. Diagrammes d'état-transition

4. Diagrammes d'activité

Analyse et Spécification de Besoins (4/6)

Identification des besoins techniques



- **Objectifs:**
  - Adapter la solution fonctionnelle à un choix technique particulier
  - Définition des structures de données et de leur enregistrement
  - Détermination des unités de traitement
  - Choix des matériels
  - Etablissement du calendrier et des budgets de réalisation

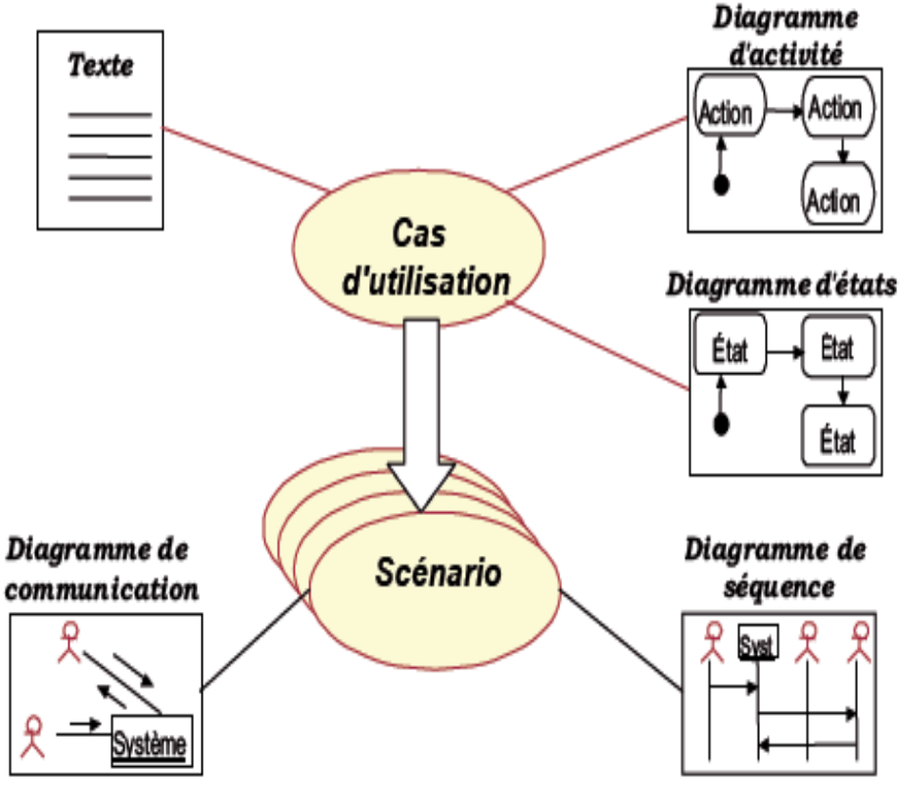
- **Etapes**

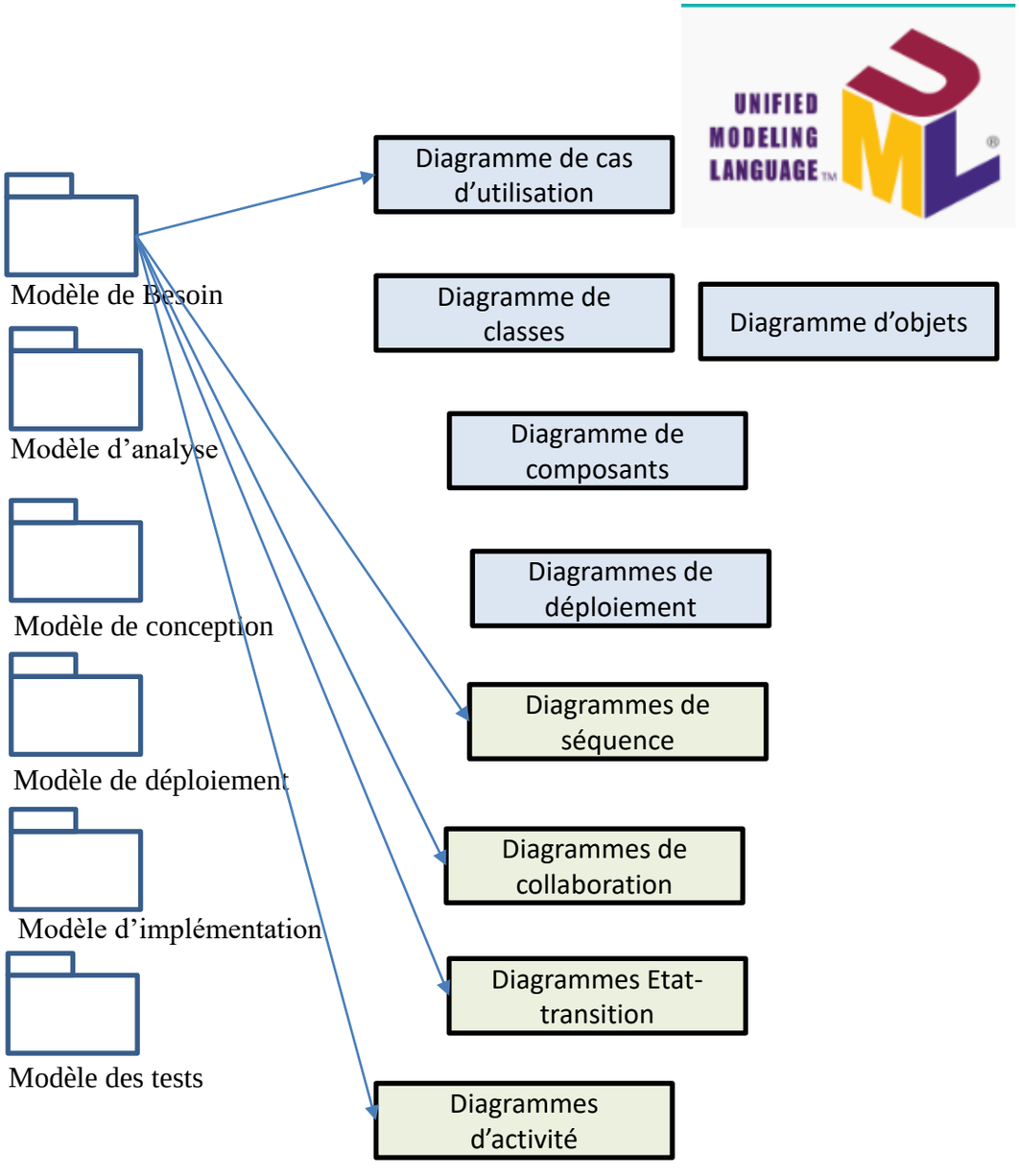
  1. Capture des besoins techniques

  - Définition des outils de modélisation, de développement, de prototypage de sécurité, etc.

Analyse et Spécification de Besoins (6/6)

Modélisation des besoins lors de la spécification et l'analyse des besoins





Analyse (1/3)

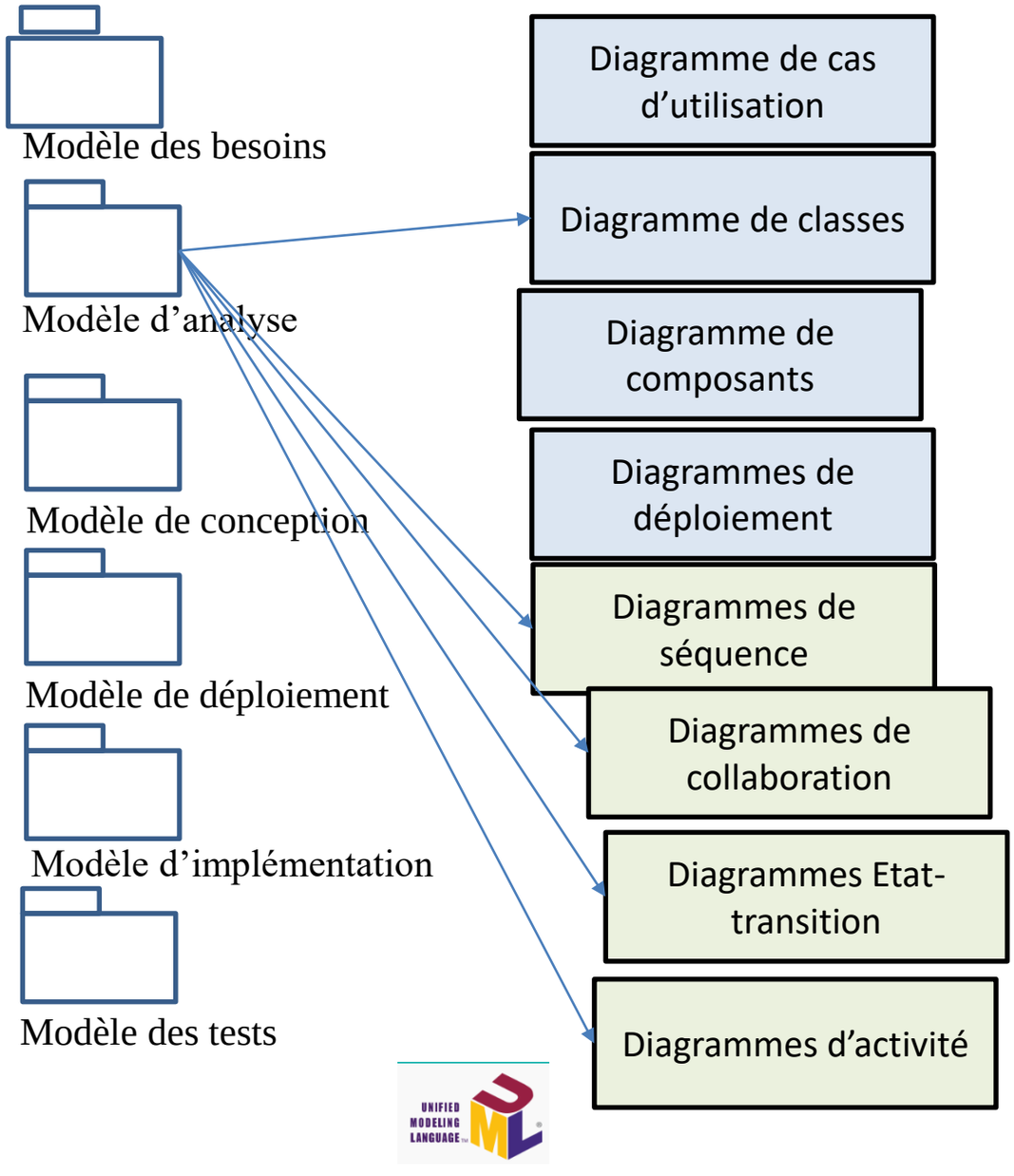


Objectifs

- L'analyse **synthétise les différents** cas d'utilisation et en donne une **vision unifiée**
- Mettre en évidence un (premier) **jeu complet de classes**

Diagrammes utilisés:

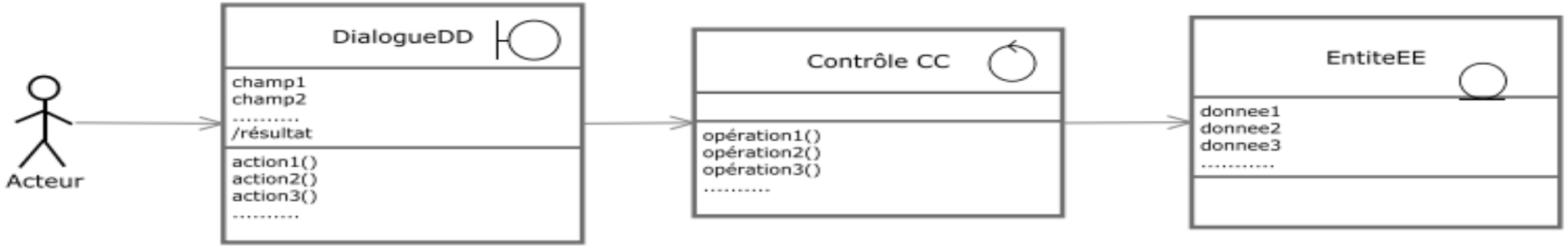
- Diagrammes de collaboration
- Etendre les classes (attributs, responsabilités, associations ...)
- Diagramme de classes participantes



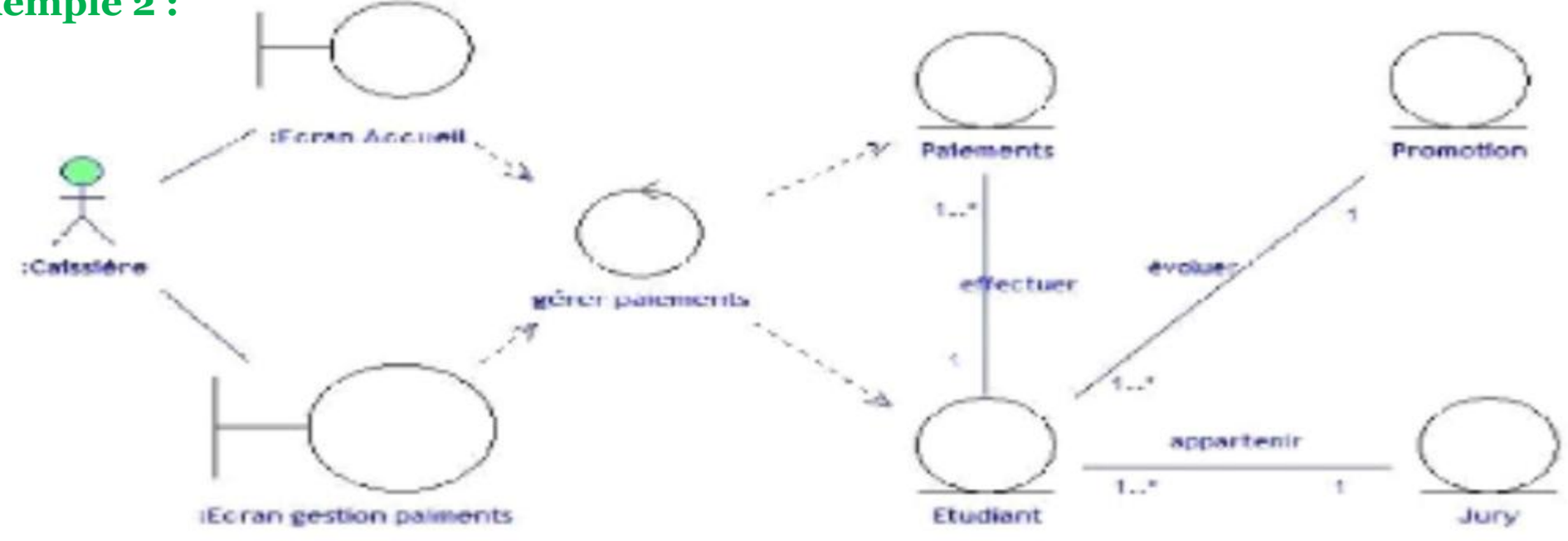
Analyse (2/3)

Exemples de diagramme de classes participantes

Exemple 1 :



Exemple 2 :

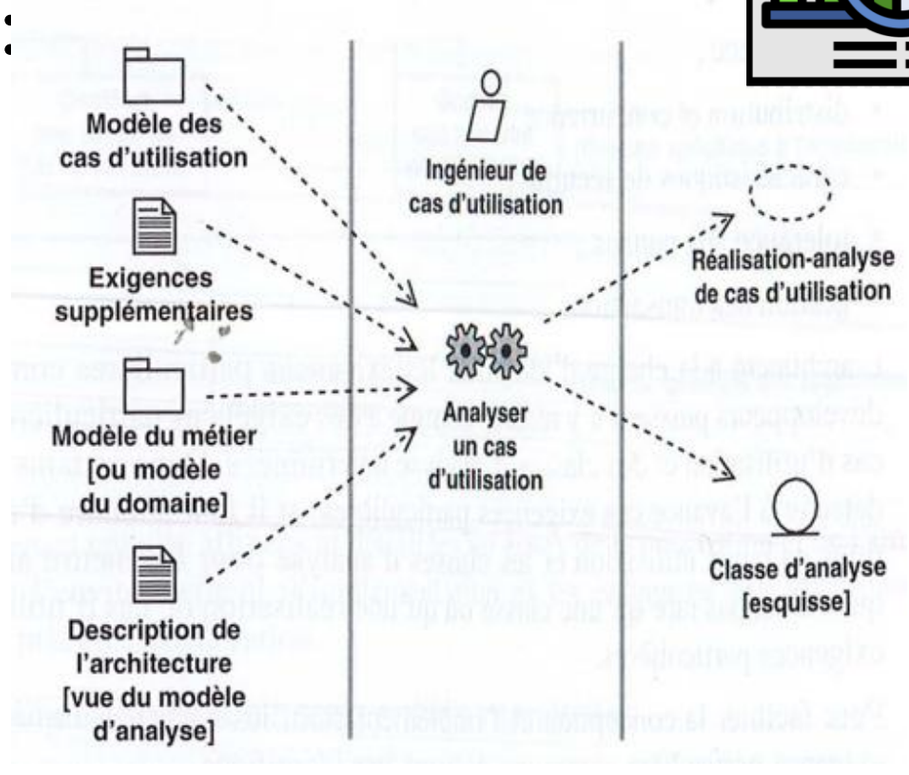


Analyse (3/3)



Réalisation des cas d'utilisation

- Pour chaque cas d'utilisation :
  - Identifier les classes d'analyse
  - Réaliser les scénarios comme des diagrammes d'interaction
- Synthétiser les classes, les valider par rapport aux cas d'utilisation



Recueil des besoins Vs Analyse

Recueil des besoins	Modèle d'analyse
• Formulé dans le langage du client • Structure la vue externe du système • Structuré avec les cas d'utilisation • Contrat entre les clients et les développeurs	• Formulé dans le langage du développeur • Structure la vue interne du système • Structuré avec des packages et des stéréotypes • Indications aux développeurs de la forme du système pour la conception et la réalisation

Conception (1/22)



Objectifs:

- Cette phase vise à assurer le bon déroulement et la réussite d'un projet de développement de logiciel. Elle implique la planification et la définition des différentes phases et activités nécessaires pour créer un logiciel fonctionnel et de haute qualité.
- Elle comprend
  - **Conception architecturale**
  - **Conception détaillée**

Conception (2/22)



- **Etapes**
  - **1.Choix du modèles architecturaux**
    - 1.1 Architecture physique
    - 1.2 Architecture logique (motif d'architecture logicielle)
  - **2.Conception de la base de données**
    - 2.1. Modèle MCD
    - 2.2. MLD
  - **3.Conception logicielle**
    - 3.1Vue statique
    - 3.2Vue dynamique
  - **4.Conception graphique**
    - Maquettage (prototypage)

Conception Architecturale

Conception Détaillée

Conception (3/22):
Conception Architecturale



- La **conception architecturale** dans le contexte du développement logiciel implique à la fois la **définition de l'architecture logicielle** et, dans certains cas, **de l'architecture matérielle (ou architecture physique)**.

  - **Architecture Logicielle :** La conception architecturale logicielle concerne la **structure globale du logiciel**, comment ses différentes parties interagissent et communiquent entre elles. Cela implique de décider **comment diviser le logiciel en modules**, en **composants** ou en **couches**, et comment ces éléments fonctionnent ensemble pour répondre aux besoins du système. Les **considérations architecturales incluent** la **séparation des responsabilités**, la gestion **des dépendances**, le **choix des motifs de conception** (comme MVC, micro services, architecture en couches, etc.) et la **définition des interfaces entre les différentes parties du logiciel.**

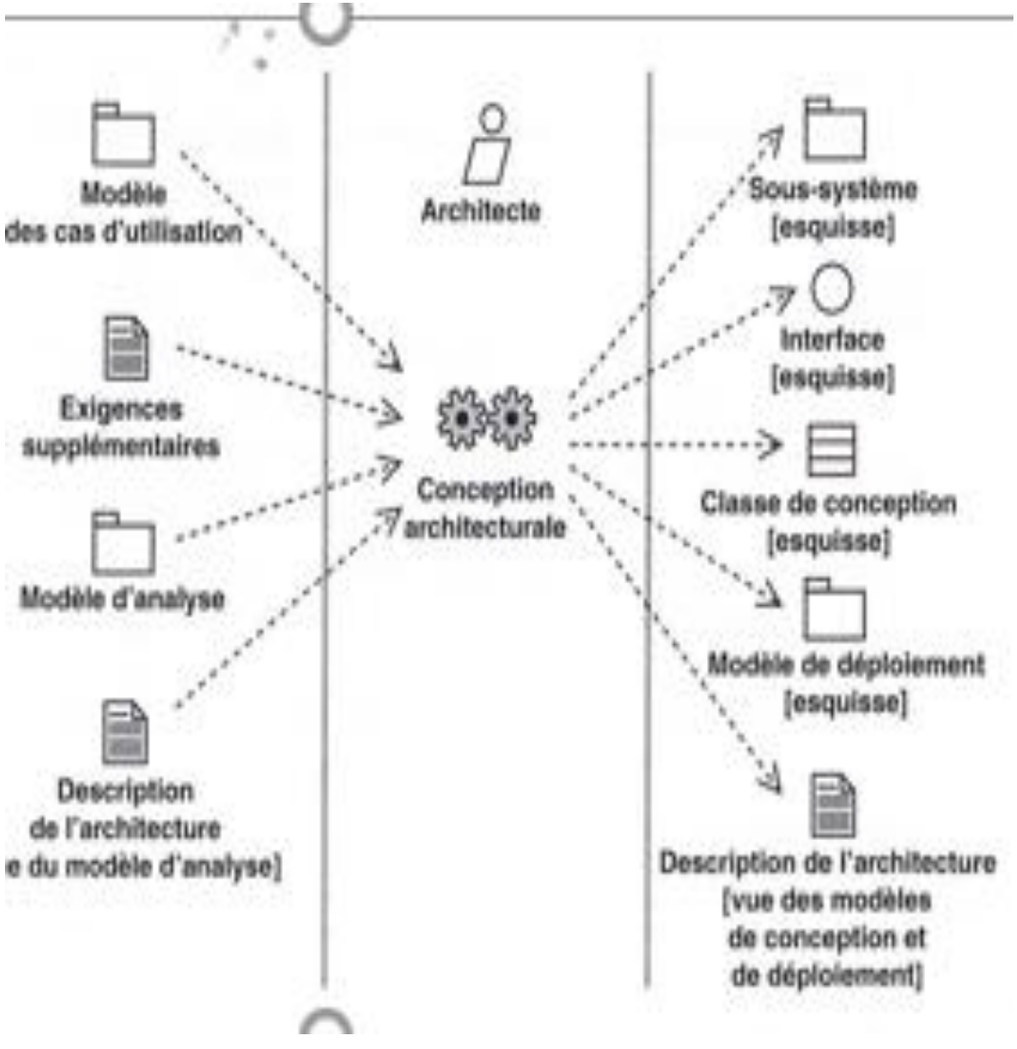
Conception (4/22):
Conception Architecturale(suite)



- **Architecture Matérielle :** Bien que la conception architecturale se concentre principalement sur **l'aspect logiciel**, dans certains cas, elle peut également **impliquer des décisions** concernant **l'architecture matérielle** sur laquelle le logiciel sera exécuté. Cela peut inclure des choix tels que la **sélection des serveurs**, des **machines virtuelles**, des **dispositifs embarqués** ou d'autres types **d'infrastructures.** Les **considérations matérielles** peuvent **influencer la performance**, la **disponibilité**, l'**évolutivité** et d'autres aspects du logiciel.

Conception(5/22):
Conception architecturale (suite)

- Concevoir l'architecture signifie:
  - Estimer les besoins (physiques) :
    - Performance, concurrence, gestion des données, IHM, robustesse
  - Organiser le système en sous systèmes
  - Choisir un style architectural
  - Intégrer l'existant
  - Choisir un modèle de déploiement



Conception (6/22):

Conception de la base de données



- **Objectifs:**
  - la conception de la base de données vise à **créer une structure solide** et **bien organisée** pour **stocker, gérer** et accéder aux **données de manière efficace**, sécurisée et cohérente.
  - Elle comprend:
    1. **MCD :** La première étape consiste **à modéliser les données** de **manière à représenter** fidèlement la **réalité** du **domaine d'application** du logiciel. Cela se fait généralement en utilisant des **modèles conceptuels**, comme les **diagrammes entité-relation (ER)**
    2. **MLD: À partir** du **modèle conceptuel**, la **conception logique** consiste à **traduire ces concepts en structures de données spécifiques à un système de gestion de base de données** (SGBD) particulier, comme les **tables**, les **colonnes**, les **clés primaires et étrangères**, etc.

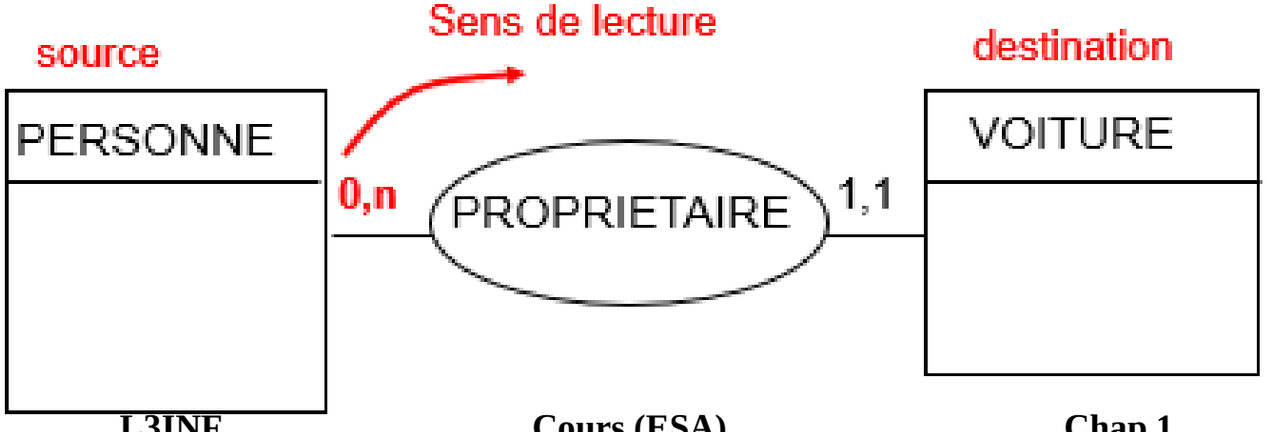
Conception (7/22):

Conception de la base de données (suite)

Modèle conceptuel de données (MCD)

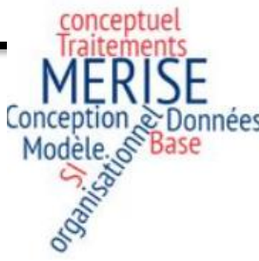
- Le MCD est généralement représenté sous forme de **diagramme entité-relation (ER)** qui met en évidence les **relations entre différentes entités** et les **attributs associés**.
- Un MCD en globe :
  - **Entités** :
  - **Relations**
  - **Attributs**
  - **Cardinalité**

Diagramme ER : Le diagramme entité-relation (ER) est un moyen visuel de représenter les entités, les relations, les attributs et les cardinalités. Il offre une vue d'ensemble claire et compréhensible des concepts et de leurs interactions.



Conception (8/22) :

Conception de la base de données (suite)

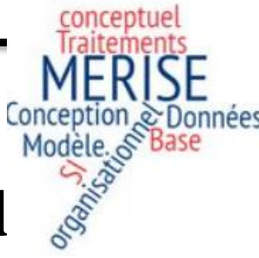


Modèle conceptuel de données (MCD)(suite)

Le MCD sert de base pour passer à la prochaine étape de conception de base de données, appelée "Modèle Logique de Données" (MLD), où les concepts abstraits du MCD sont transformés en structures de données spécifiques à un système de gestion de base de données (SGBD) particulier. Le MCD facilite la communication entre les concepteurs, les développeurs et les parties prenantes en fournissant une représentation claire des exigences de données du système.

Conception (9/22) :

Conception de la base de données(suite)



MLD (Modèle Logique de Données)

- Le MLD **est la deuxième étape** de la conception de bases de d après la **création du MCD**
- Le MLD consiste à **transformer le modèle conceptuel** abstrait **en** une **représentation** plus **concrète**, spécifique **à un système de gestion de base de données (SGBD)** particulier.
- Le MLD prend en compte **les détails techniques** de la manière dont les données seront stockées et gérées dans la base de données.

Exemples de règles de passage du MCD vers MLD:

- **Conversion des entités et des relations** : Chaque entité devient une table, et les relations deviennent des associations entre les tables.
- **Définition des clés** : Les clés primaires et étrangères sont définies pour chaque table. La clé primaire est un attribut unique qui identifie de manière unique chaque enregistrement dans une table. Les clés étrangères établissent des liens entre les tables en référençant les clés primaires d'autres tables.

Conception (10/22) :

Conception de la base de données(suite)



MLD (Modèle Logique de Données) (suite)

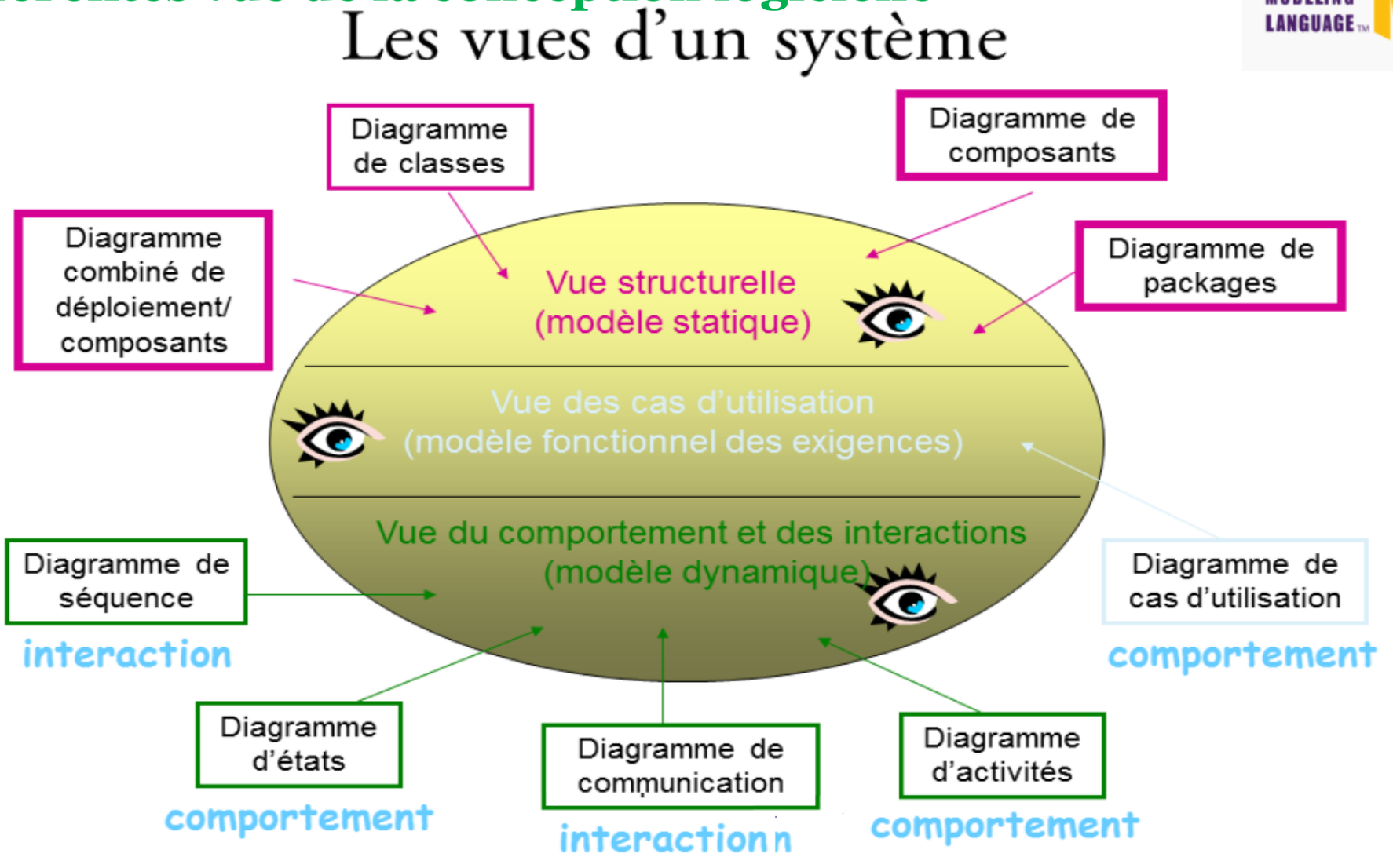
- **Exemples de règles de passage du MCD vers MLD (suite):**

  - **Attributs et types de données** : Les attributs définis dans le MCD sont détaillés en spécifiant les types de données appropriés pour le SGBD choisi. Par exemple, un attribut "Date de naissance" pourrait être de type "DATE" dans la base de données.

Conception (11/22) :

Conception logicielle

Les différentes vue de la conception logicielle



Conception (12/22):

Conception logicielle (suite)

Vue Statique

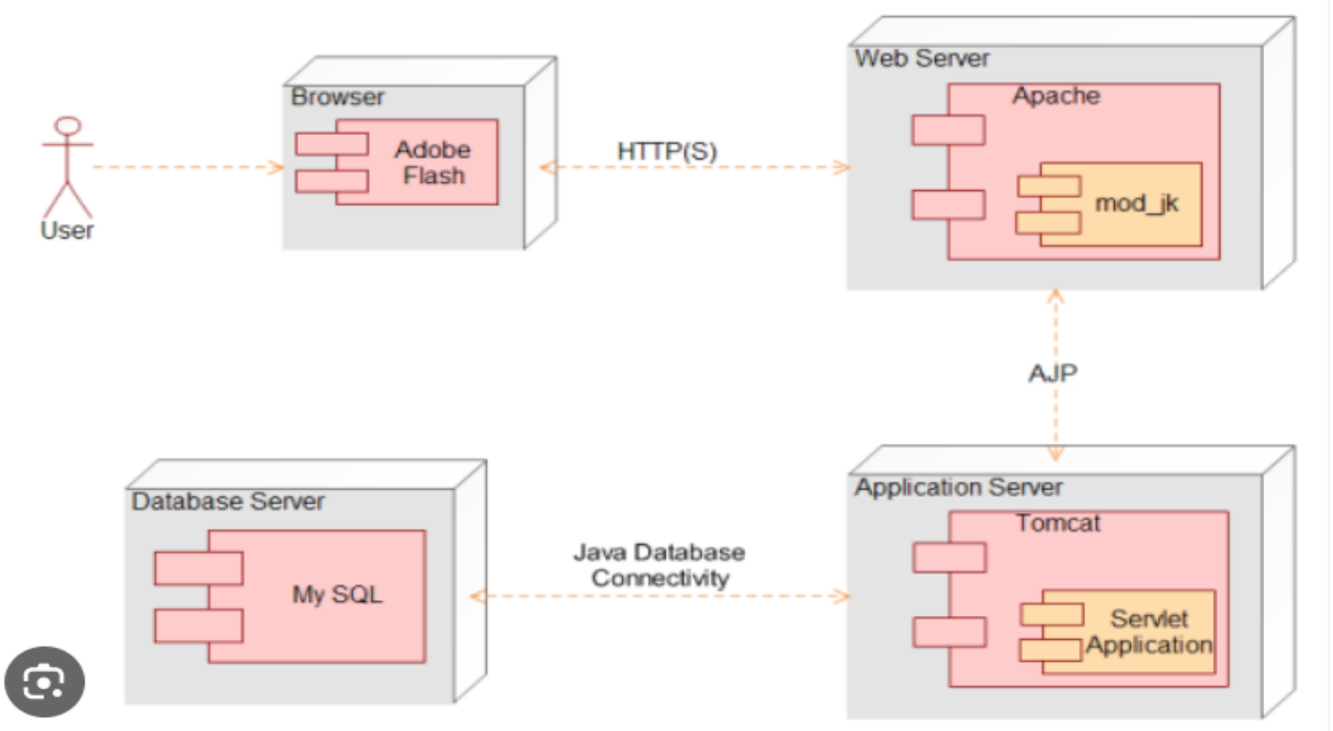
Les diagrammes de structure (structure diagrams) ou diagrammes statiques (static diagrams) rassemblent :

- **Diagramme de classes (class diagram)** : représentation des classes intervenant dans le système.
- **Diagramme d'objets (object diagram)** : représentation des instances de classes (objets) utilisées dans le système.
- **Diagramme de composants (component diagram)** : représentation des composants du système d'un point de vue physique, tels qu'ils sont mis en œuvre (fichiers, bibliothèques, bases de données...)
- **Diagramme de déploiement (deployment diagram)** : représentation des éléments matériels (ordinateurs, périphériques, réseaux, systèmes de stockage...) et la manière dont les composants du système sont répartis sur ces éléments matériels et interagissent entre eux.
- **Diagramme des paquetage (package diagram)** : représentation des dépendances entre les paquets (un paquet étant un conteneur logique permettant de regrouper et d'organiser les éléments dans le modèle UML), c'est-à-dire entre les ensembles de définitions.
- **Diagramme de structure composite (composite structure diagram) :** représentation sous forme de boîte blanche des relations entre composants d'une classe (depuis UML 2.x).
- **Diagramme de profils (profile diagram)** : spécialisation et personnalisation pour un domaine particulier d'un meta-modèle de référence d'UML (depuis UML 2.2).

Conception (13/22):

Conception logicielle (suite)

Vue Statique

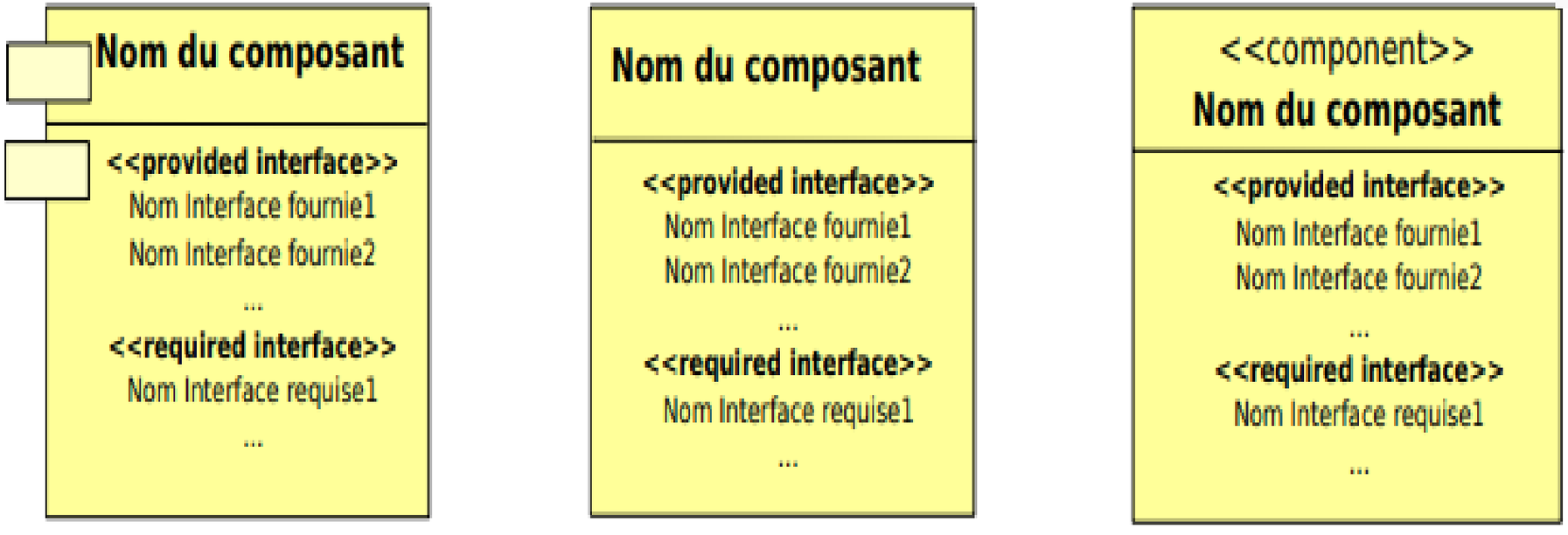


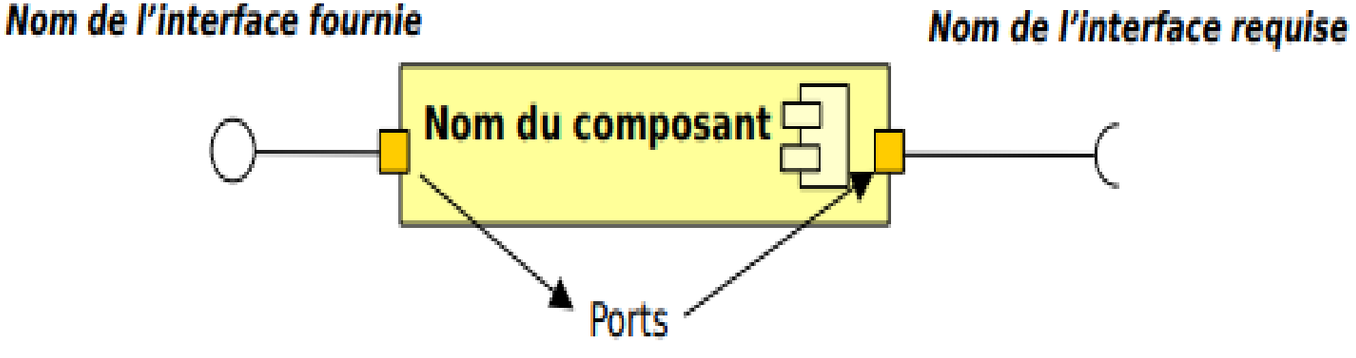
Exemple de diagramme de déploiement UML

Conception (14/22):

Conception logicielle (suite)

Vue Statique: diagramme de composant

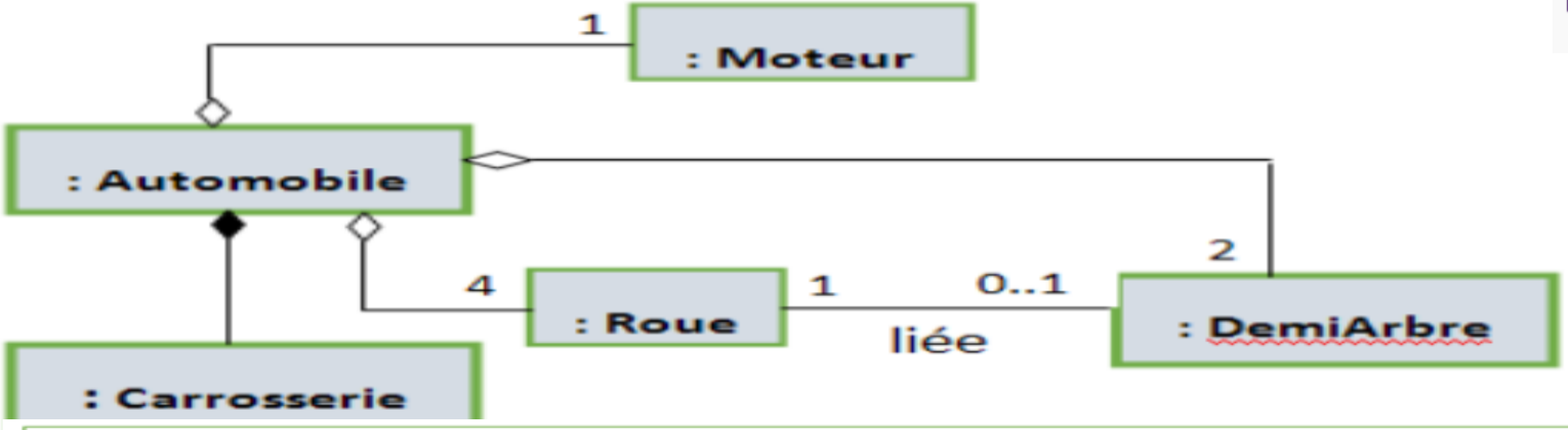


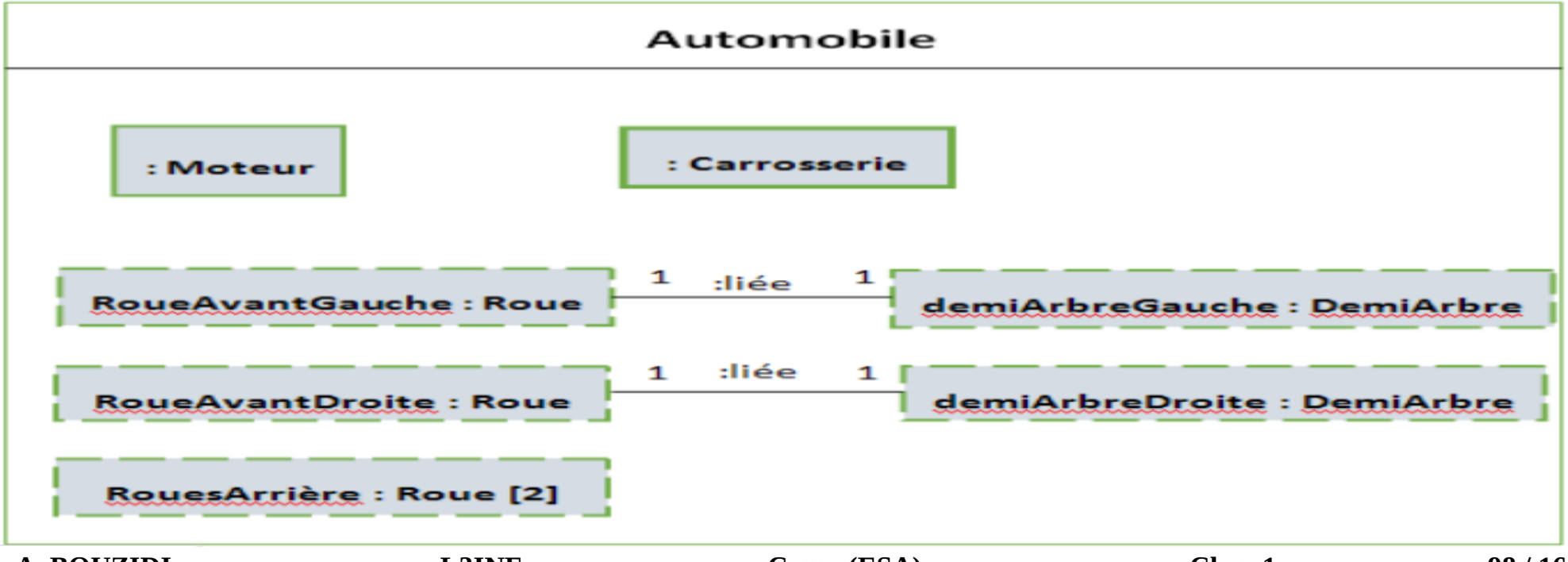


Conception (15/22):

Conception logicielle (suite)

Vue Statique: diagramme de structure composite





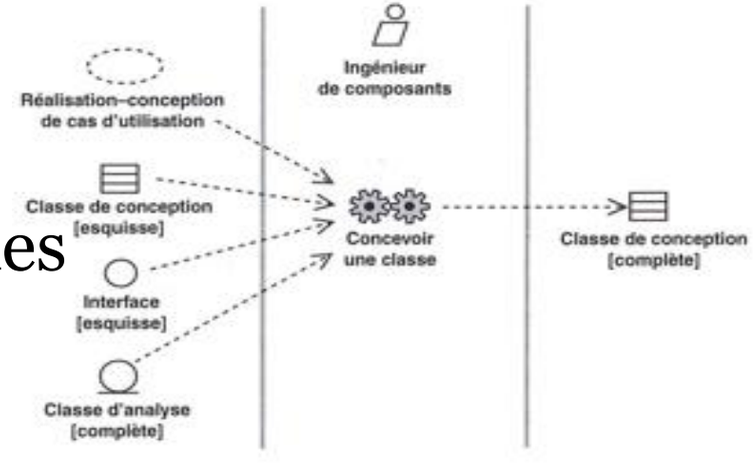
Conception (16/22):

Conception logicielle(suite)

Vue Statique (suite)

Exemple concevoir un diagramme de classes de conception

- Finalisation des classes
  - Choix des attributs, des opérations, des associations
  - Choix de mise en œuvre des attributs, des opérations, des associations
  - Choix des algorithmes



Conception (17/22):
Conception logicielle (suite)

Vue Dynamique : Diagrammes de comportement

Les diagrammes de comportement (behavior diagrams) rassemblent :

- **Diagramme états-transitions (state machine diagram)** : représentation sous forme de machine à états finis du comportement du système ou de ses composants.
- **Diagramme d'activité (activity diagram) :** représentation sous forme de flux ou d'enchaînement d'activités du comportement du système ou de ses composants.

Conception (18/22):

Conception logicielle(suite)

Vue dynamique: Diagrammes d'intéraction

Les diagrammes d'interaction (interaction diagrams) ou diagrammes dynamiques (dynamic diagrams) rassemblent :

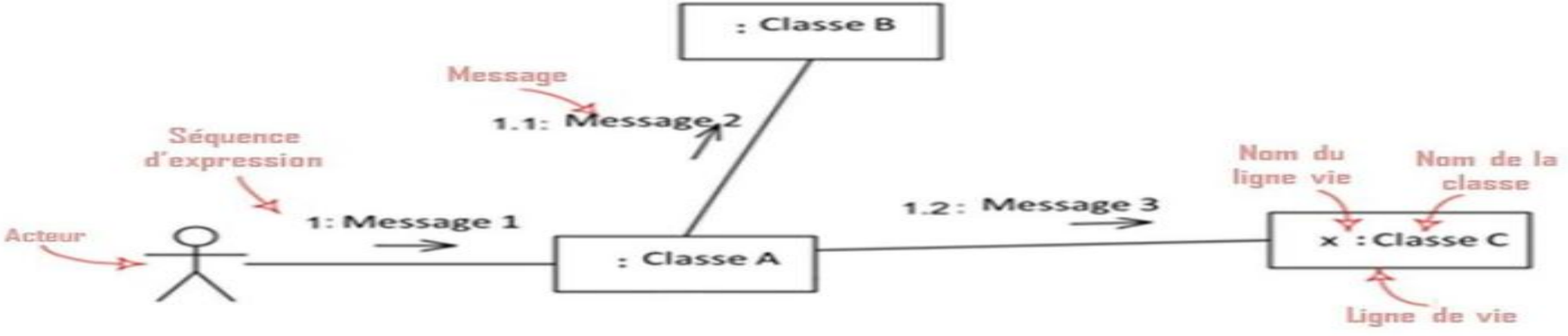
- **Diagramme de séquence (sequence diagram)** : représentation de façon séquentielle du déroulement des traitements et des interactions entre les éléments du système et/ou de ses acteurs.
- **Diagramme de communication (communication diagram)** : représentation de façon simplifiée d'un diagramme de séquence se concentrant sur les échanges de messages entre les objets (depuis UML 2.x).
- **Diagramme global d'interaction (interaction overview diagram)** : représentation des enchaînements possibles entre les scénarios préalablement identifiés sous forme de diagrammes de séquences (variante du diagramme d'activité) (depuis UML 2.x).
- **Diagramme de temps (timing diagram)** : représentation des variations d'une donnée au cours du temps (depuis UML 2.3).

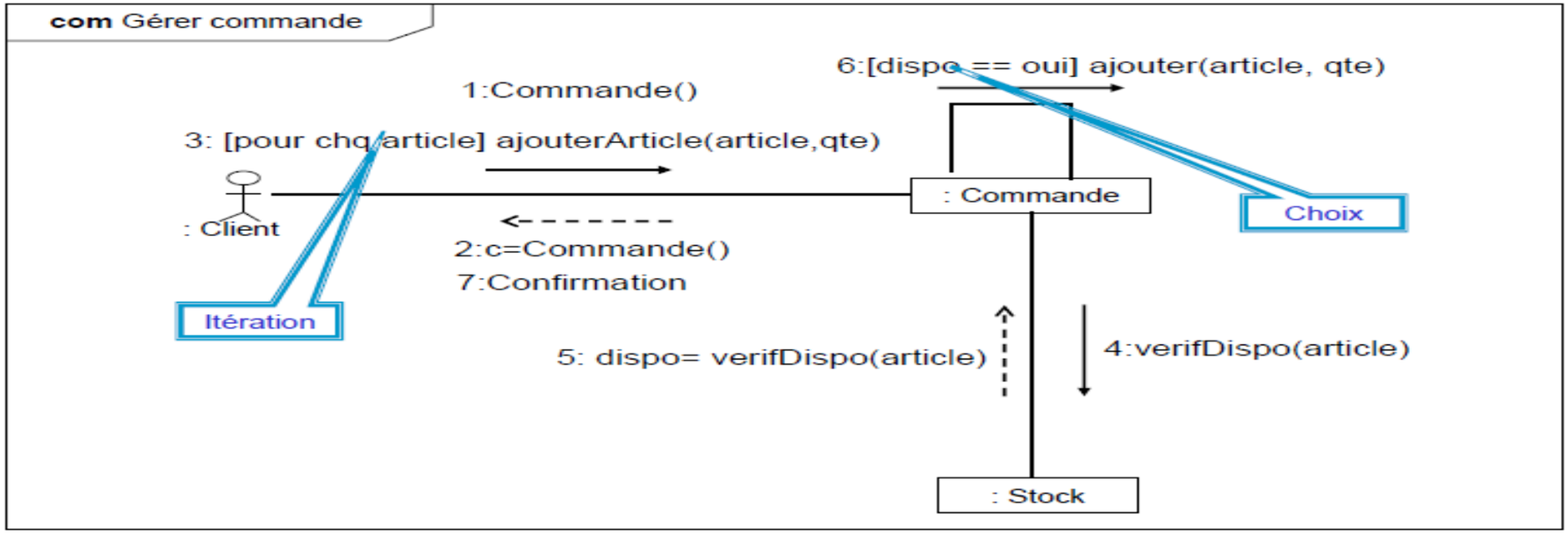
Conception (19/22):

Conception logicielle(suite)

Vue dynamique: Diagrammes d'intéraction

Exemple de diagramme de communication



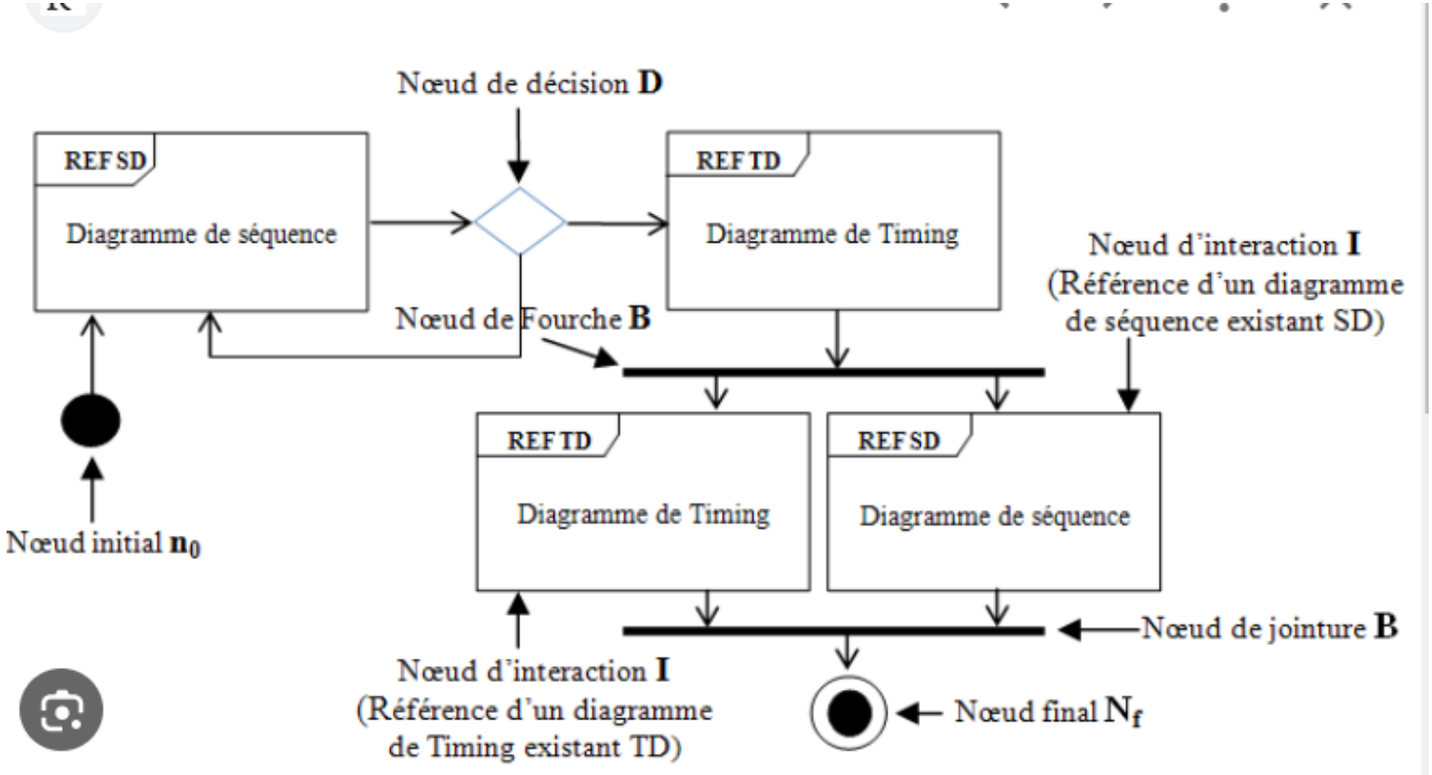


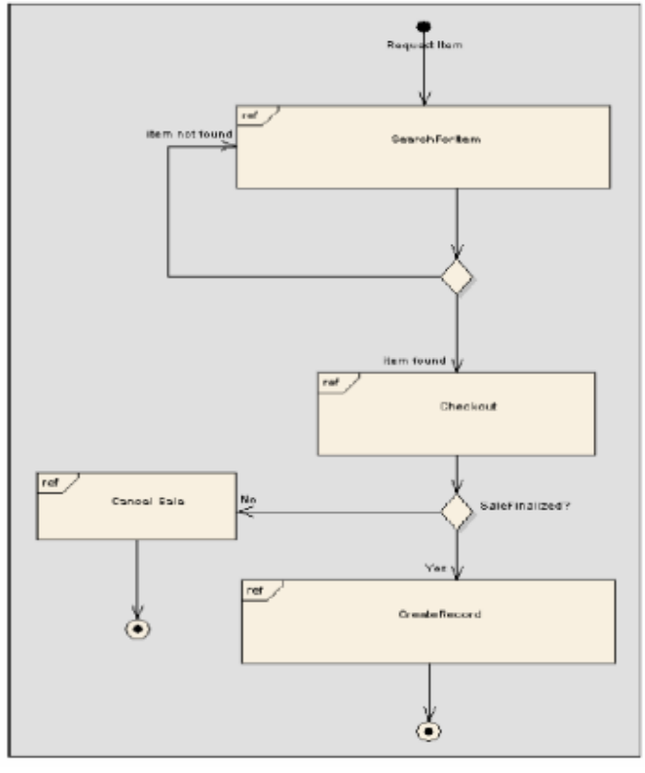
Conception (19/22):

Conception logicielle(suite)

Vue dynamique: Diagrammes d'intéraction

Exemple de diagramme global d'intéraction



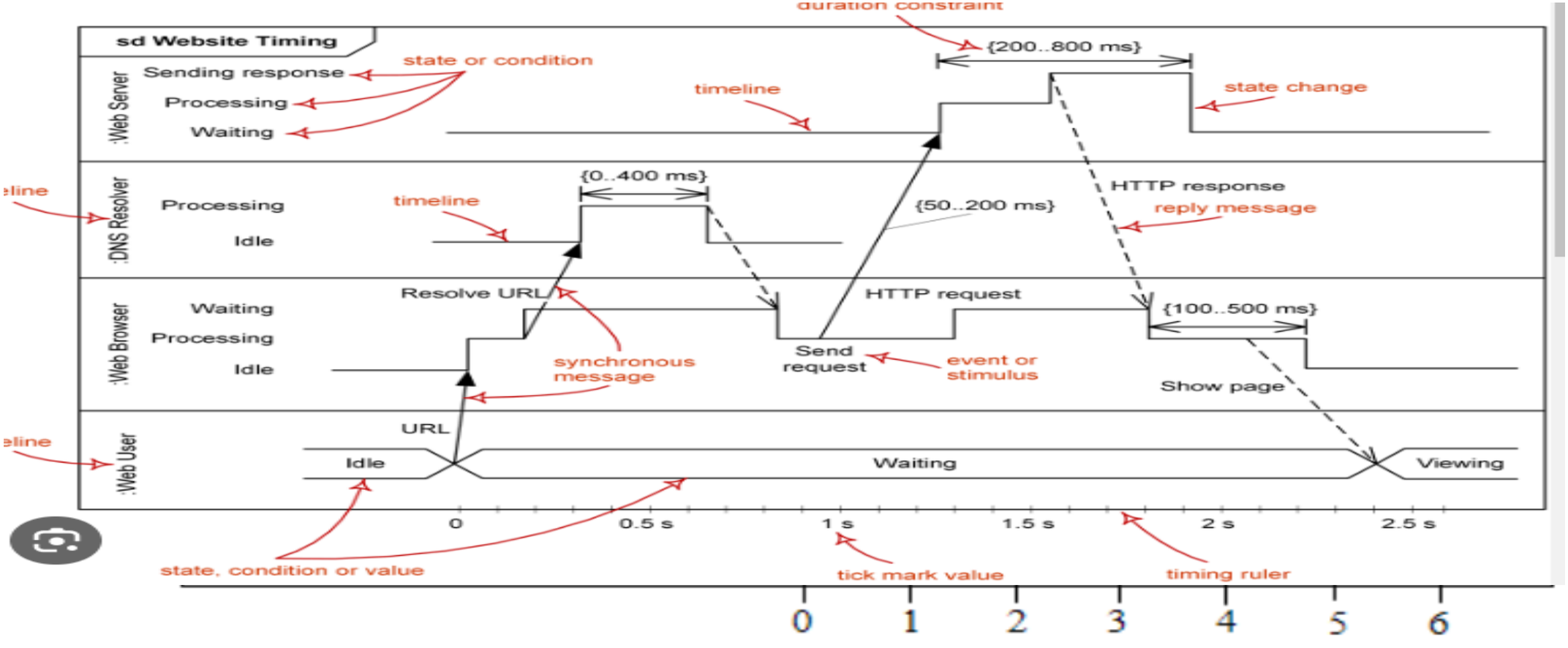


Conception (20/22):

Conception logicielle(suite)

Vue dynamique : Diagrammes d'intéraction

Exemple de diagramme de timing

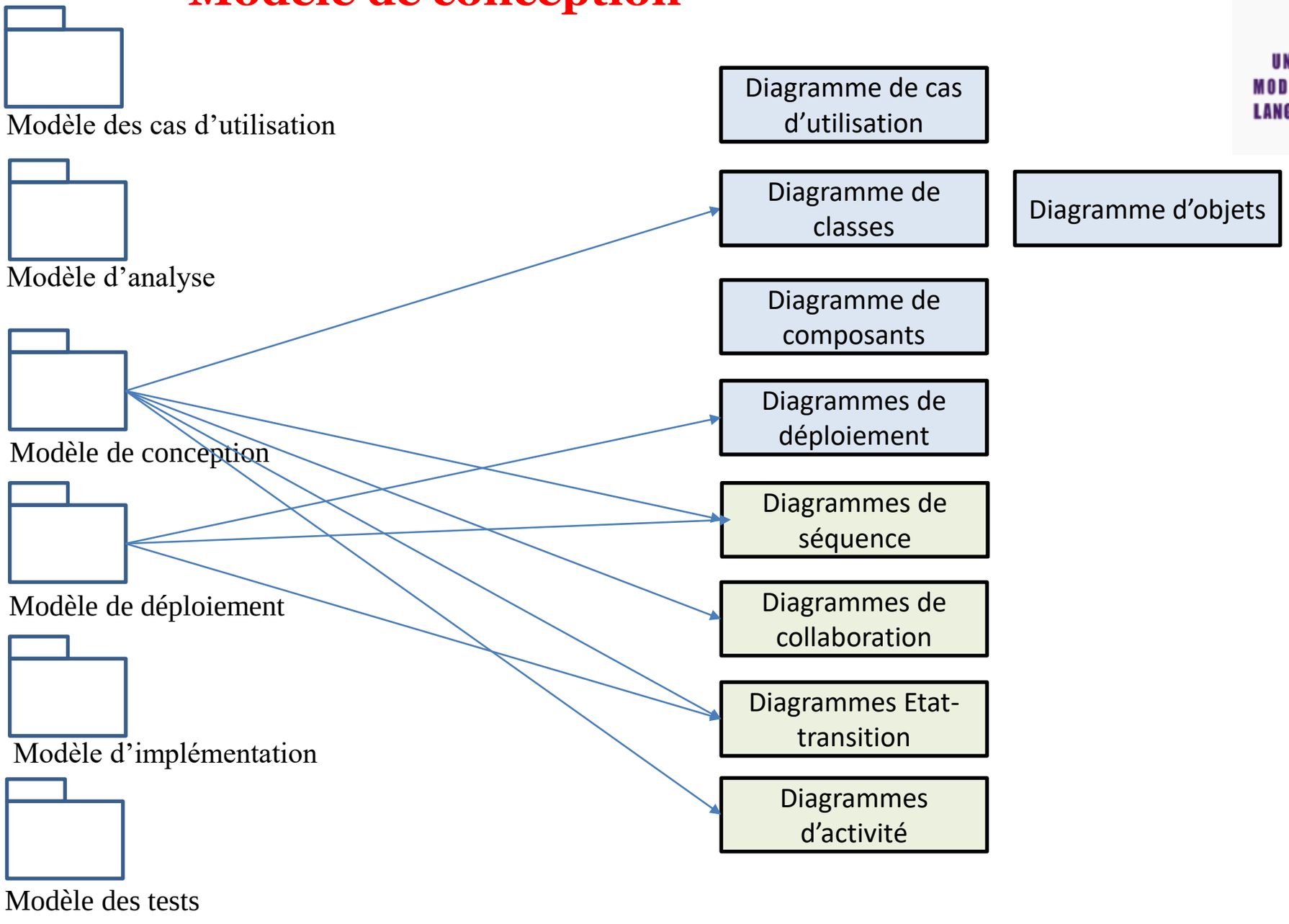


Conception (21/22):
Conception graphique

- **Définition**
  - La conception graphique, également appelée **conception visuelle**, est une étape clé dans le processus de développement de produits visuels tels que les sites web, les applications mobiles, les interfaces utilisateur, les affiches, les logos, les brochures (annonces) et d'autres **éléments visuels**.

- **Exemples d'outils de maquettage**
  - Balsamiq
  - Figma
  - Adobe XD
  - Sketch
  - Etc.

Modèle de conception



Analyse Vs Conception

Recueil d'analyse	Modèle de conception
• Modèle conceptuel • Autorise plusieurs conceptions • Peu de stéréotypes • Peu de couches	• Modèle physique • Spécifique à une implantation • Stéréotypes dépendant de l'environnement cible • Plusieurs couches

Comparaison entre Merise & UML

Merise	UML
Méthode d'analyse et de conception de système d'information	Langage de représentation d'un SI
Méthodes de modélisation de données et traitements orienté bases de données relationnelles	Système de notation orienté objet
relationnel	Objet
Franco-Français	Internationale
Schéma directeur, étude préalable, étude détaillée et la réalisation	Langage de modélisation des systèmes standards, qui utilise des diagrammes pour représenter chaque aspect d'un système : statique, dynamique… en s'appuyant sur la notion d'orienté objet

Implémentation

- **Objectifs :**
- Mise en œuvre logicielle
- S'appuie sur les modèles précédents pour produire des composants logiciels
- Mise en place de l'architecture logicielle sur une architecture matérielle conforme au modèle de déploiement
- Produit un « modèle d'implantation »

- **Etapes**

1. Environnement de développement

1.1Environnement matériel

1.2.Environnement logiciel

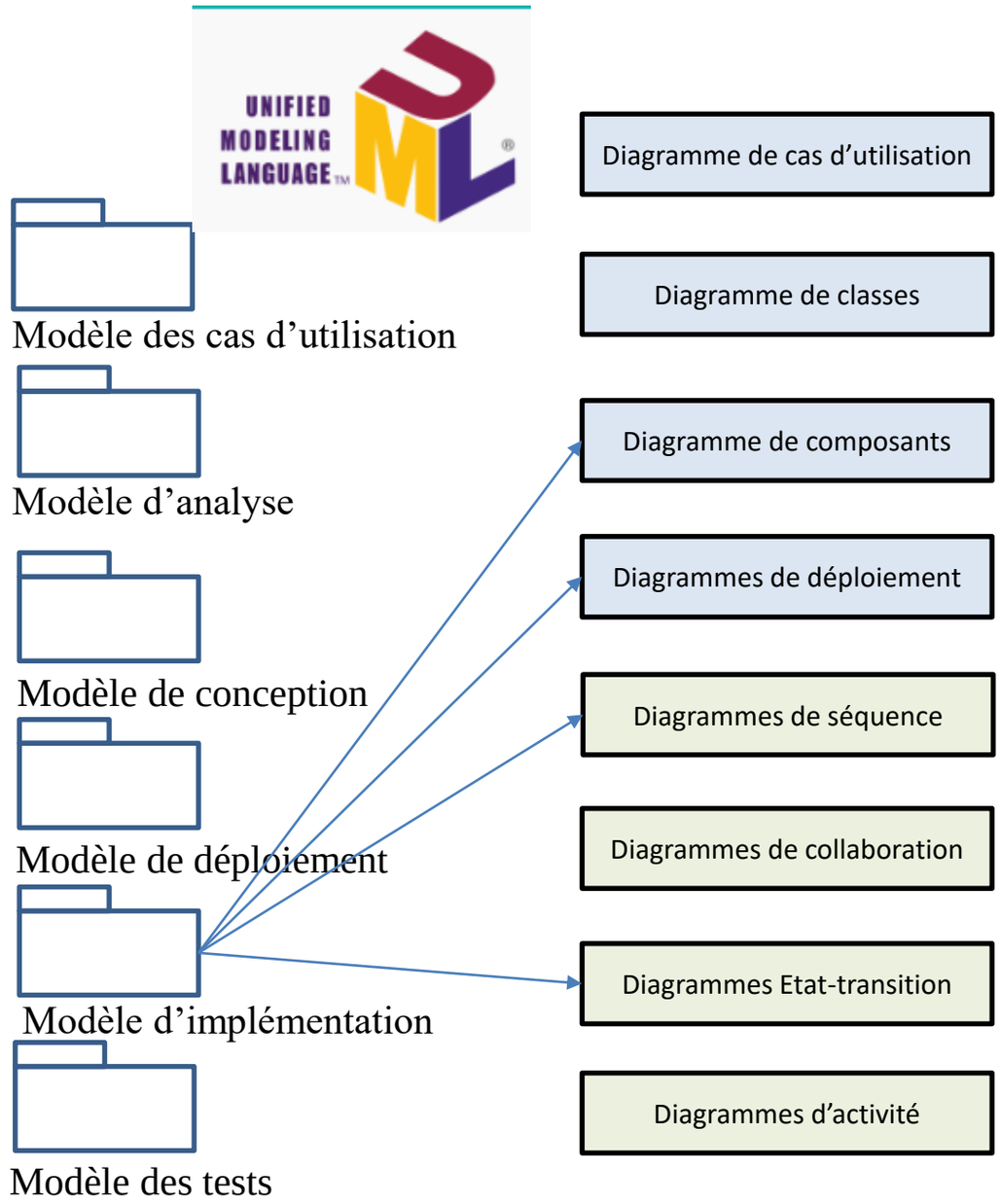
2 Farameowrk front end

3. Framework backend

4. Réalisation et résultat

4.1 Diagramme de navigation

4.2 Interfaces réalisées



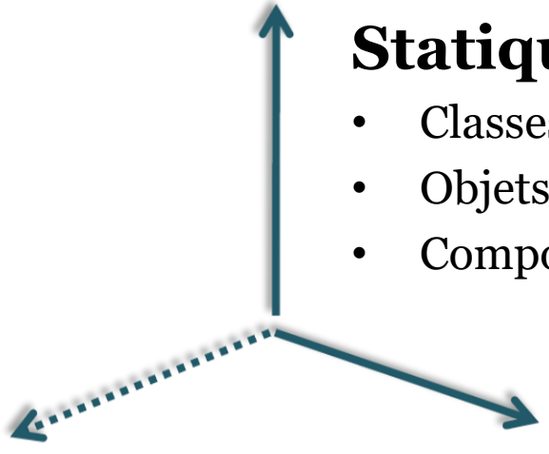
Annexes



Partie 1: Les Diagrammes UML

Les diagrammes UML

répartition selon trois points de vues



Statique:

- Classes (class diagram)
- Objets (Object diagram)
- Composants (Component diagram)

Fonctionnelle:

- cas d'utilisation (Use Case diagram))

Dynamique

- Etats-Transitions (State Machine diagram)
- Activité (Activitydiagram)
- Séquence (Sequencediagram)
- Collaboration (Collaboration diagram)

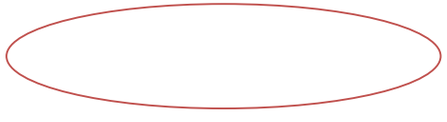
UML 2.0

- Paquetage (Package diagram)
- Structures composites (Composite structure diagram)
- Global d'interaction (Interaction overview diagram)
- Temps (Timing diagram)

Cas d'utilisation (Axe fonctionnel)

- Structurer les besoins des utilisateurs et les objectifs correspondants du système.
- Préoccuper des cas "réels" des utilisateurs ; ils ne présentent pas de solutions d'implémentation et ne forment pas un inventaire fonctionnel du système.

NOTATION



Cas d'utilisation

Objectif du système, motivé par un besoin d'un (ou plusieurs) acteur(s)



Acteur

Personne ou composant d'origine d'une interaction avec le système

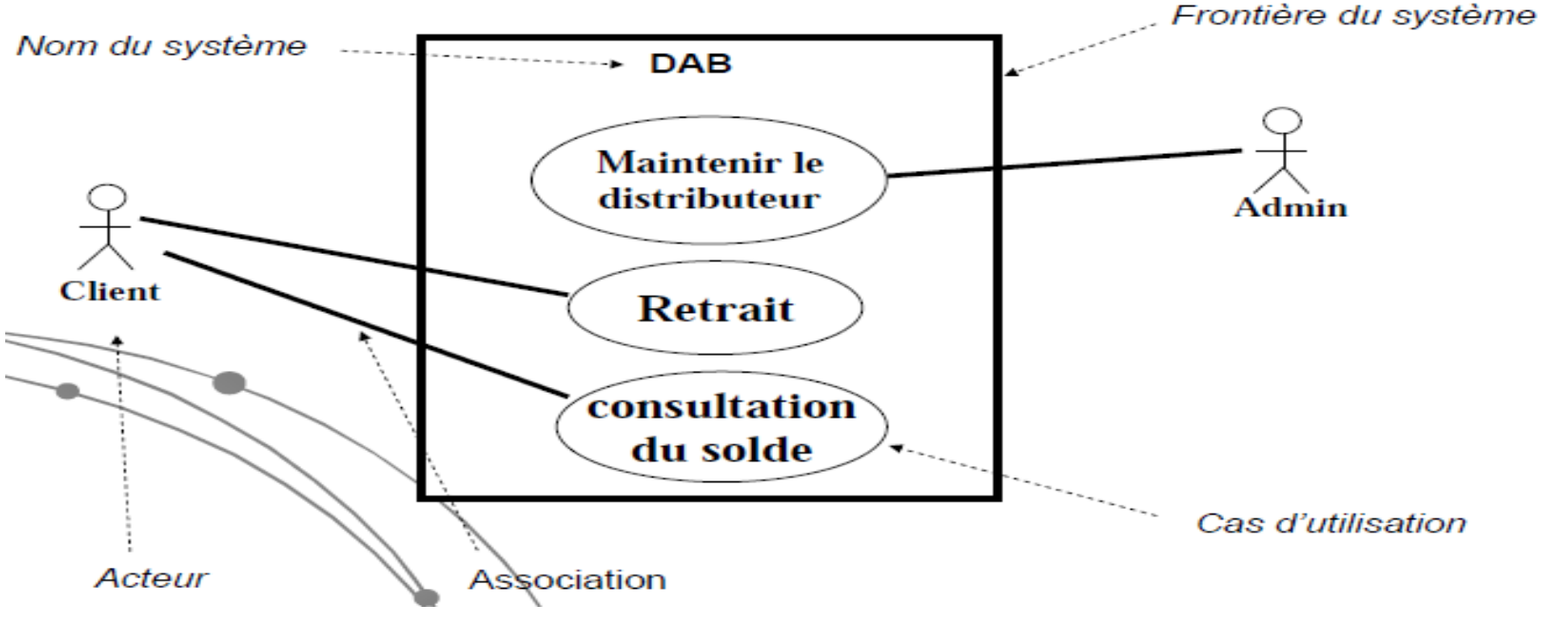


Relation d'utilisation

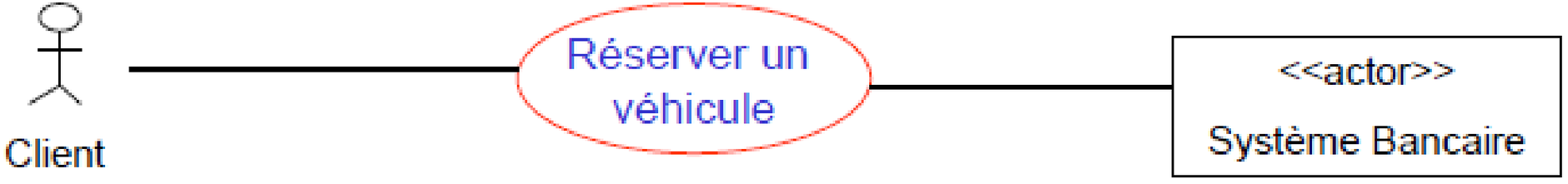
Le cas source contient aussi le comportement décrit dans la cas destination

Cas d'utilisation (Axe fonctionnel)

- Etapes à suivre :
  - Comprendre le problème.
  - Identifier les acteurs.
  - Identifier les cas d'utilisations
  - Dresser un premier diagramme de cas d'utilisation



Cas d'utilisation(Description textuelle)



Sommaire d'identification

Titre : Réserver une voiture
But : ce cas d'utilisation permet à un client internaute de saisir une demande de réservation.
Acteurs principaux : Client
Acteurs secondaires :Système Bancaire
Pré-conditions : le parc de véhicules n'est pas vide.
Post-conditions :Une demande de réservation a été enregistrée par le système avec toutes les informations nécessaires.

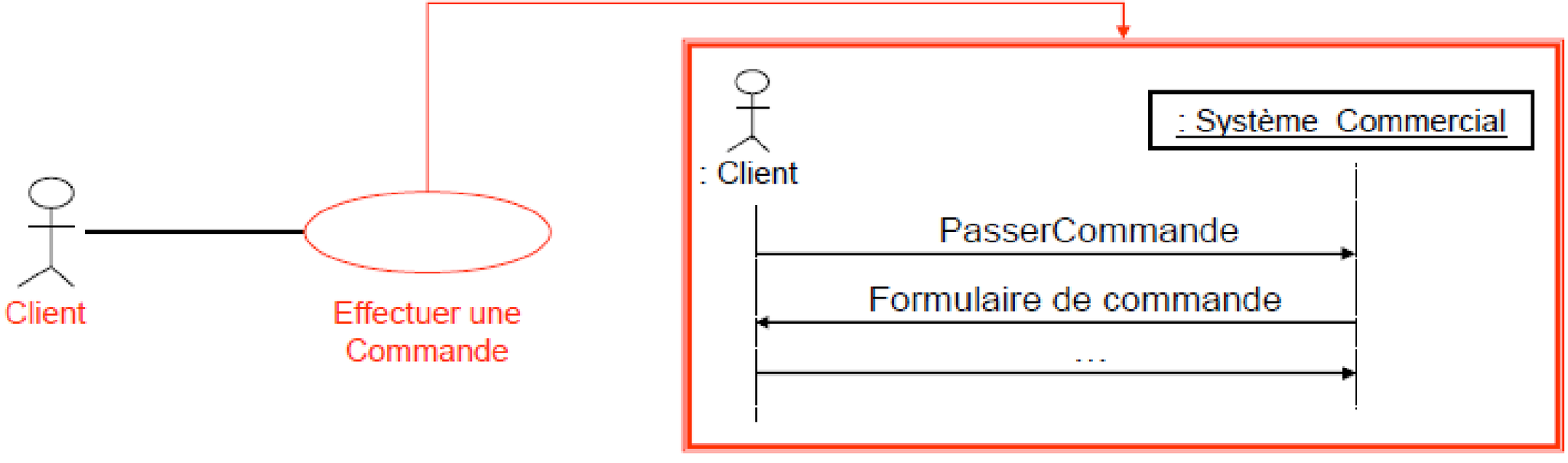
Cas d'utilisation(Description textuelle)

description des enchaînements

- **Pré-conditions** : Ce qui doit être vérifié avant que le Cas d'utilisation ne commence.
- **Des scénarii** : Description pas a pas de la conversation entre le cas d'utilisation et l'utilisateur. On trouve Le scénario nominal, les scénarii alternatifs qui sont les variantes du scénario nominal et les scénarii d'exception qui décrivent les cas d'erreurs.
- **Exceptions** : les traitements alternatifs ou d'exception
- **Post-conditions** : ce qui est vrai après que le Cas d'utilisation se soit déroulé.

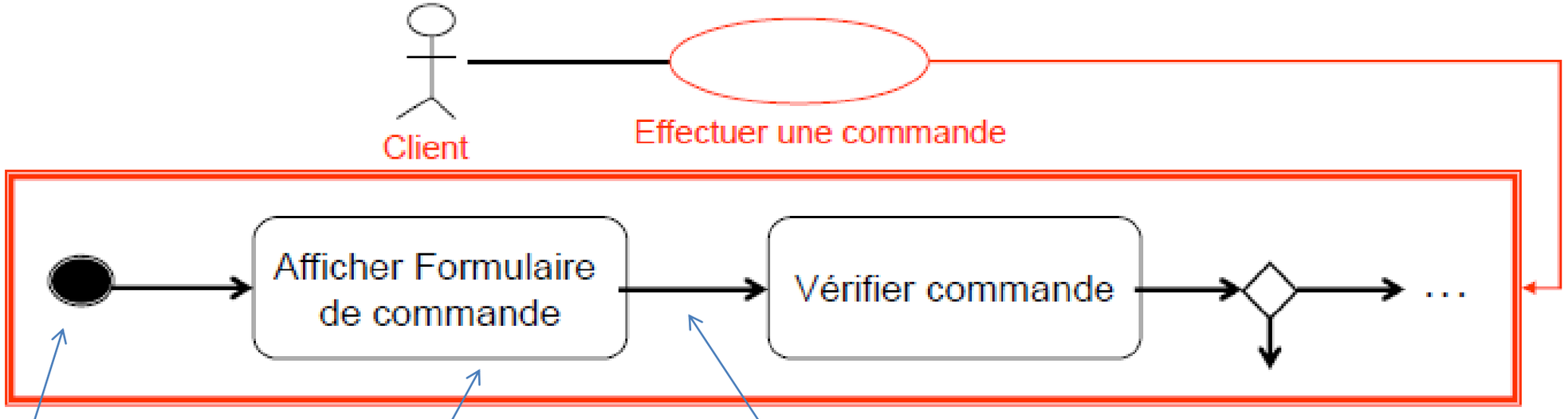
Cas d'utilisation(Description)

Compléter la description d'un cas d'utilisation par un diagramme de séquence Système.



Cas d'utilisation(Description)

Compléter la description d'un cas d'utilisation par un diagramme d'activités



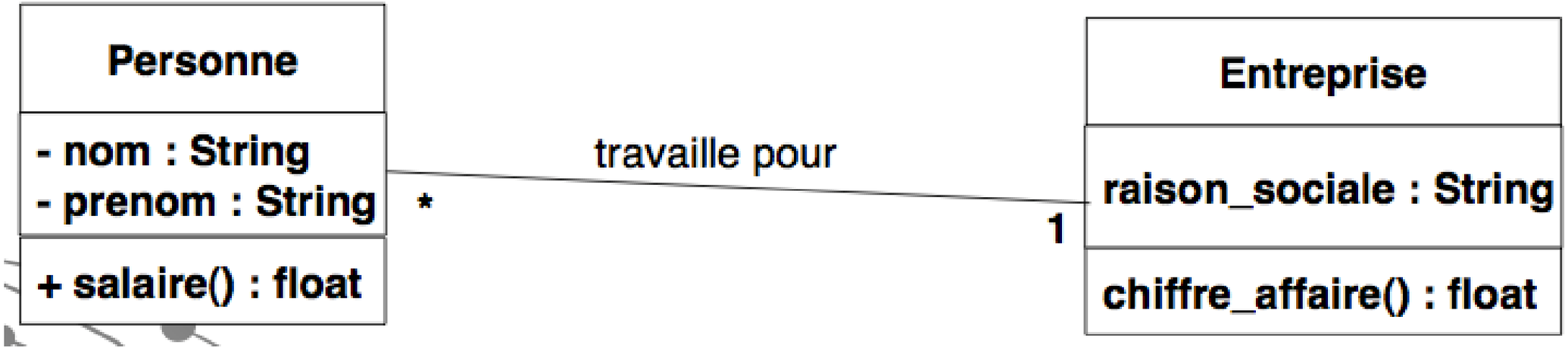
Point de départ

Activité

Transition

Diagrammes des classes(Axe Statique)

Alors que le diagramme de cas d'utilisation montre le système du point de vue des acteurs, le diagramme de classes en montre la structure interne. Il représente une description purement statique du système.



Diagrammes des classes(Axe Statique)

Alors que le diagramme de cas d'utilisation montre le système du point de vue des acteurs, le diagramme de classes en montre la structure interne. Il représente une description purement statique du système.

Diagramme de classes d'analyse

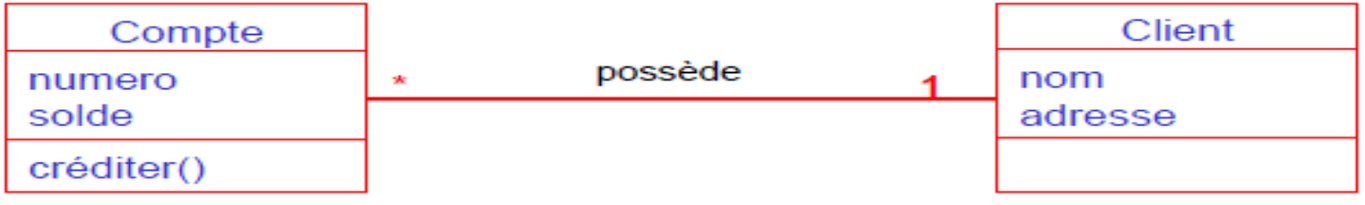
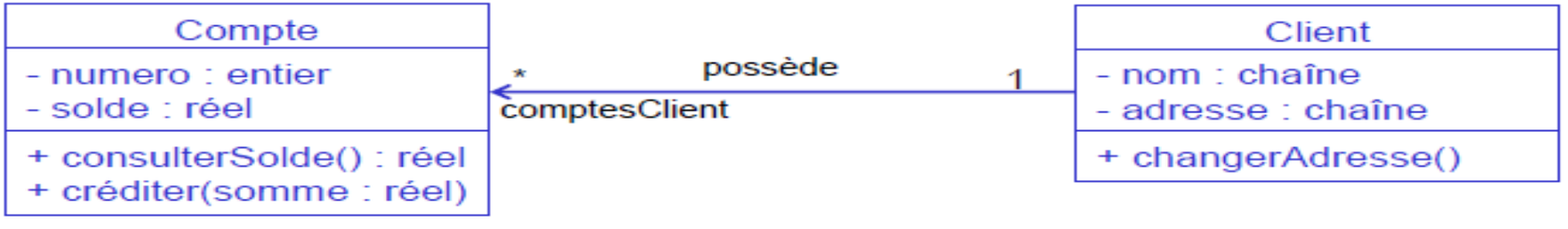
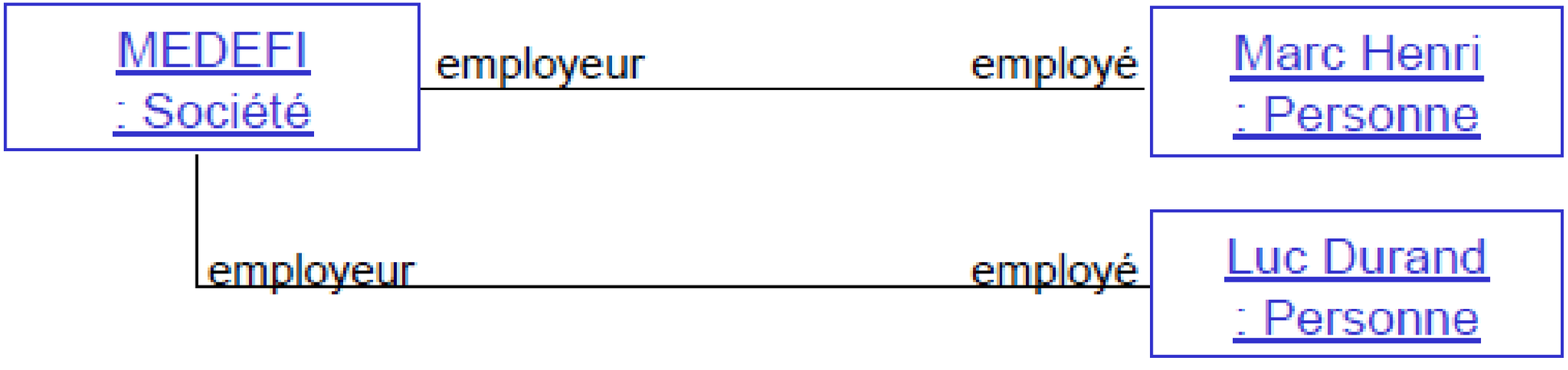


Diagramme de classes de conception



Diagrammes des objets(Axe Statique)

Le diagramme d'objets sert à illustrer des structures de classes compliquées. Il est utilisé en analyse pour vérifier l'adéquation d'un diagramme de classes à différents cas possibles.



Diagrammes des composants(Axe Statique)

Un diagramme de composants montre le découpage du système en unités (logiciels) pouvant être distribuées.

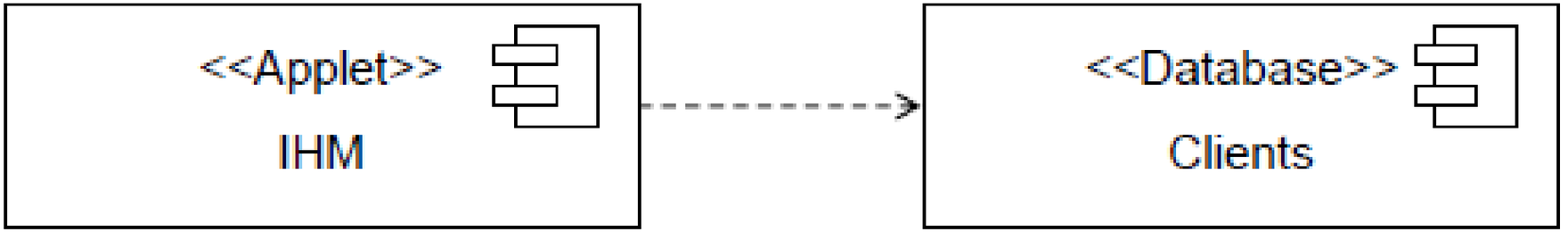


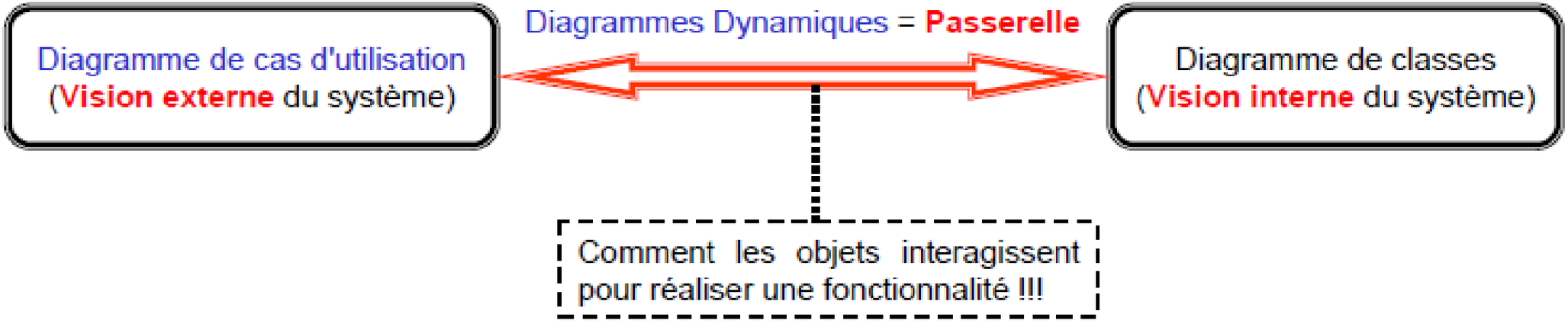
Diagramme de déploiement (Axe Statique)

Le diagramme de déploiement représente à la fois :
(1) la structure du système informatique qui prend en charge le système logiciel,
(2) et la façons dont les composants y sont installés.



Les Diagrammes UML (Axe Dynamique)

Diagrammes dynamiques : passerelle entre vision externe et interne.



Les Diagrammes d'interaction (Axe Dynamique)

Les diagrammes de séquence et de communication sont appelés diagrammes d'interaction. Ils représentent les échanges de messages entre objets du système, dans le cadre d'un fonctionnement particulier (scénario) du système.
Ils peuvent être notamment utilisés en conception, pour définir et concevoir les méthodes des classes.

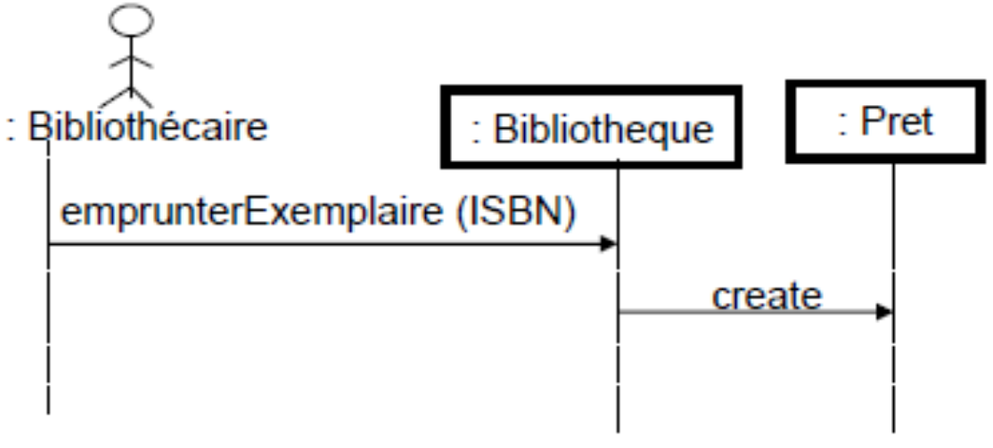


diagramme de séquence

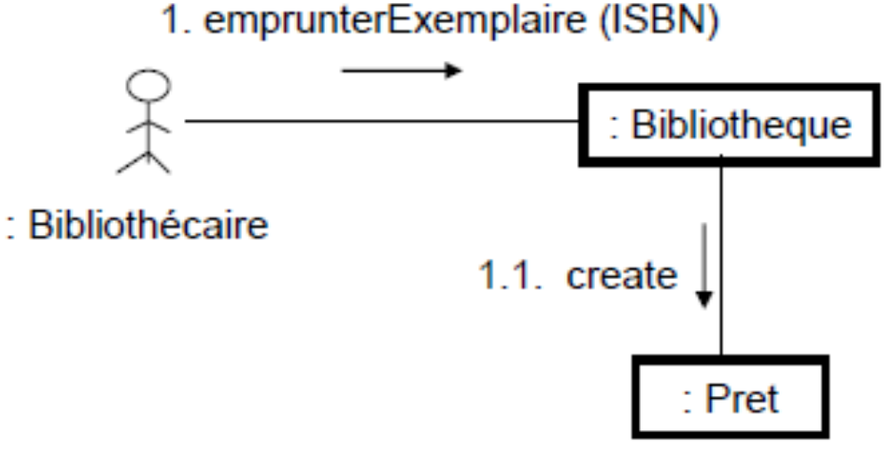
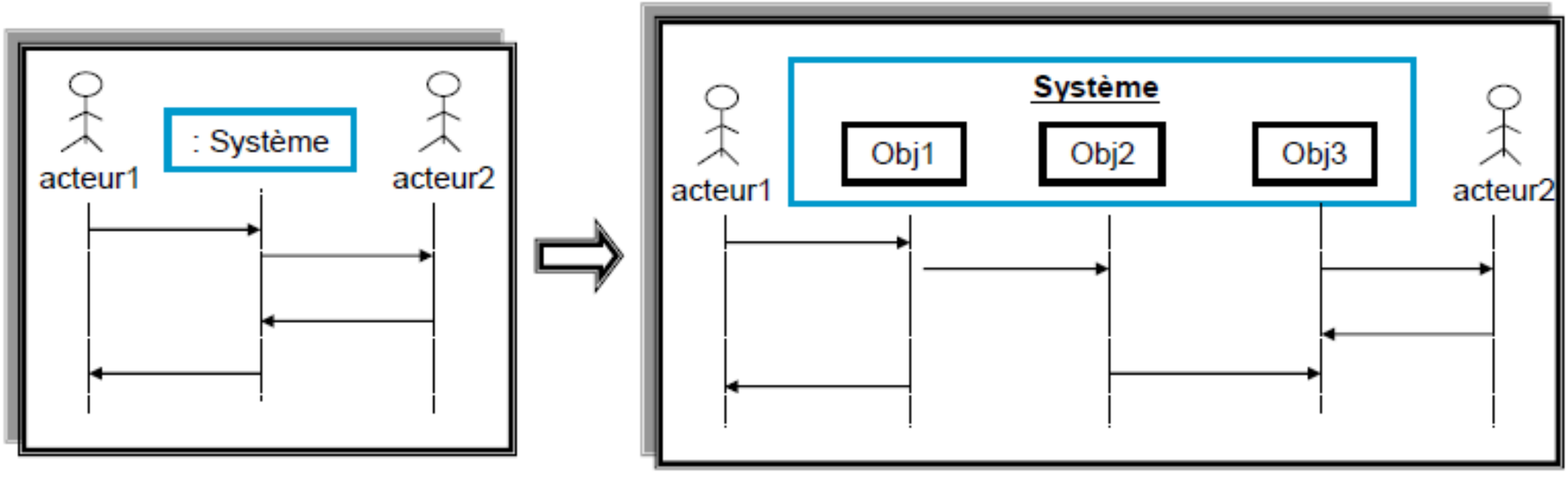


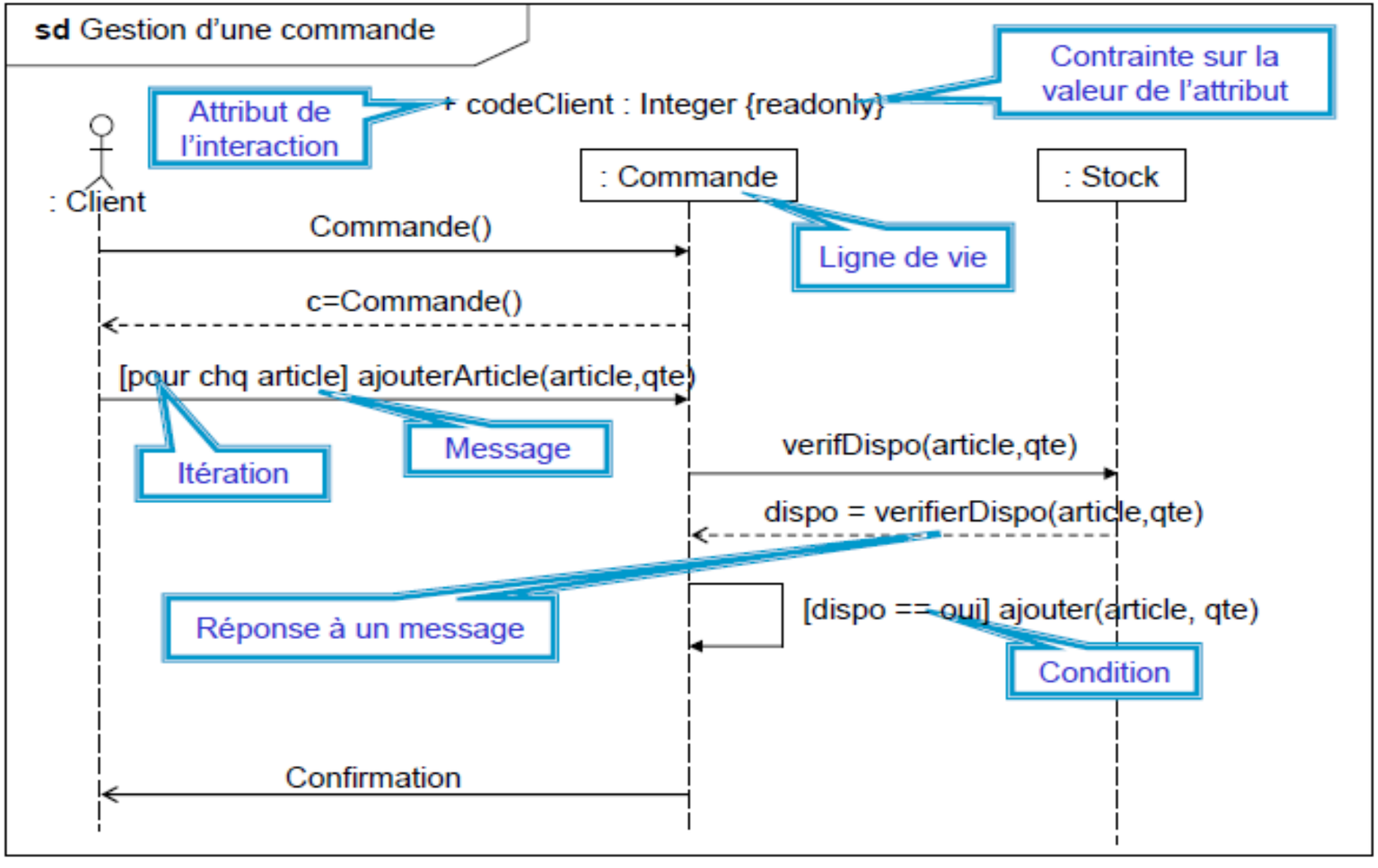
diagramme de communication

Le Diagramme de Séquence (Axe Dynamique)

La construction d'un diagramme de séquence revient à remplacer dans le Diagramme de Séquence Système le
système (la boîte noire) par une collaboration d'objets.



Le Diagramme de Séquence (Axe Dynamique)



Le Diagramme de Communication (Axe Dynamique)

Le diagramme de communication permet donc de représenter les relations structurelles entre les objets qui communiquent.

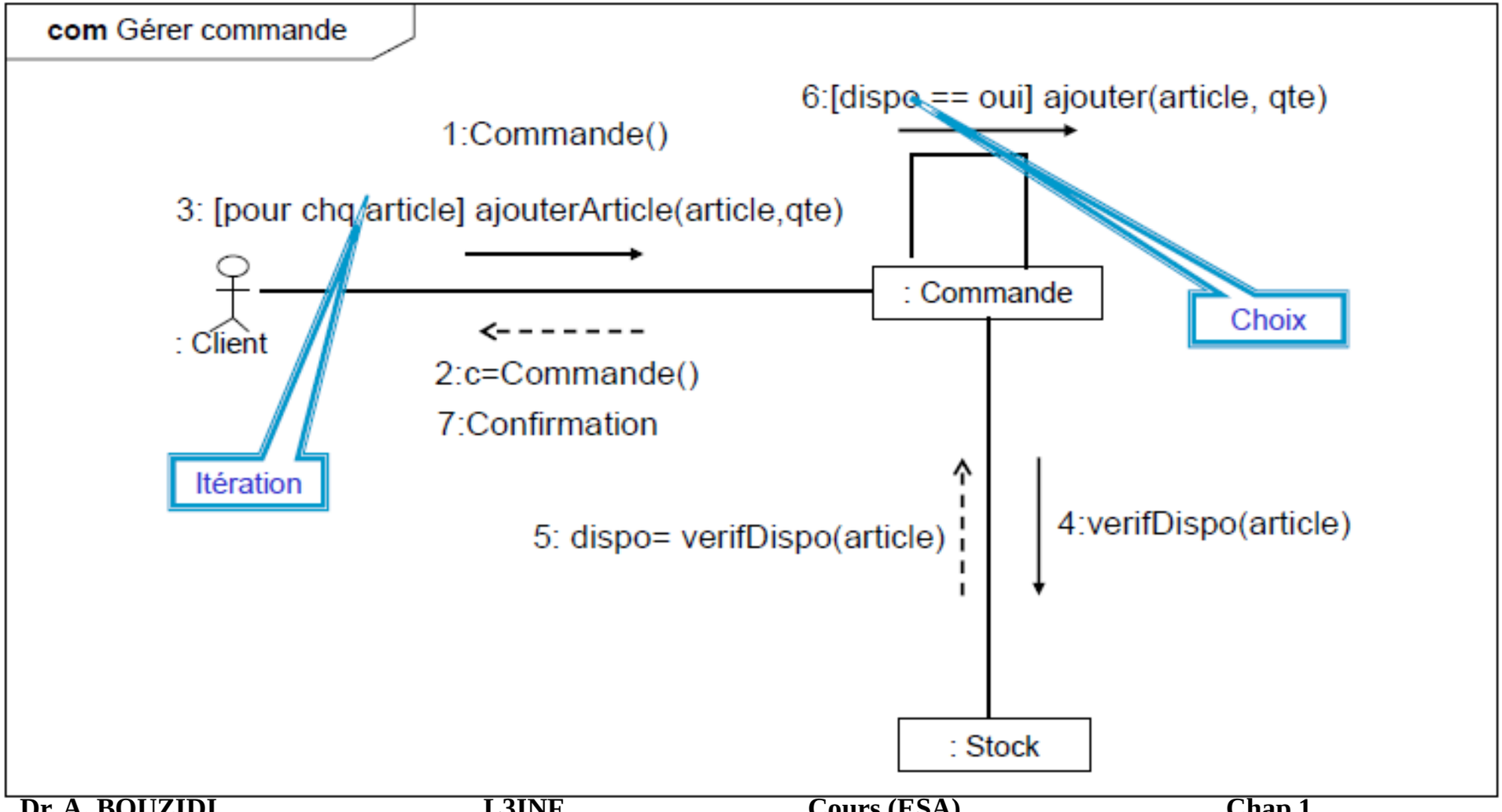


Diagramme d'états (Axe Dynamique)

Les diagrammes d'interaction (séquence et communication) modélisent les interactions entre les objets du système; le diagramme d'états modélise les effets de ces
interactions sur la configuration interne des objets.

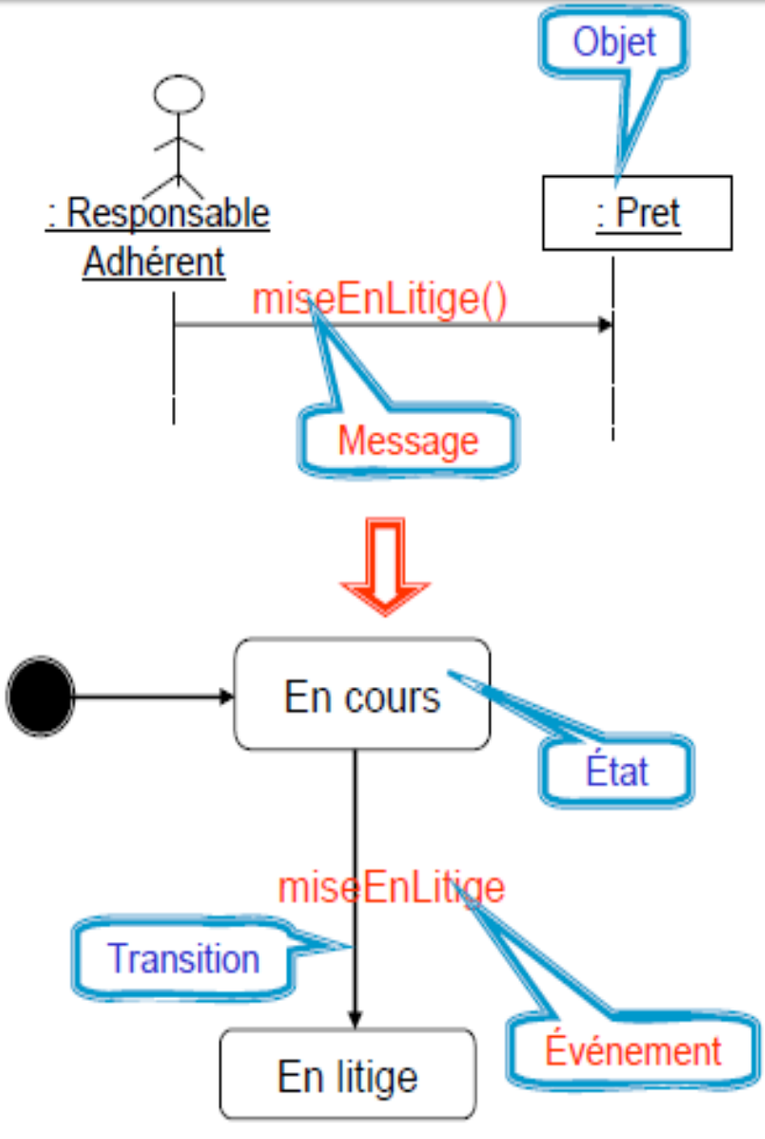


Diagramme d'états (Axe Dynamique)

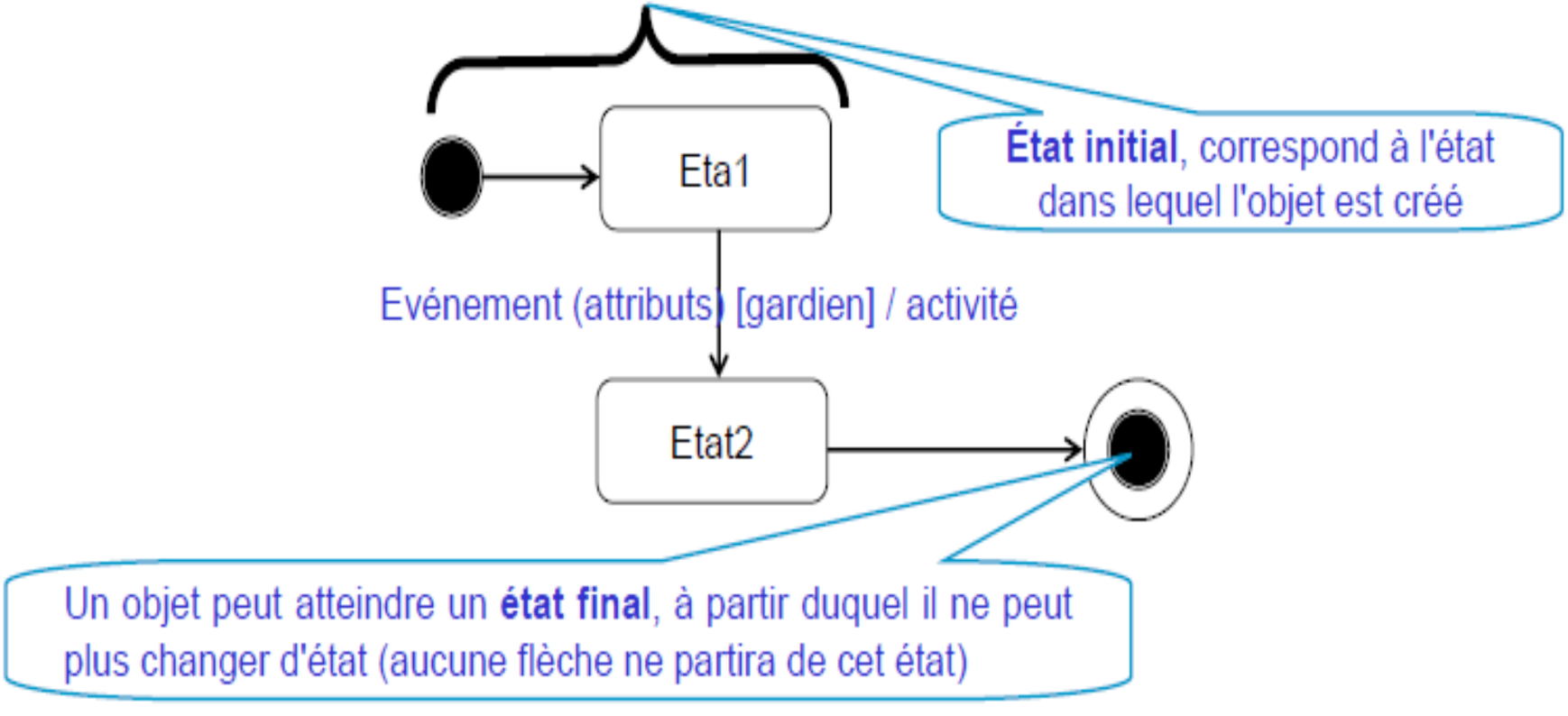
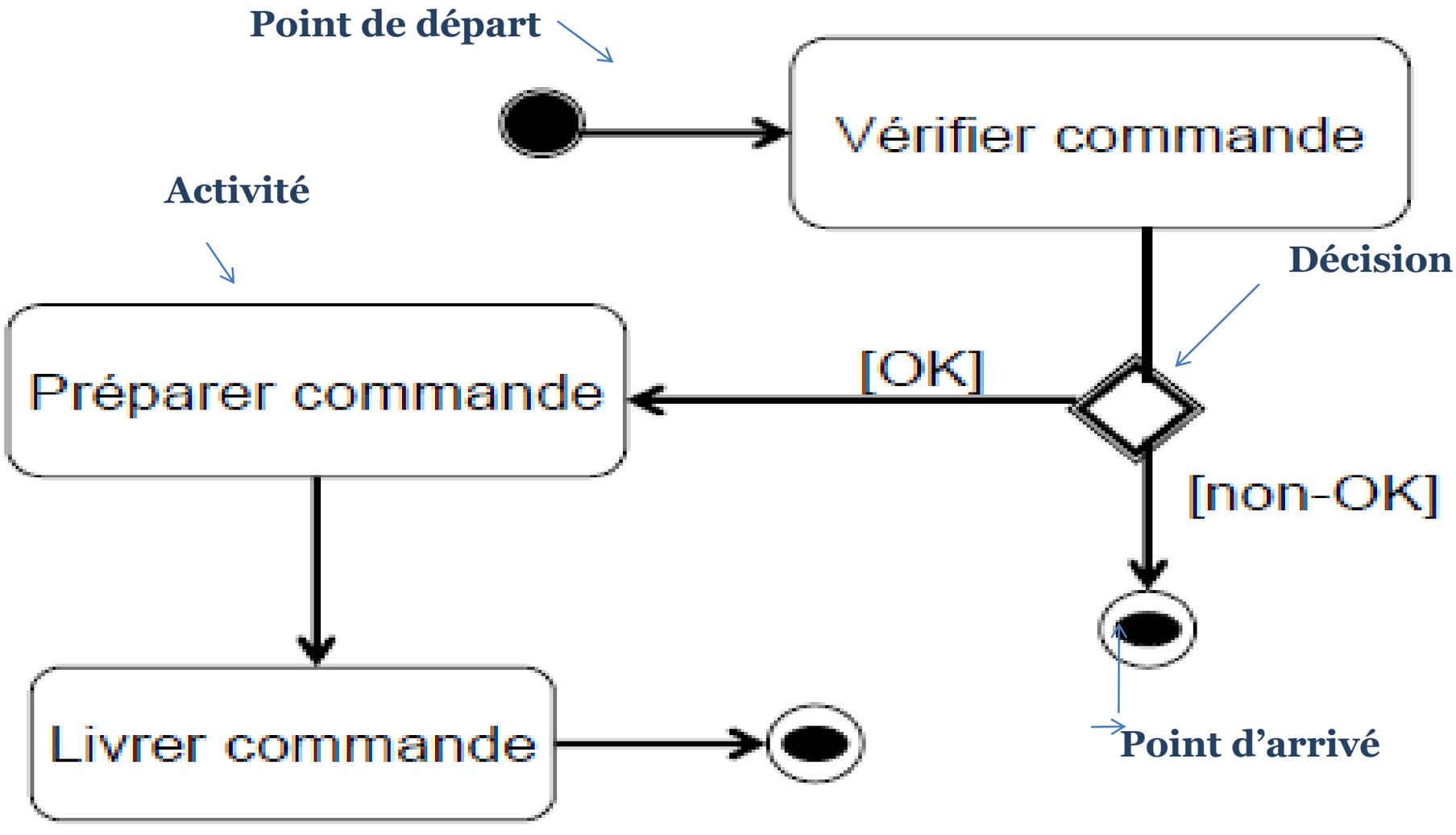
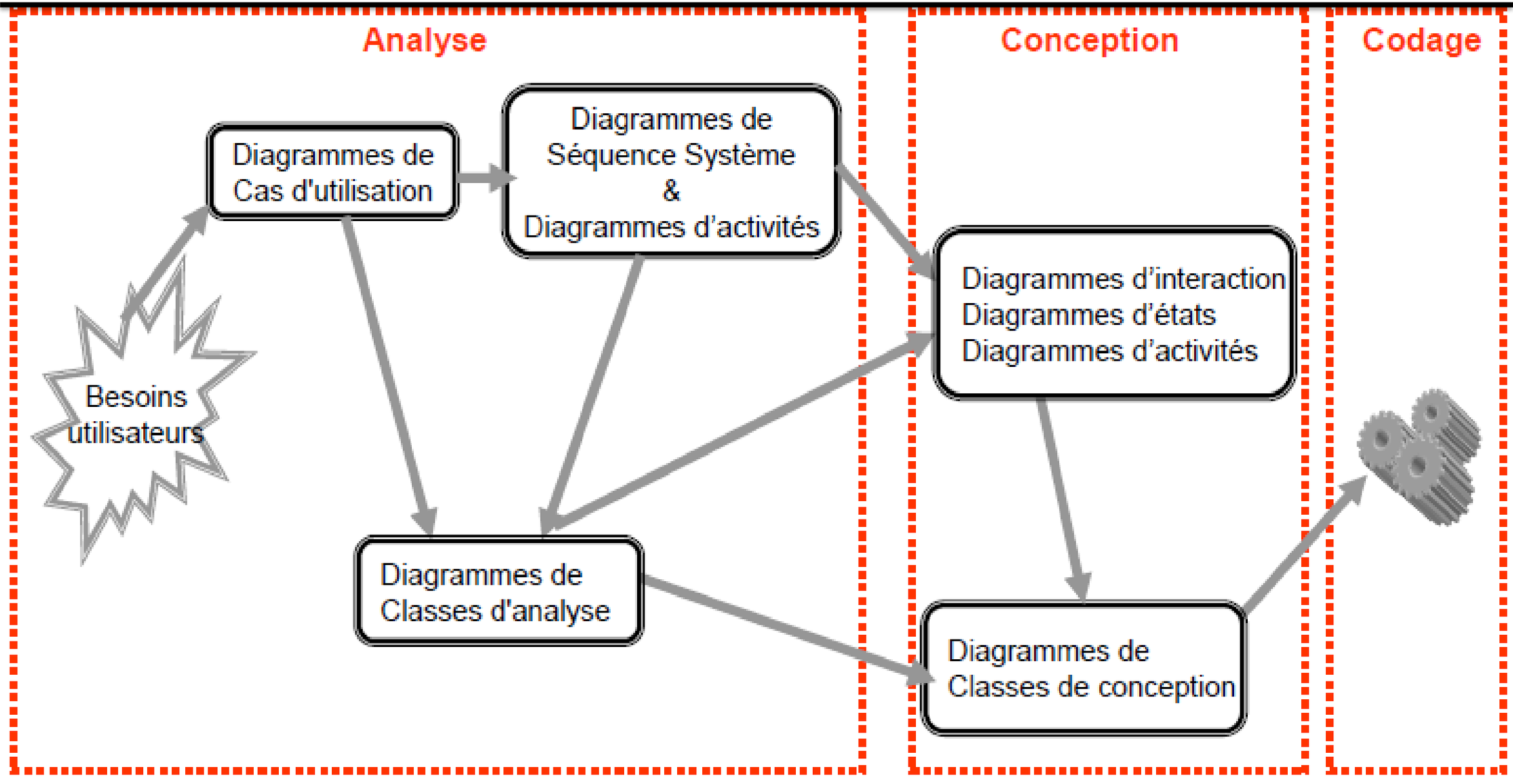


Diagramme d'activités (Axe Dynamique)



Vue d'ensemble des diagrammes UML



Utilisation de UML dans l'industrie

Modèle	Niveau d'utilisation
Classe et package	**5 Le plus utilisé, le plus essentiel.**
Use case	**3 Dépend des cultures. Il y a des domaines où cela a peu d'intérêt.**
Séquence	**3 Assez employé, bien compris.**
Collaboration	**2 Les diagrammes d'objet sont utilisés, mais la modélisation des collaboration et des rôles est encore marginale.**
Etat/transition	**2 Très utilisé dans le temps réel**
Activité	**1 Arrivée tardive dans les ateliers, habitude culturelle locale à certaines parties du tertiaire.**
Composant/Déploiement	**1 Peu de personnes l'utilisent, sur peu de parties de leurs applications.**



Partie 1: Modèle Conceptuel de Données (MCD) et Modèle Logique de Données (MLD)

a) Les concepts de base

Entité : tout objet concret ou abstrait ayant une existence propre et conforme aux besoins de gestion de l'organisation.

Ex: le client «Dupond», le produit de référence «a456», …

Classe d'entités (ou entité-type) : ensemble des entités décrites par les mêmes caractéristiques.

Ex: la classe CLIENT dont «Dupond» est une occurrence (ou instance).

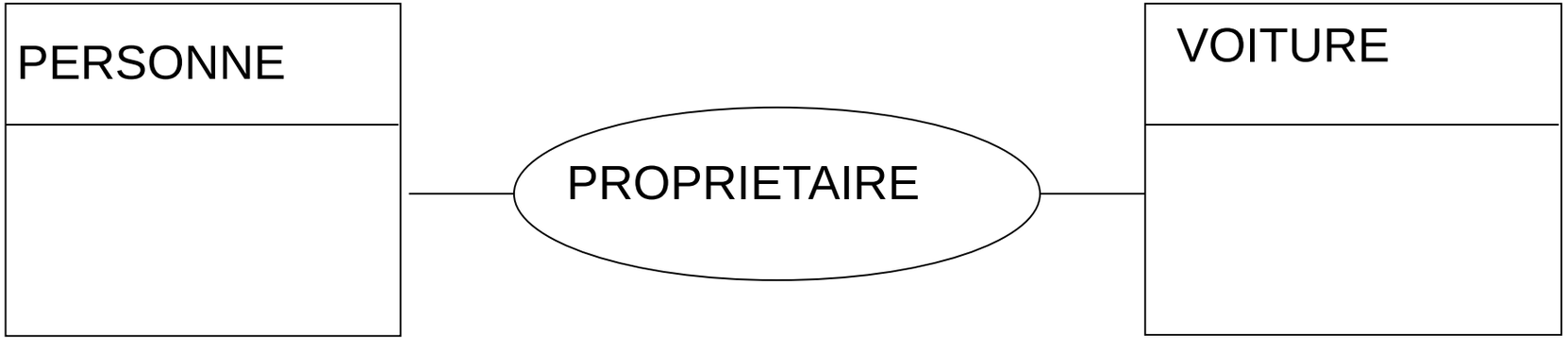
Association : n-uplet d'entités « sémantiquement liées ».

Ex: («Dupond», «1367 VS 54») indiquant que la personne Dupond est propriétaire de la voiture immatriculée 1367 VS 54.

Classe d'associations (ou association-type) : regroupe toutes les associations constituées des mêmes types d'entités jouant le même rôle dans l'association.

Ex: PROPRIETAIRE(PERSONNE, VOITURE)

Les occurrences de cette classe d'association sont un sous ensemble du produit cartésien PERSONNE x VOITURE (c.à.d. une partie de l'ensemble des couples possibles de personnes et de voitures).



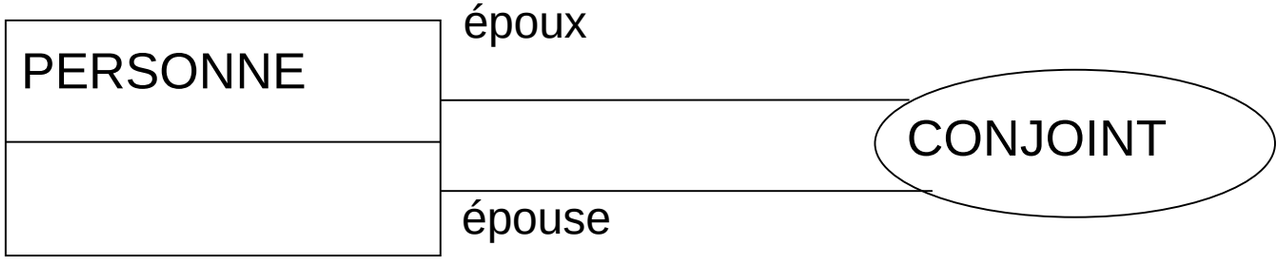
Remarques

- On peut avoir plusieurs classes d'associations sur les mêmes classes d'entités.

  Ex : PROPRIETAIRE(PERSONNE, VOITURE)
  et CONDUIRE(PERSONNE, VOITURE)

- On peut avoir une classe d'association sur une seule classe d'entités (on parle d'association 'réflexive'). On ajoute souvent dans ce cas des noms de **rôles** pour distinguer les deux occurrences.

  Ex : CONJOINT(PERSONNE, PERSONNE)

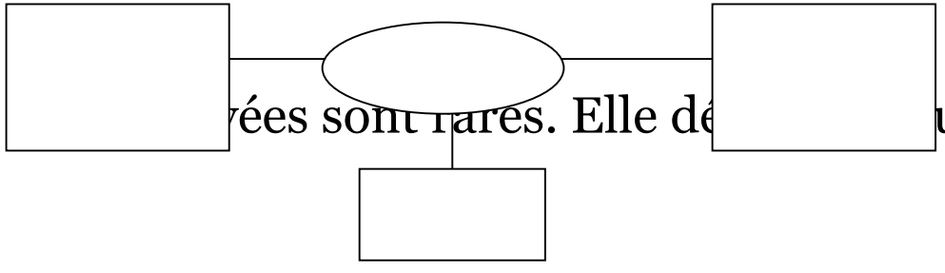


MCD

- On peut avoir une classe d'association définie sur n classes d'entités (association n-aire ou d'arité n ou de dimension n ou à « n pattes »).

  Ex: COURS(MATIERE, CLASSE, PROF)

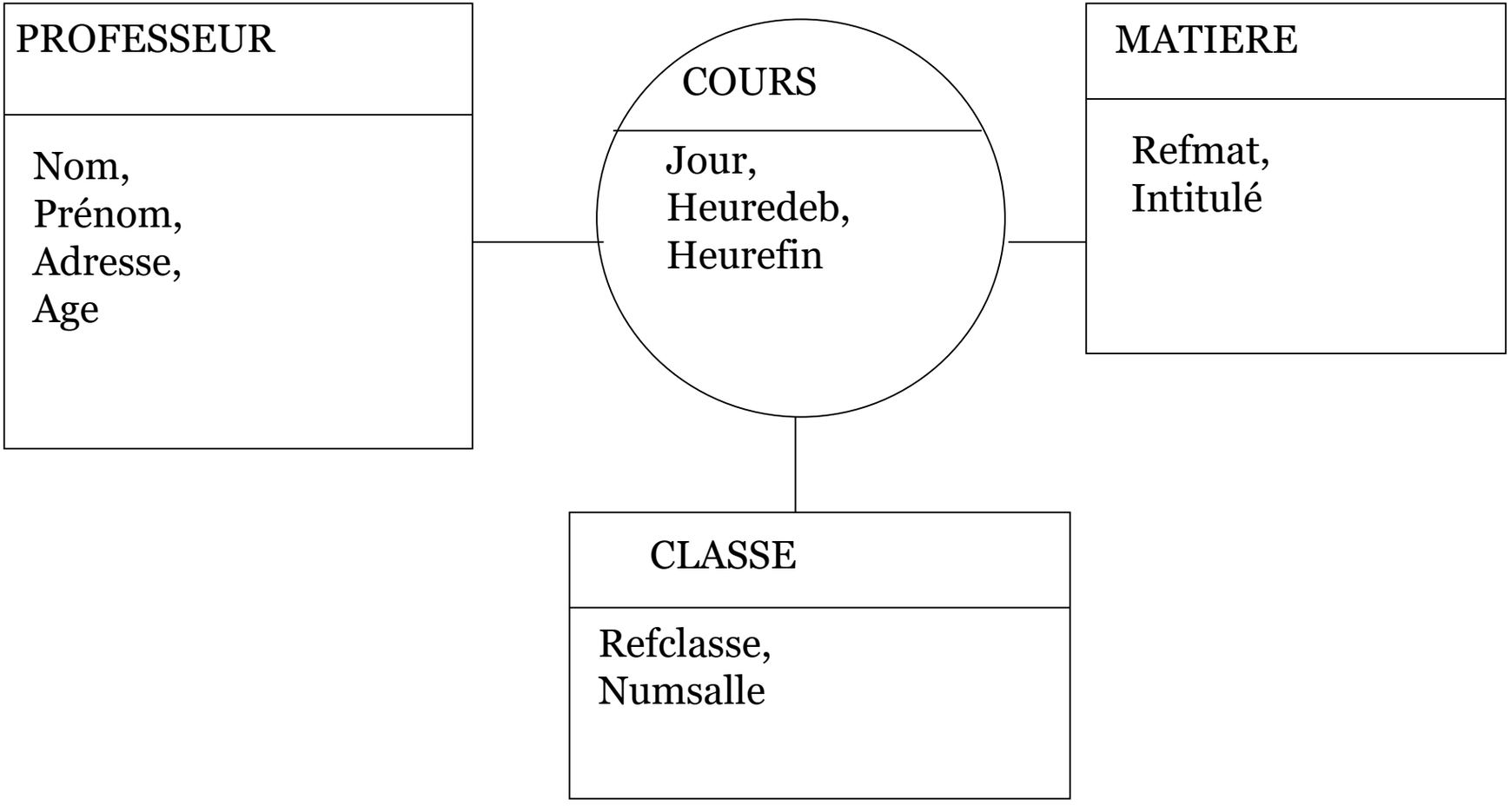
Attention : les [illegible]vées sont rares. Elle dé[illegible]uvent des faiblesses dans l'analyse.



arité 2 : 80%
arité 3 : <20%
arité > 3 : ε

Propriété : donnée élémentaire permettant de caractériser les entités et associations

Ex : nom, prénom, adresse propriétés de PERSONNES



MCD

Identifiant : propriété ou groupe de propriétés permettant d'identifier de manière unique chaque occurrence de la classe d'entités.

Ex : N° immatriculation pour VOITURE. Nom ne suffit pas pour PERSONNE. N° Client pour CLIENT (propriété ajoutée)

Les identifiants sont en général soulignés.

Cardinalités : indiquent pour chaque classe d'entités de la classe d'association, les nombres mini et maxi d'occurrences de l'association pouvant exister pour une occurrence de l'entité.

La cardinalité minimum est 0 ou 1.

La cardinalité maximum est 1 ou n.

Une cardinalité minimum à 0 signifie qu'il est possible d'observer (un jour) une occurrence d'entité sans occurrence d'association.

Donc 4 combinaisons possibles :

0,1	au plus 1
1,1	1 et 1 seul
1,n	au moins 1
0,n	un nombre quelconque

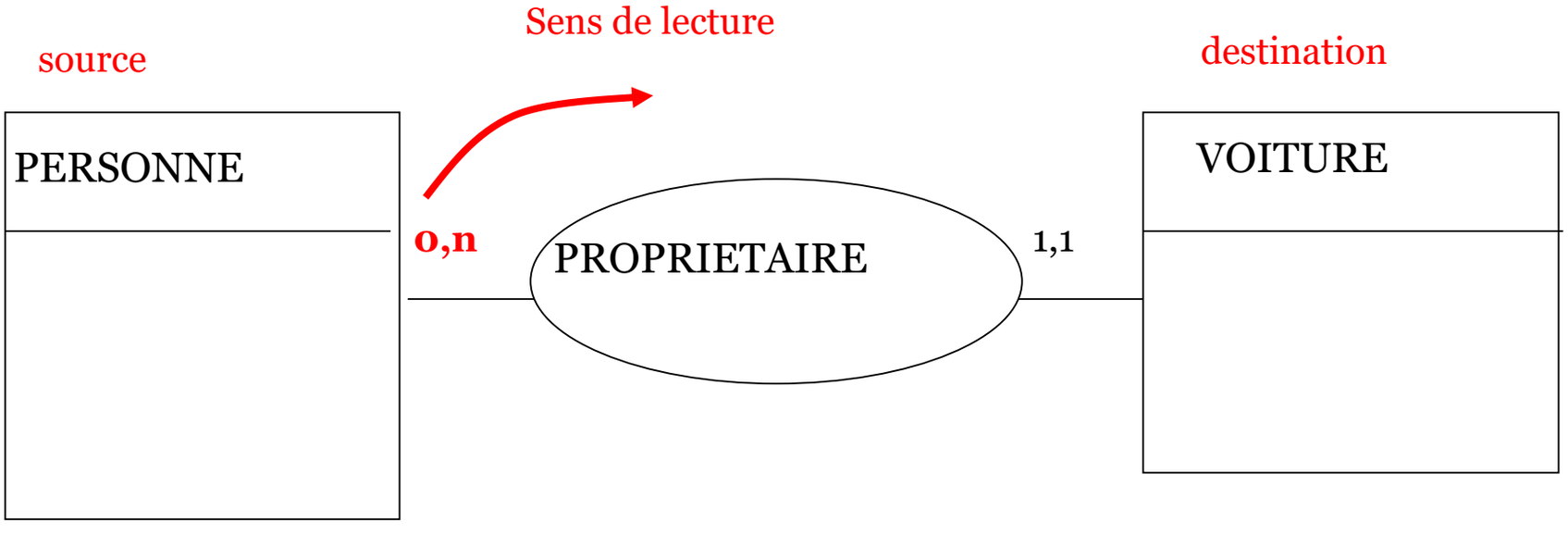
Ex: PROPRIETAIRE(PERSONNE [0,n],
VOITURE [1,1])

Une personne a 0 à n voitures; une voiture a 1 et 1 seul propriétaire.

CONDUIT(PERSONNE [0,n], VOITURE [1,1])

Une personne conduit 0 à n voitures; une voiture est conduite par 1 à 1 personnes.

Représentation graphique :

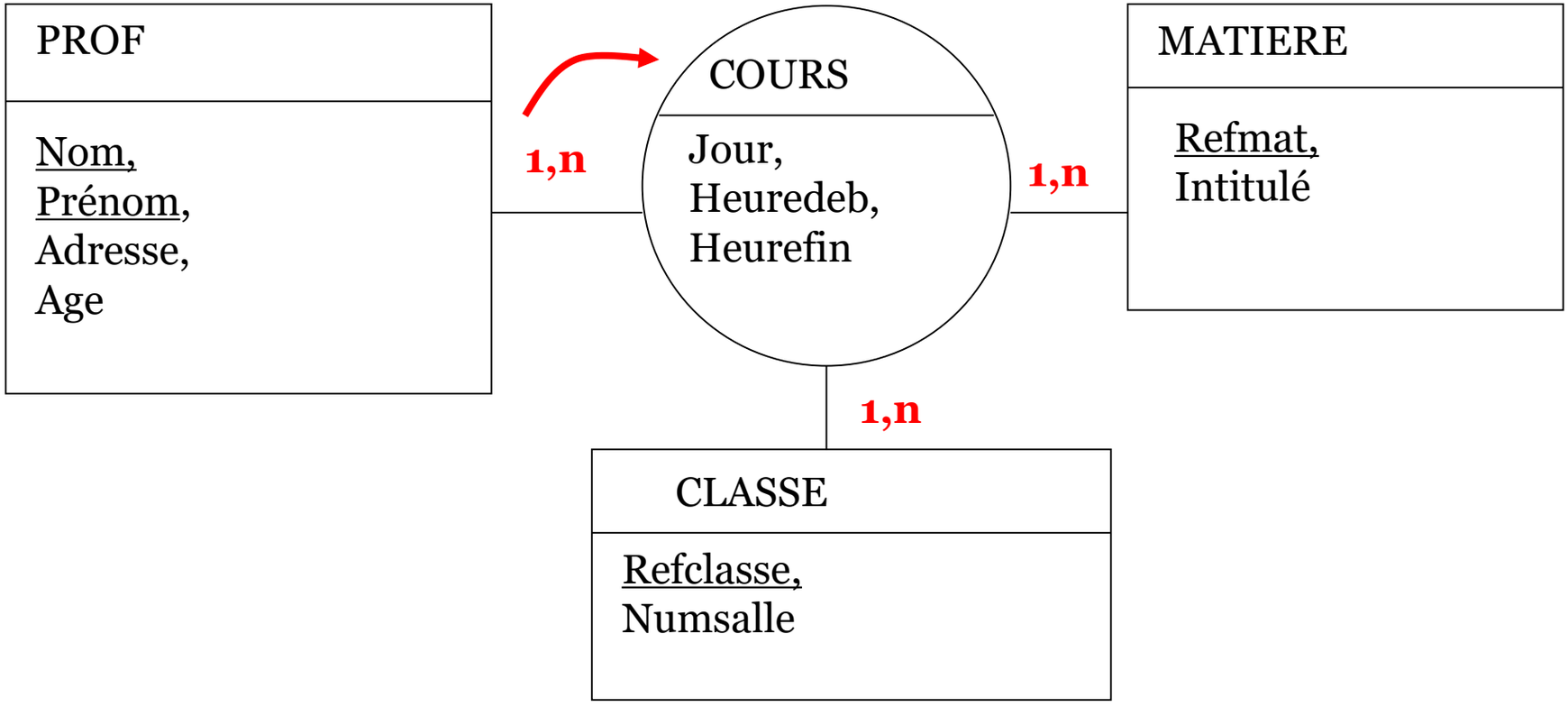


!! Dans les méthodes anglo-saxonnes la cardinalité est placée du côté opposé à l'entité source !!

MCD

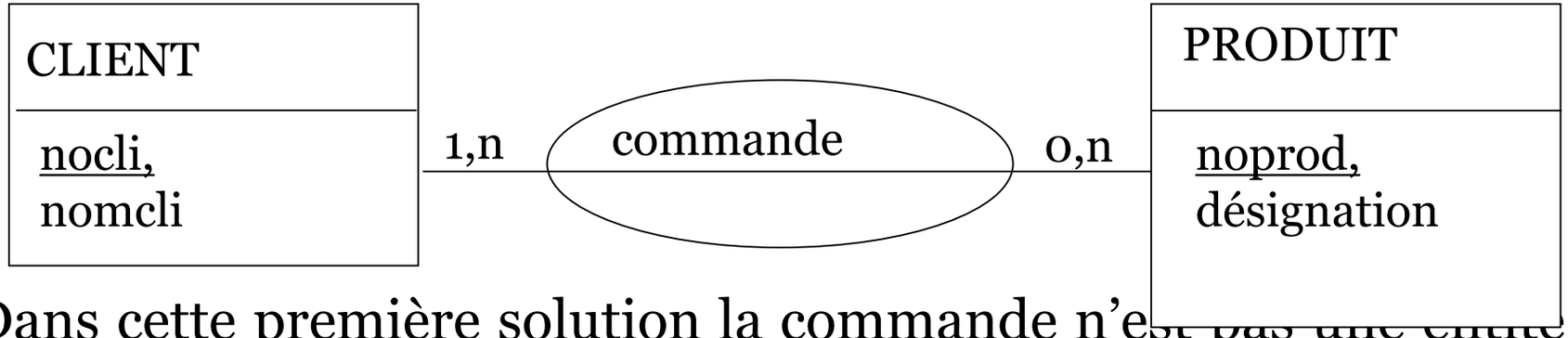
COURS(MATIERE [1,n], CLASSE [1,n], PROF[1,n])

Un prof. a 1 à n cours dans la semaine, une matière a 1 à n cours dans la semaine, une classe a 1 à n cours dans la semaine.



Difficultés : choix entre entité et association ?

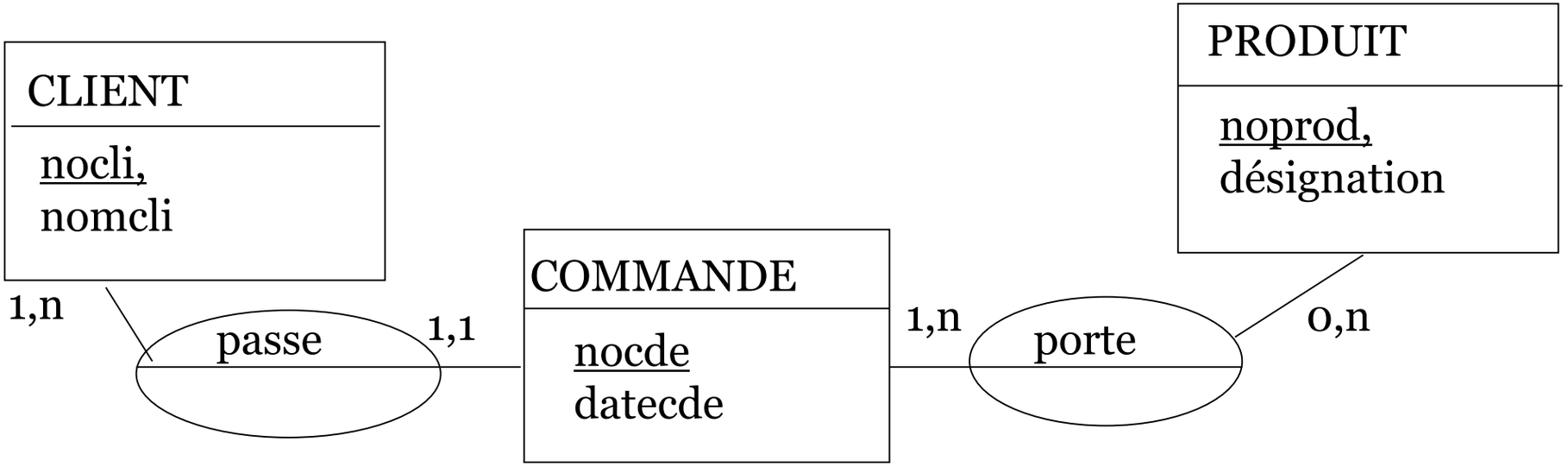
1) Solution avec association



Dans cette première solution la commande n'est pas une entité gérée pour elle même. Elle existe tant que le client et le produit existent.

Ce peut être le SI du domaine 'fabrication' : on a juste besoin de savoir que les produits sont destinés à des clients.

2) Solution avec entité



Dans cette seconde solution, les commandes sont identifiées (identifiant nocde) et décrites : on les gère en tant que telles.

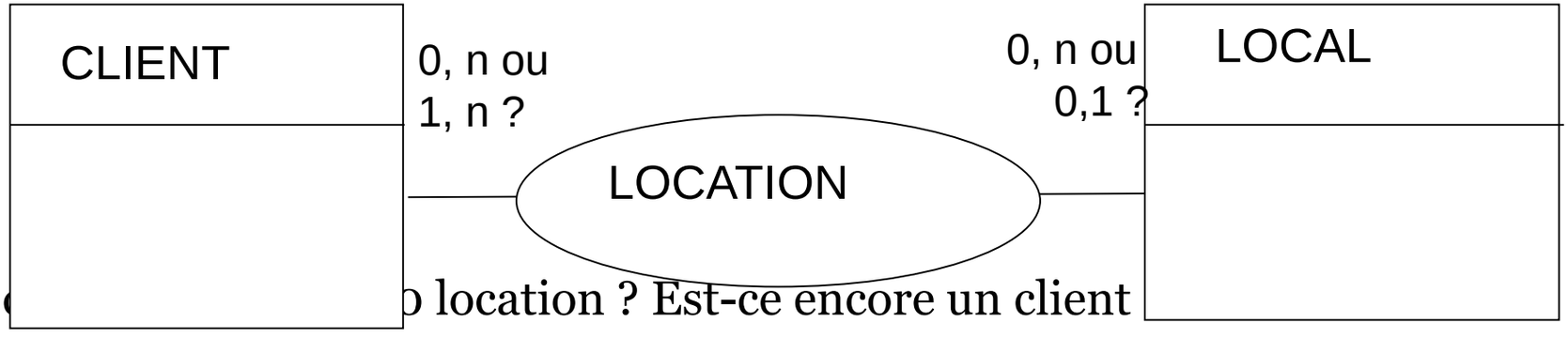
Ce peut être le SI du domaine financier.

MCD

Quelques ‘critères’ de choix :

- Une entité a une **existence propre** et un **identifiant**.
- Une association n’existe que si ses extrémités existent et n’a pas d’identifiant explicite.
- Une entité peut être associée à d’autres entités, une association non.

Difficultés : choix des cardinalités



Un client peut il louer plusieurs fois le même local à des périodes différentes ? Oui si on gère un historique des locations. Non si on ne gère que la location ? Est-ce encore un client

Un local peut il être loué plusieurs fois ? Non si la base représente une situation instantanée et si le local n'est pas partageable. Oui si on gère un historique ou si le local est partageable.

Les cardinalités sont élément essentiel pour définir la sémantique des données, pas une « décoration » accessoire. Derrière cette notion on trouvera des contrôles (par le SGBD ou les programmes).

b) Vérification et Normalisation

Contrôler la qualité du modèle vis-à-vis :

- des fondements du modèle d'une part (règles de vérification),
- de la redondance de données d'autre part (règles de normalisation) .

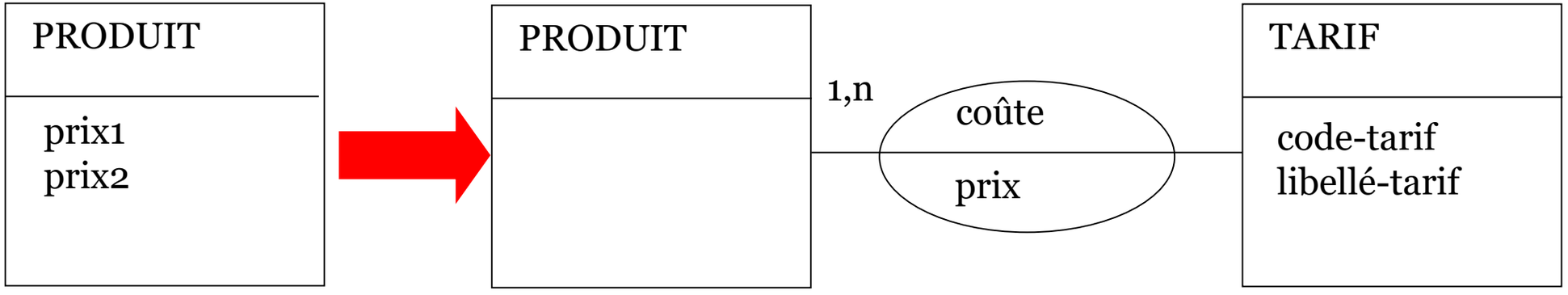
Permet de détecter certaines incohérences dans la construction des modèles.

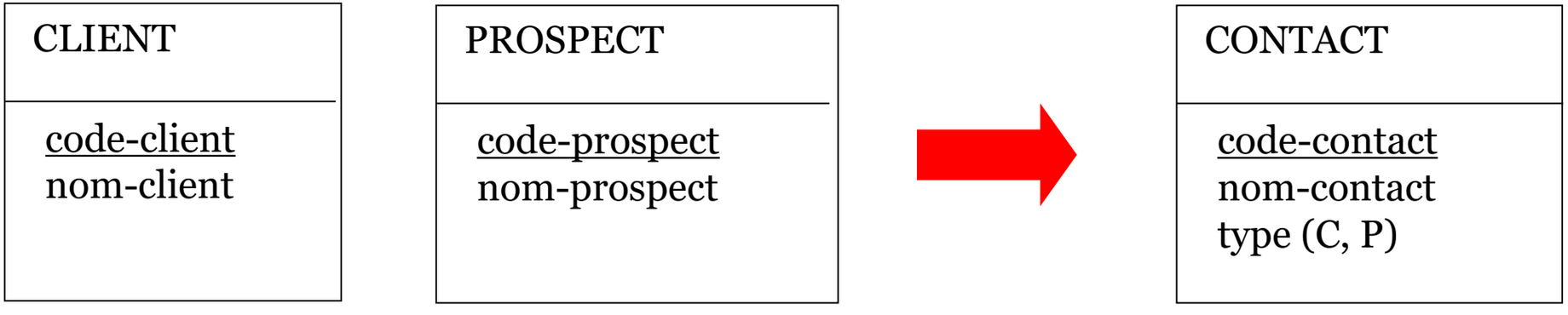
1. Règles Générales

- Toute propriété doit apparaître une seule fois dans un modèle.

  Il faut éliminer la redondance des propriétés dans la même entité (avec des noms différents) ou dans des entités distinctes :

MCD





- Pas d'héritage dans le modèle E/A de base !
- Toutes les propriétés identifiées doivent apparaître dans le modèle.

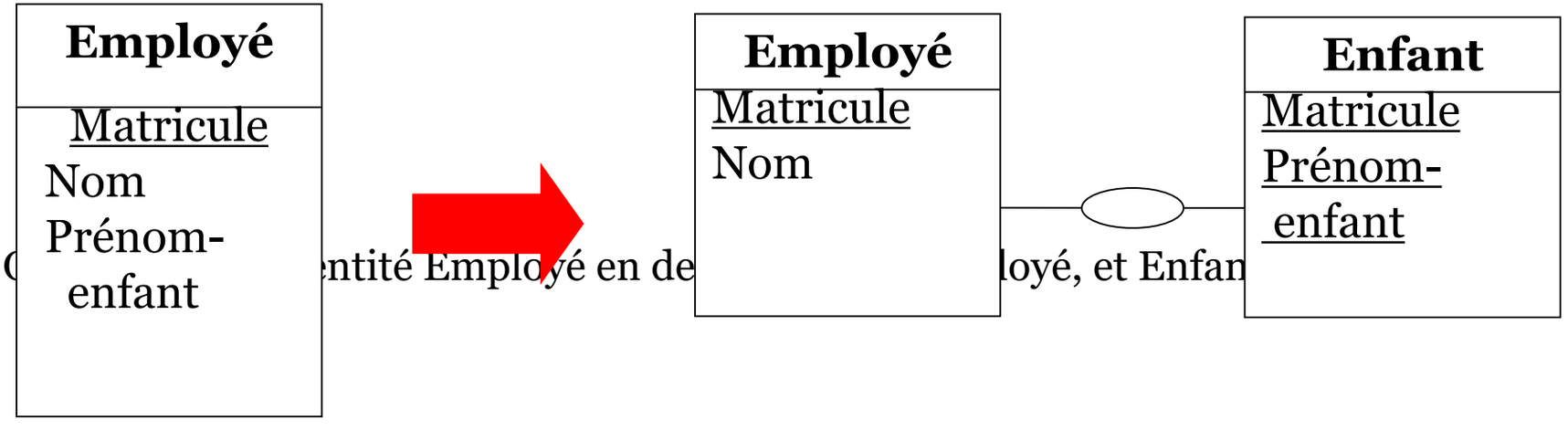
2. Règles sur les entités

2.a ***Règle de l'identifiant***

Toutes les entités ont un identifiant.

2.b ***Règle de vérification des entités***

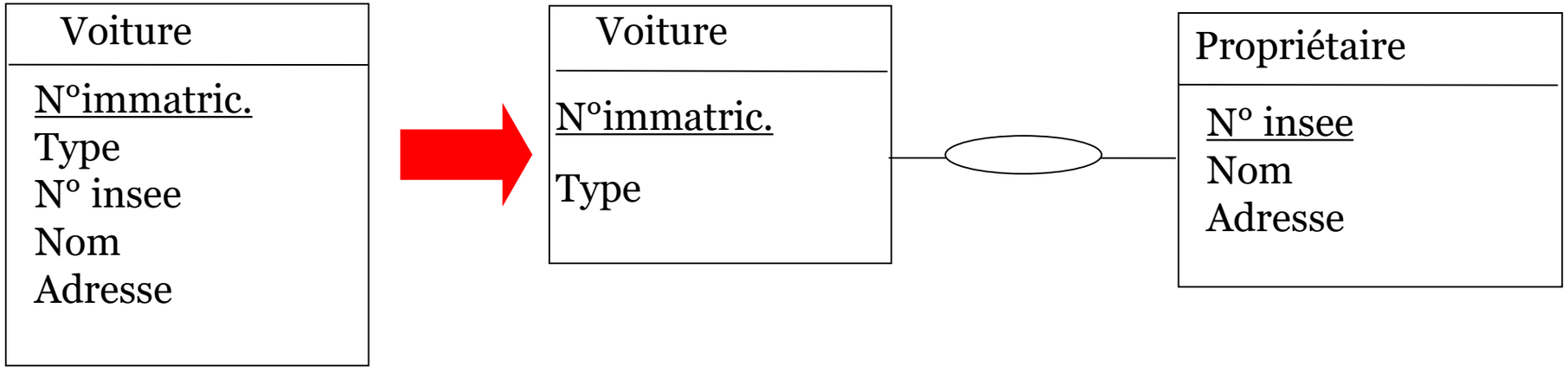
Pour une occurrence d'une entité, chaque propriété ne prend qu'une seule valeur (cf. la 1FN du modèle relationnel); MONO-VALUEE



2.c *Règles de normalisation des entités*

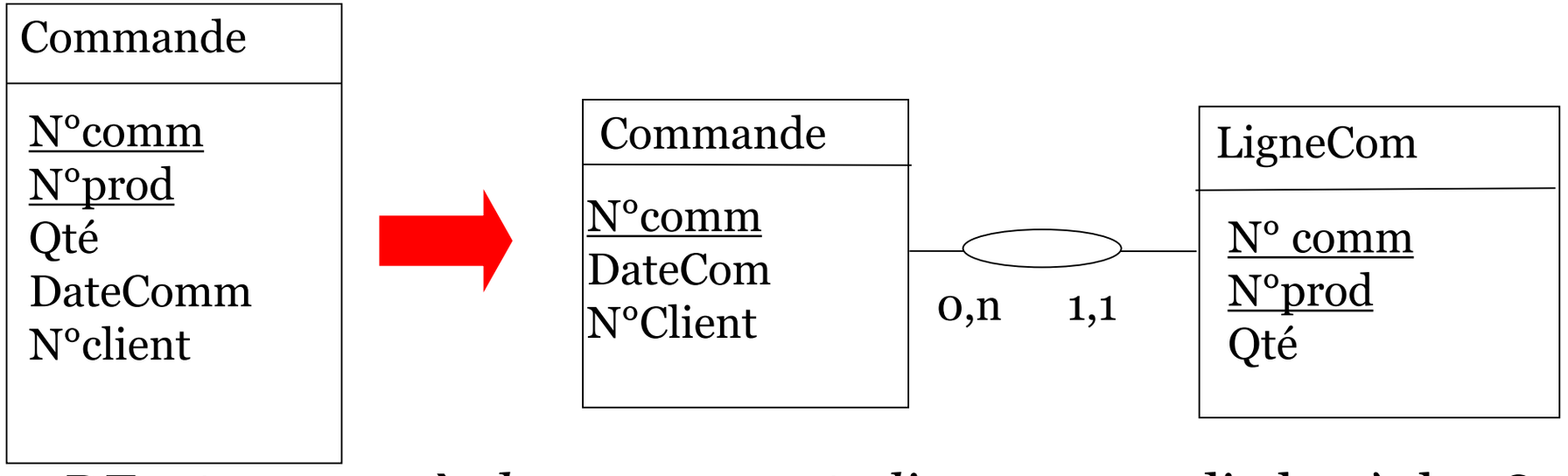
a) Les dépendances fonctionnelles (DF) entre les propriétés d'une entité doivent vérifier la règle suivante : **toutes** les propriétés de l'entité dépendent fonctionnellement de l'identifiant **et uniquement** de l'identifiant.

Rappel : ∃ une DF X→Y si à une valeur de X correspond une et une seule valeur de Y (réciproque pas vraie).



La DF: *N°insee → Nom, Adresse* contredit la règle.

b) Une partie de l'identifiant ne peut pas déterminer certaines propriétés.



La DF *n°-comm* → *date-comm, n°-client* contredit la règle. On décompose l'entité Commande en deux entités.

Ces règles correspondent aux 2FN et 3FN du modèle Relationnel (dépendance pleine et directe des clés).

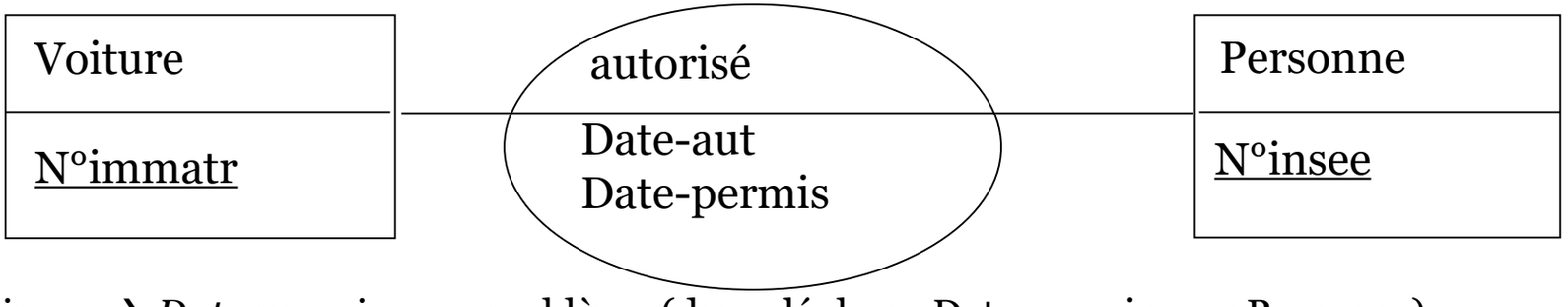
3. Règles sur les associations

3.a ***Règle de vérification des associations***

Pour une occurrence d'association, chaque propriété ne prend qu'une seule valeur.

3.b ***Règle de normalisation sur les propriétés des associations***

Toutes les propriétés de l'association doivent dépendre fonctionnellement de tous les identifiants des entités portant l'association, et uniquement d'eux.

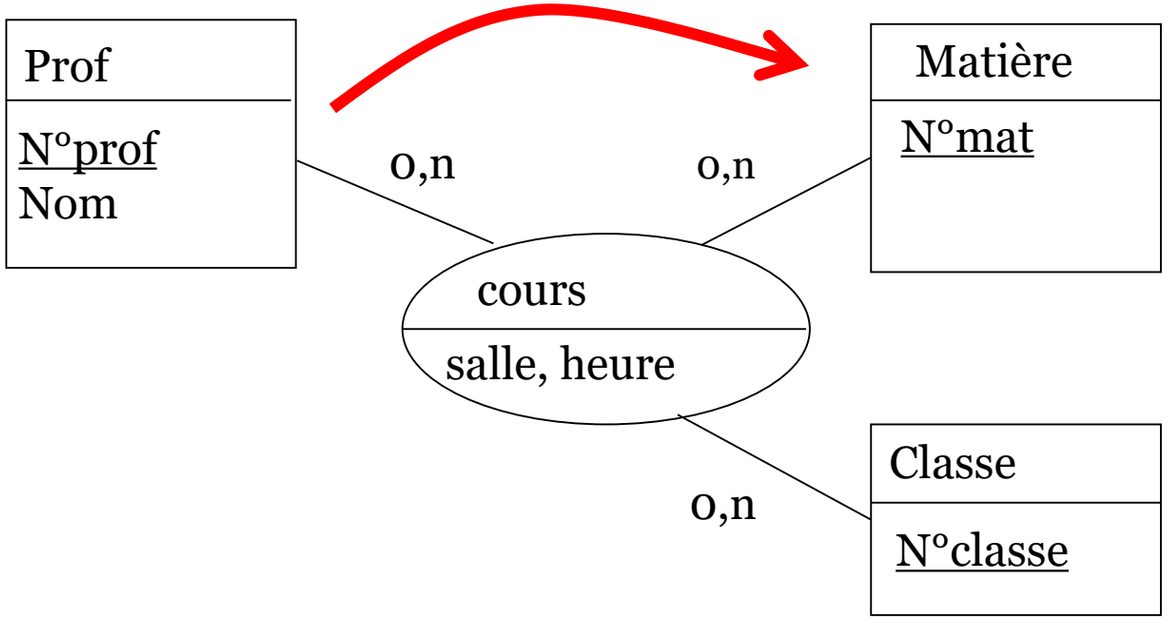


N°-insee → *Date-permis* pose problème (donc déplacer Date-permis vers Personne)

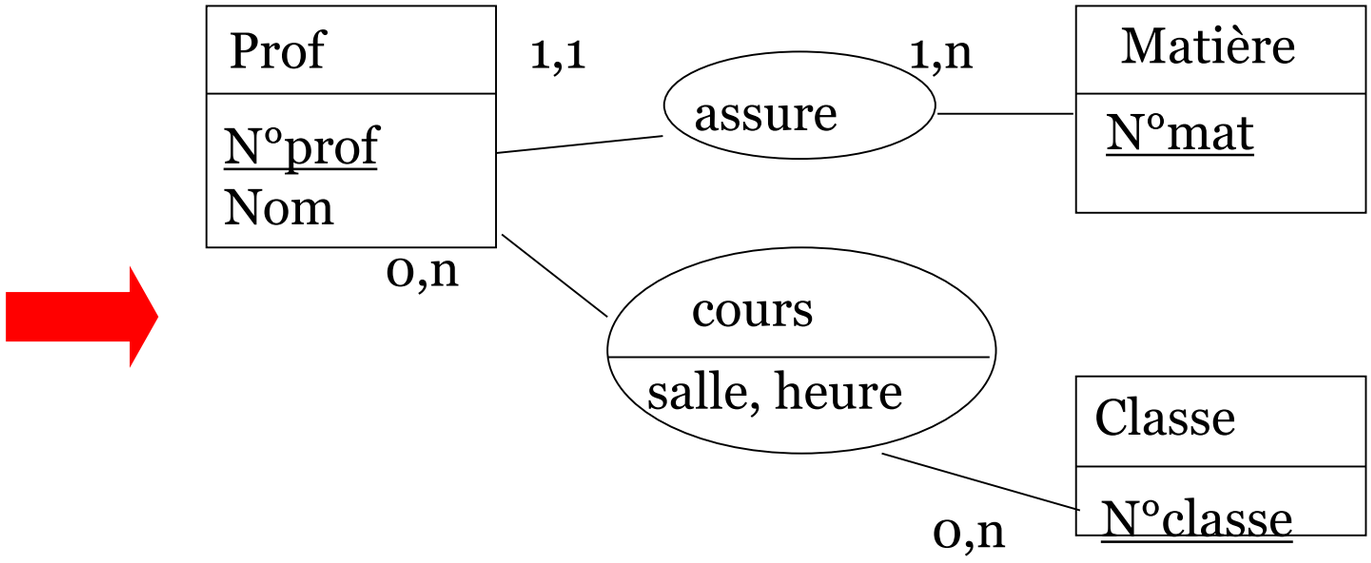
3.c La décomposition des associations n-aires

Il faut garder un minimum d'associations d'arité > 2.

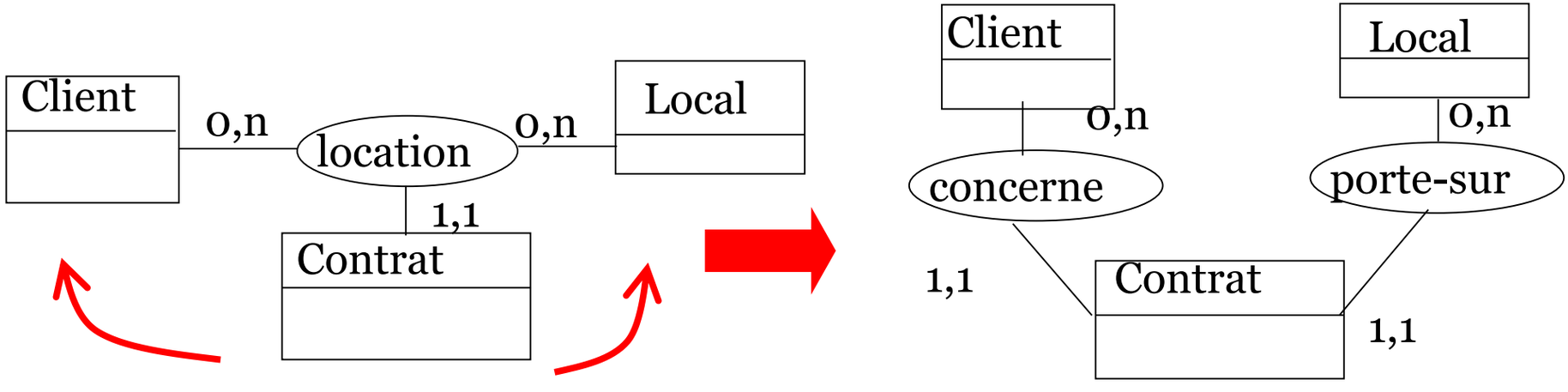
Si on observe une DF entre un sous-ensemble des entités d'une association, on peut la décomposer en deux associations (on parle aussi de 'contrainte d'intégrité fonctionnelle' ou CIF).



Une éventuelle DF *N°prof* → *N°mat* donne la décomposition :

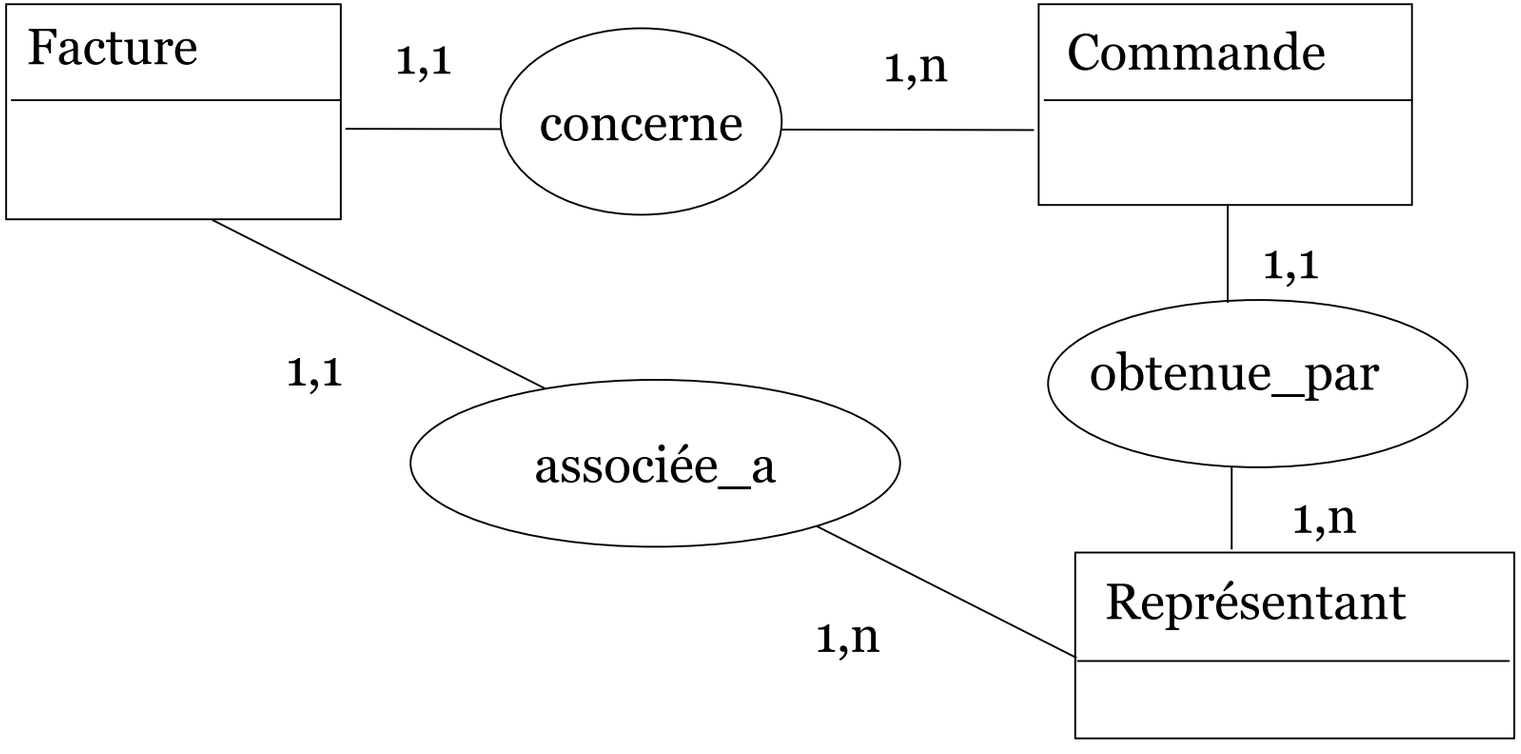


C'est le cas, quand une patte a une cardinalité 1,1.
Par exemple à 1 contrat est associé un client et un local :



3.d La suppression des associations transitives

Toute association pouvant être obtenue par transitivité de n autres associations peut être supprimée.



On supprime l'association *associée_a*, car elle peut être obtenue par transitivité sur les associations *concerne* et *obtenue_par*

c) Les Contraintes d'Intégrité

Les CI définissent des propriétés qui doivent être vérifiées par les données de la base.

1. Contraintes liées au schéma

1.a Contrainte d'identifiant

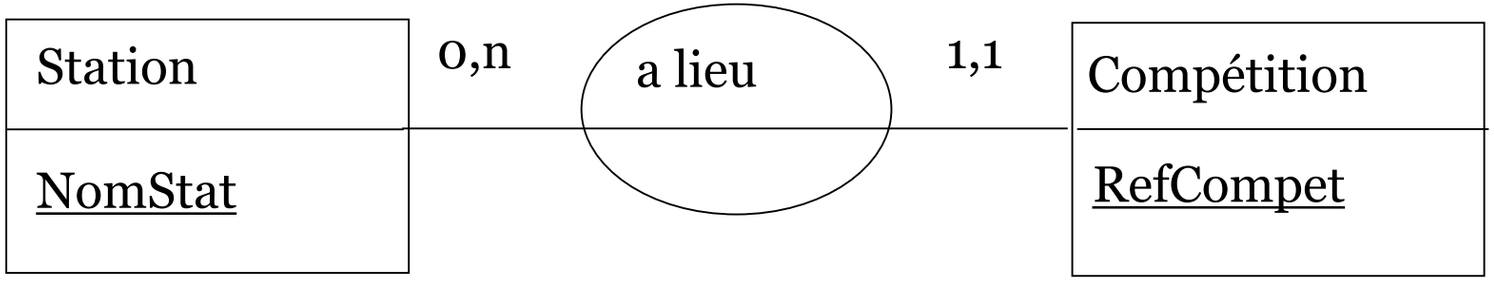
Les valeurs prises par l'identifiant sont uniques et toujours définies.

1.b Contraintes de cardinalité

Les cardinalités portées par les entités membres d'association imposent des nombres minis et maxis d'occurrence dans l'association.

Une cardinalité mini de 1 rend l'existence d'une occurrence d'entité dépendante de l'existence d'une occurrence d'une autre entité.

MCD



Une compétition ne peut exister que si la station où elle se déroule existe

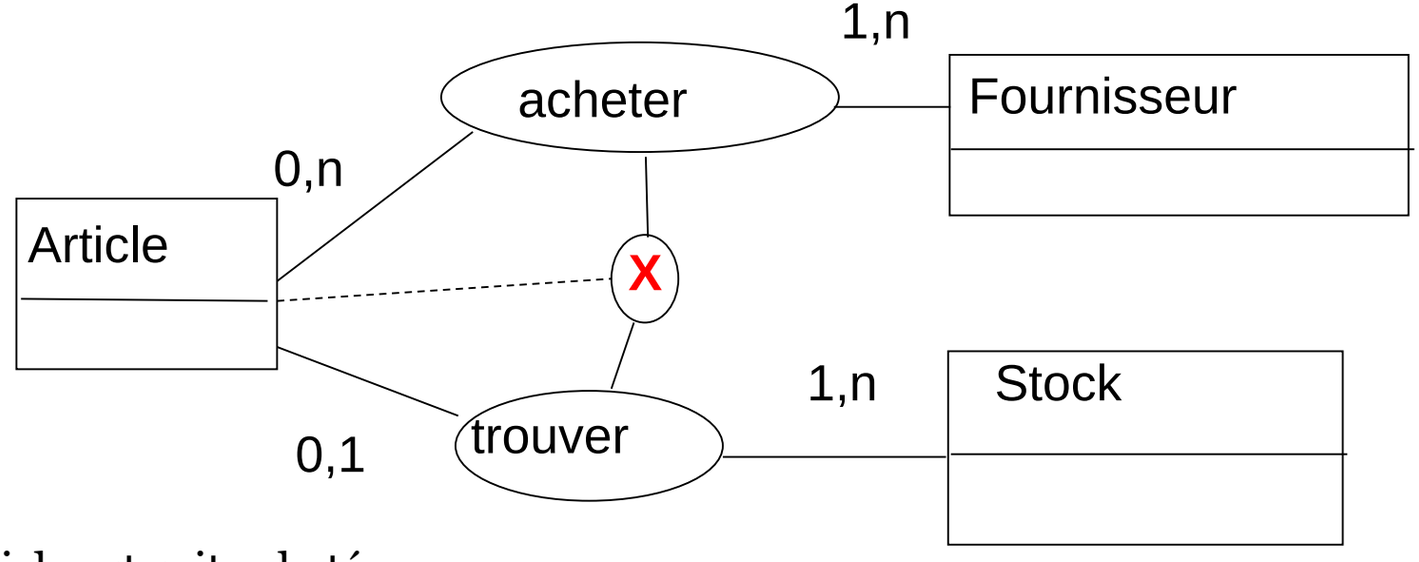
Une station peut exister de manière indépendante de toute compétition.

2. Contraintes additionnelles

Contraintes de participation des entités aux associations.

2.a Exclusivité de participation d'une entité à plusieurs associations

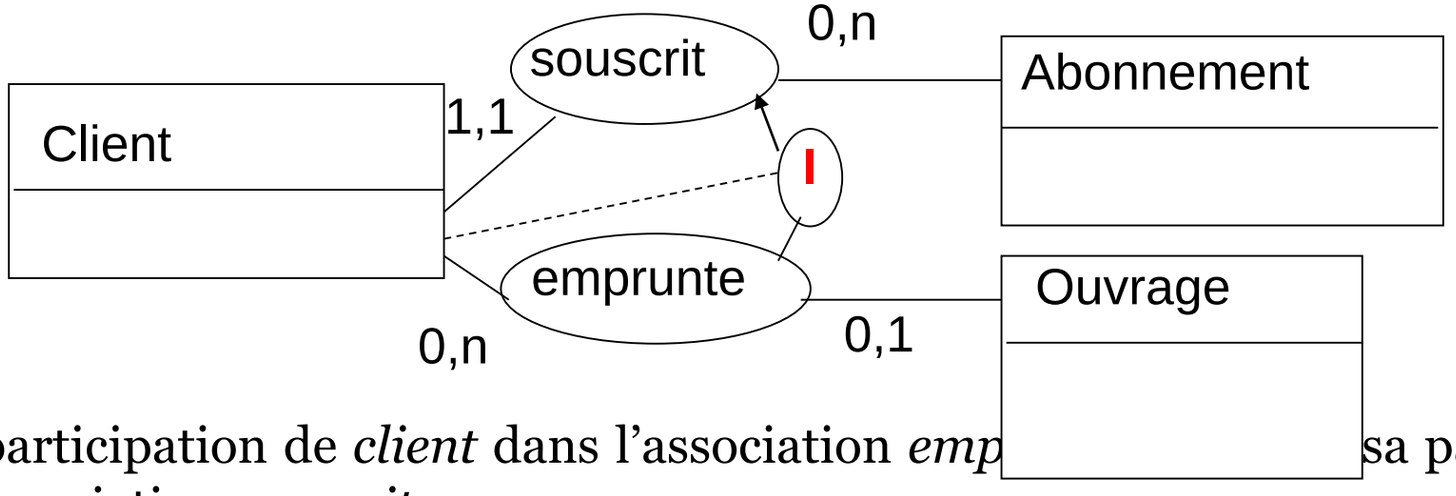
Si l'entité E participe à l'association A1, elle ne peut participer à l'association A2.



Un Article est soit acheté
auprès d'un fournisseur, soit figure dans le Stock

2.b Inclusion de participation d'une entité à plusieurs associations

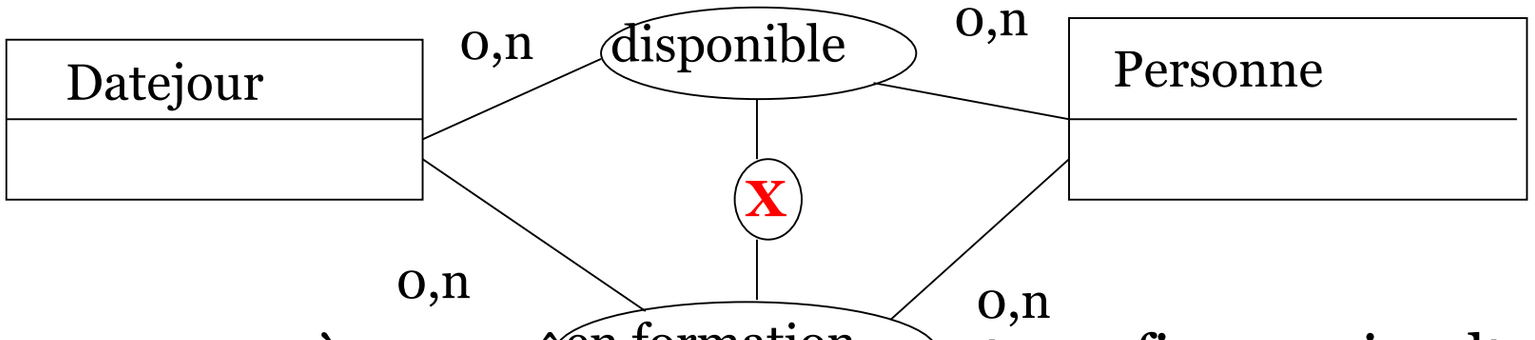
La participation d'une entité E à une association A1 implique sa participation à l'association A2.



La participation de *client* dans l'association *emp* sa participation à l'association *souscrit*.

2.c *Exclusion de participation entre associations*

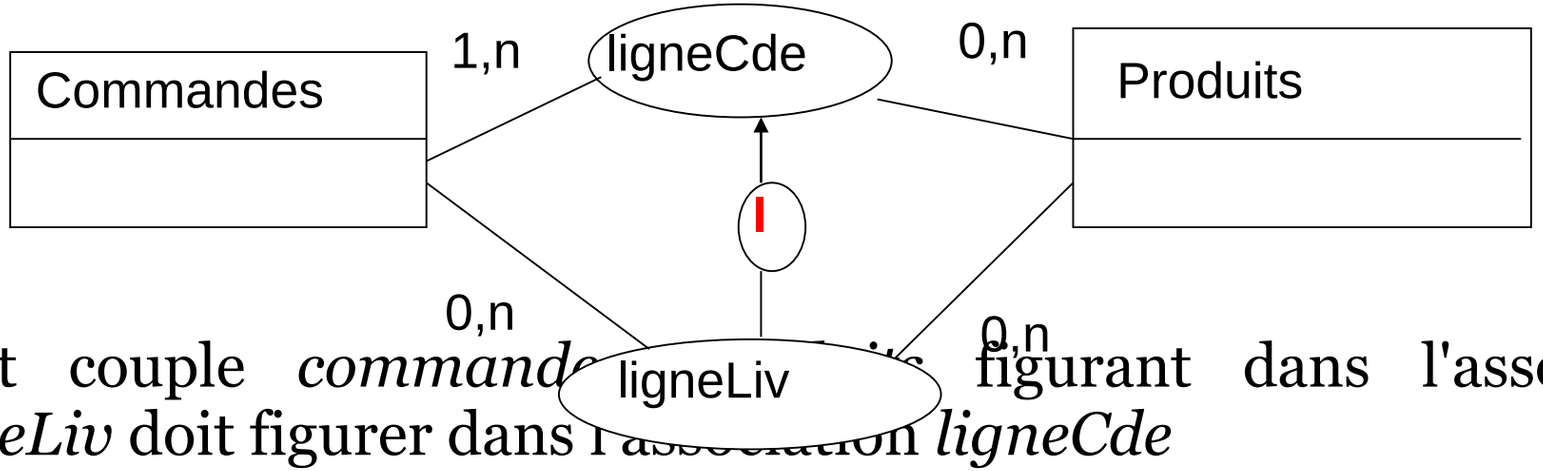
Il y a exclusion de participation entre associations si la participation des entités à l'association A1 exclut leur participation à l'association A2.



Une personne à une mê[illegible]eut pas figurer simultanément dans les deux associations: *disponible* et *en formation*.

2.d *Inclusion de participation entre associations*

Il y a inclusion de participation entre associations si la participation des entités à l'association A1 implique leur participation à l'association A2.



Tout couple *commande*, *produits* figurant dans l'association *ligneLiv* doit figurer dans l'association *ligneCde*

Le problème va être de vérifier toutes ces contraintes dans les programmes qui mettent à jour les données !

Règles de passage du MCD vers MLD

Le passage d'un Modèle Conceptuel de Données (MCD) vers un Modèle Logique de Données (MLD) implique l'application de plusieurs règles pour traduire les concepts abstraits du MCD en une structure de base de données plus concrète. Voici les règles clés à suivre lors de ce processus :

1.Entités et Attributs :

1. Chaque entité du MCD devient une table dans le MLD.
2. Les attributs de chaque entité deviennent les colonnes de la table correspondante.
3. Les attributs faisant partie de la clé primaire de l'entité devraient être marqués comme clés primaires de la table.

2.Relations :

1. Une relation 1:1 entre deux entités peut être traduite en ajoutant la clé primaire de l'une comme clé étrangère dans l'autre.
2. Une relation 1:N entre deux entités peut être traduite en plaçant la clé étrangère de l'entité "N" dans la table de l'entité "1".
3. Une relation N:M nécessite généralement une table de liaison (table de jonction) avec les clés primaires des deux entités comme clés étrangères.

Règles de passage du MCD vers MLD

1.Clés Primaires et Étrangères :

1. Les clés primaires des entités du MCD deviennent les clés primaires correspondantes dans les tables du MLD.
2. Les clés étrangères sont utilisées pour maintenir les relations entre les tables. Chaque clé étrangère dans une table fait référence à la clé primaire de l'entité associée.

2.Attributs Multi-valués :

1. Les attributs multivalués dans le MCD peuvent être traduits en créant de nouvelles tables pour ces attributs. Ces tables auront une clé étrangère faisant référence à la table parente.

3.Héritage :

1. Les hiérarchies d'héritage peuvent être modélisées de différentes manières, comme la création d'une table par sous-type ou la combinaison des attributs de tous les sous-types dans une seule table.

4.Normalisation :

1. Appliquez les règles de normalisation pour organiser les données de manière à éviter les redondances et les anomalies.

Règles de passage du MCD vers MLD

1.Types de Données :

1. Attribuez des types de données appropriés à chaque colonne en fonction de la nature des données qu'elle contiendra (entier, chaîne de caractères, date, etc.).

2.Contraintes :

1. Ajoutez des contraintes d'intégrité référentielle (clés étrangères) pour maintenir la cohérence entre les tables.

3.Indices :

1. Identifiez les colonnes souvent utilisées dans les requêtes et ajoutez des indices pour améliorer les performances de recherche.

4.Documentation :

1. Documentez soigneusement la signification de chaque table, de chaque colonne, des clés primaires et étrangères, et des contraintes pour une meilleure compréhension.

Chaque règle peut nécessiter des ajustements en fonction des spécificités du projet et des outils utilisés, mais ces directives générales vous aideront à passer du MCD au MLD de manière méthodique

Fin

aljia.bouzidi95@gmail.com